RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 – Fundamentals of Data Science Lab

Laboratory Record Note Book

Name : SASMITHA R

Year / Branch / Section : IInd year CSE 'H'

University Register No. : 211625070 1668

College Roll No. : 250701668

Semester : IIIrd

Academic Year : 2026 - 2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

NAME	SASMITHA R
ROLL NO	2116260701668
YEAR	IInd
SEC	CSE 'H'

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: SASMITHA R

Academic Year: 2026-2027 **Semester:** III rd **Branch:** CSE

Register No. 2116250701668

Certified that this is the bonafide record of work done by the above student in the FUNDAMENTALS OF DATA SCIENCE CS2333H ***Laboratory during the academic year 2026 – 2027.***

02/11/2026

Signature of Faculty in-charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

INDEX

Sl.NO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	
3.a	Conduct an experiment to show data visualization using line plot	19/08/26	
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26	
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26	
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26	
5	Data Collection and Initial Exploration	16/09/26	
6	Time Series Analysis of Website Traffic	16/09/26	

7	Random Sampling and Sampling Distribution	23/09/26	
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	
19	Experiment to understand Logistic Regression for a given data set.	07/10/26	
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	

In [27]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [12]:
```
df=pd.read_csv('sales_data - Sheet1.csv')
print(df.head())
```

```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
```

In [17]:
```
print(df.isnull().sum())
```

```
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
```

In [18]:
```
df['Sales']=df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
```

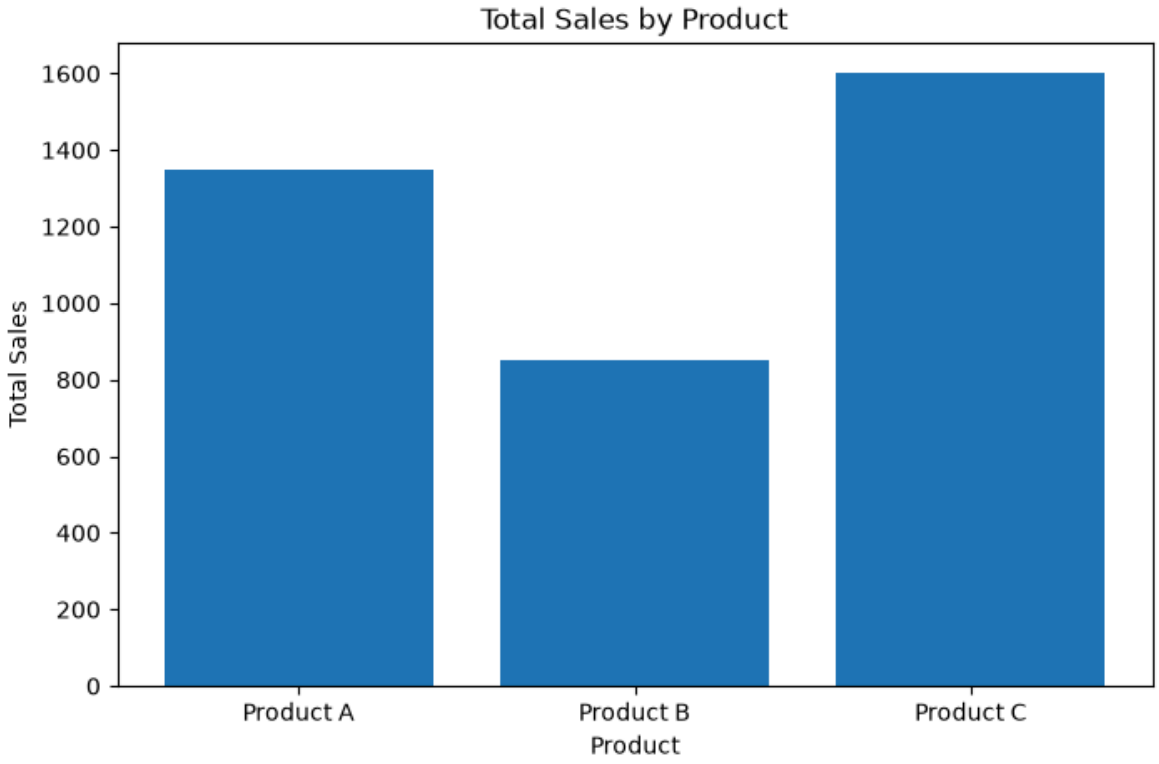
In [19]:
```
print(df.describe())
```

```
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
```

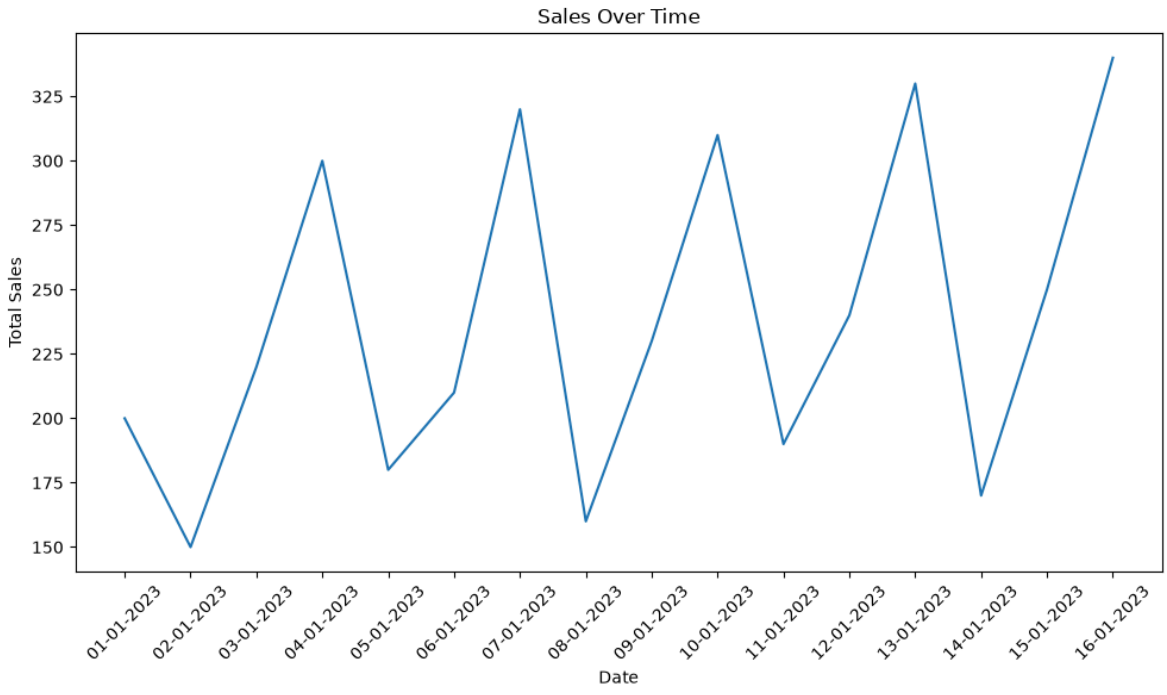
In [20]:
```
product_summary=df.groupby('Product').agg({'Sales': 'sum','Quantity': 'sum'}).re
print(product_summary)
```

```
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```

In [21]:
```
plt.figure(figsize=(8,5))
plt.bar(product_summary['Product'],product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
```



In [33]:
```
df['Data']=pd.to_datetime(df['Date'],format='%d-%m-%Y')
sales_over_time=df.groupby('Date').agg({'Sales':'sum'}).reset_index()
plt.figure(figsize=(10,6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [34]:
```
pivot_table=df.pivot_table(
    values='Sales',
    index='Region',
```

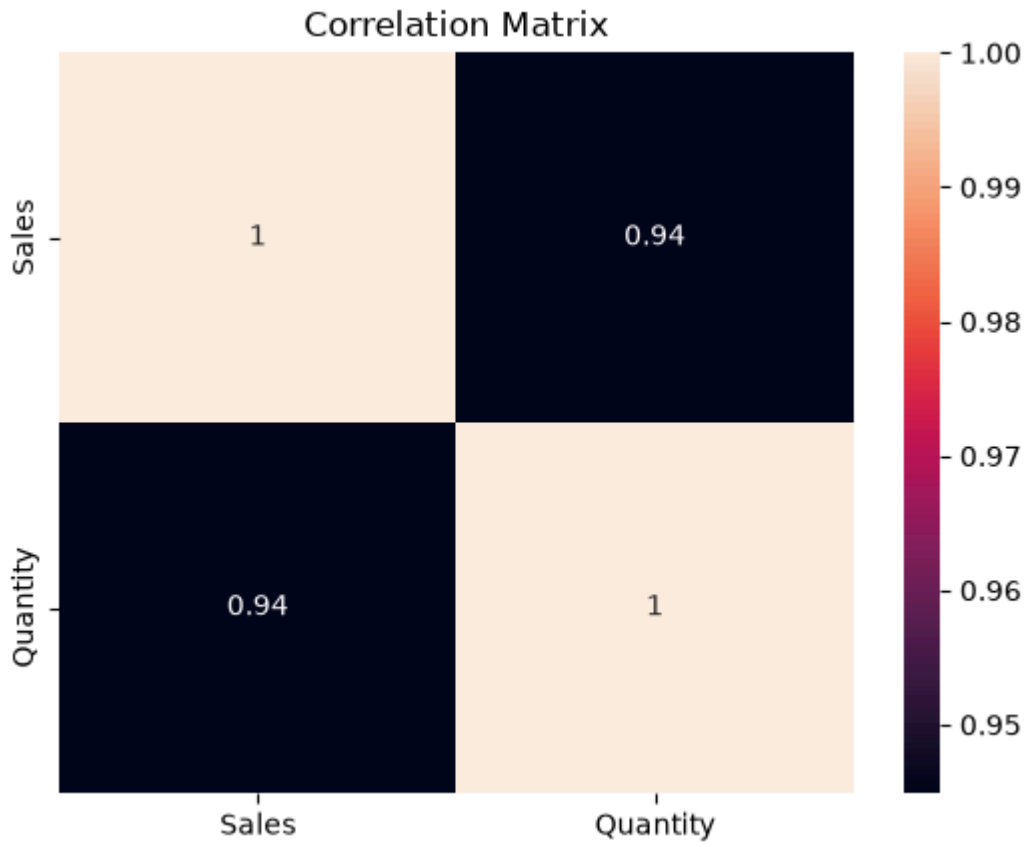
```
    columns='Product',
    aggfunc=np.sum,
    fill_value=0)
print(pivot_table)
```

```
Product  Product A  Product B  Product C
Region
East             0          0       1600
North         1350          0          0
South            0        480          0
West             0        370          0
```

In [37]:
```
correlation=df[['Sales','Quantity']].corr()
print(correlation)
```

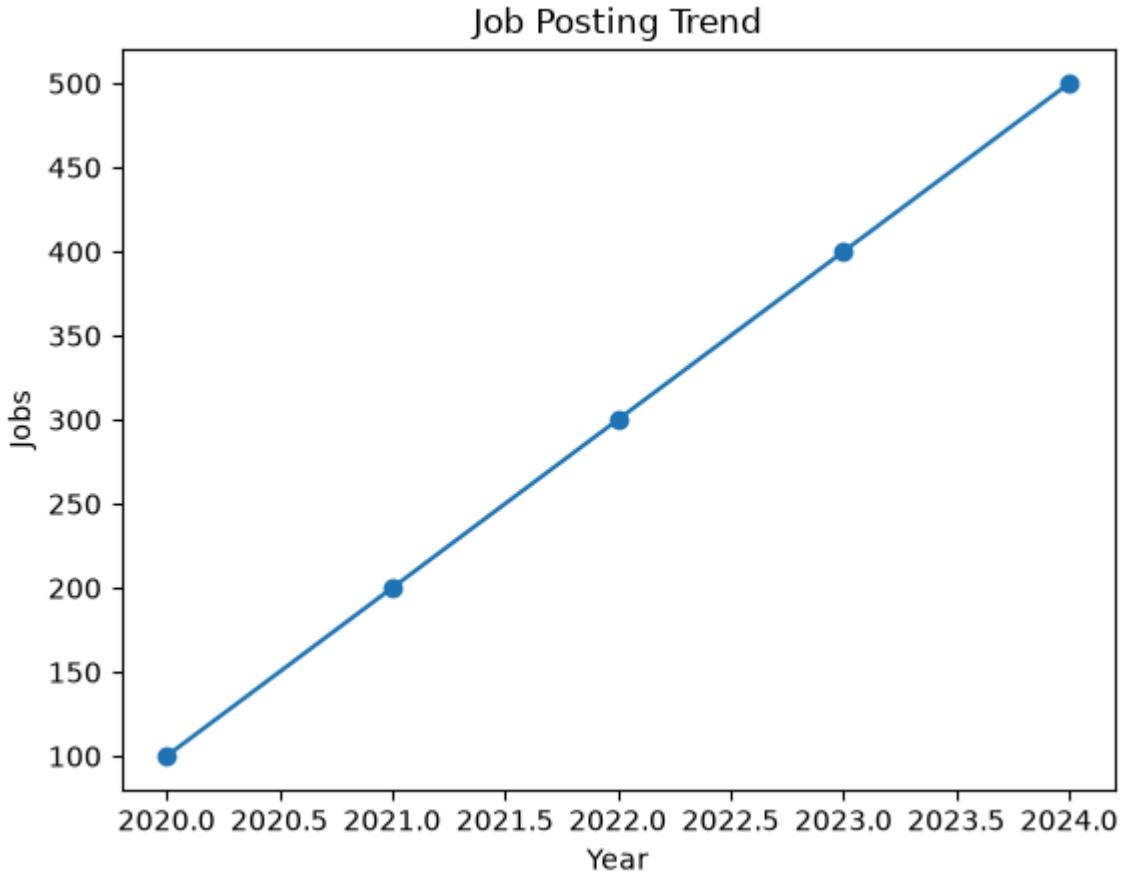
```
              Sales  Quantity
Sales     1.000000  0.944922
Quantity  0.944922  1.000000
```

In [38]:
```
sns.heatmap(correlation, annot=True)
plt.title('Correlation Matrix')
plt.show()
```

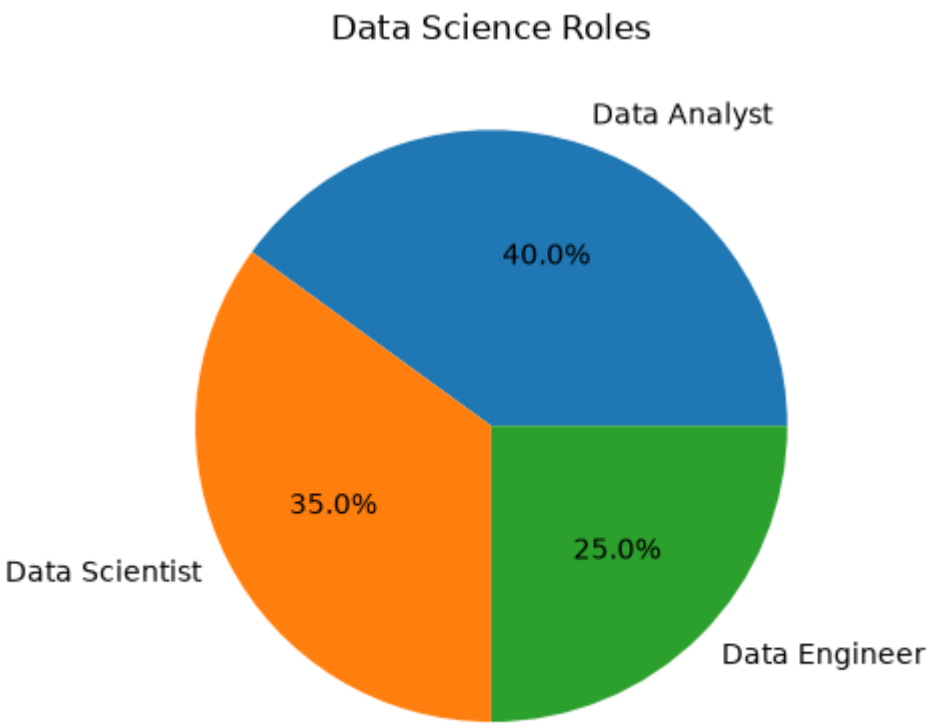


In [ ]:

In [8]:
```
import pandas as pd
import matplotlib.pyplot as plt
df=pd.DataFrame({
    "Year" :
    [2020,2021,2022,2023,2024],
    "Jobs":
    [100,200,300,400,500]})
plt.plot(df["Year"], df["Jobs"],marker="o")
plt.title("Job Posting Trend")
plt.xlabel("Year")
plt.ylabel("Jobs")
plt.show()
```



In [14]:
```
import pandas as pd
import matplotlib.pyplot as plt
df=pd.DataFrame({
    "Role":["Data Analyst","Data Scientist","Data Engineer"],
    "Count":[40,35,25]})
plt.pie(df["Count"],
labels=df["Role"], autopct="%1.1f%%")
plt.title("Data Science Roles")
plt.show()
```



In [16]:
```
import pandas as pd
df=pd.DataFrame({"Name":
["Asha","Rahul"],"Marks":[90,85]})
print("Structured Data")
print(df)
semi={"Name":"Asha","Marks":90}
print("\nSemi-Structured Data")
print(semi)
text="Data Science is interesting."
print("\nUnstructured Data")
print(text)
```

```
Structured Data
    Name  Marks
0   Asha     90
1  Rahul     85

Semi-Structured Data
{'Name': 'Asha', 'Marks': 90}

Unstructured Data
Data Science is interesting.
```

In [18]:
```
from cryptography.fernet import Fernet
key=Fernet.generate_key()
f=Fernet(key)
msg="Hello"
enc=f.encrypt(msg.encode())
print(enc)
dec=f.decrypt(enc)
print(dec.decode())
```

```
b'gAAAAABqciD-yh-glWqBeudLZOdHY39gfsUKfbllteNKz4bMznFybQHgdcbHH8MnrQSsS9hKhQ2X5JZ
pQpqc5gkXF8BHZTG-tQ=='
Hello
```

In [ ]:

In [26]:
```
import numpy as np
import pandas as pd
df=pd.read_csv("Book1.csv")
```

In [27]:
```
df.duplicated()
```

Out[27]:
```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool
```

In [28]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     str
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     str
 4   FoodPreference   11 non-null     str
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  10 non-null     float64
 8   Age_Group.1      11 non-null     str
dtypes: float64(1), int64(4), str(4)
memory usage: 1.1 KB
```

In [29]:
```
df.drop_duplicates(inplace=True)
df
```

Out[29]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	101	18-22	4	Taj	Veg	1450	2	
1	102	23-27	5	Marriott	Non-Veg	2100	3	
2	103	28-32	6	Hyatt	Veg	1580	2	
3	104	18-22	-1	Marriott	Vegetarian	1320	2	
4	105	33+	3	Taj	Veg	980	2	
5	106	33+	4	Tajj	Non-Veg	1950	2	
6	107	33+	4	Hyatt	Vegetarian	1100	-1	
7	108	18-22	7	Marriott	Veg	3100	-10	
8	109	28-32	2	Taj	Non-Veg	3600	3	
10	110	23-27	5	Hyatt	non-Veg	4	87700	

In [30]:
```
len(df)
```

Out[30]:
```
10
```

In [31]:
```
index=np.array(list(range(0,len(df))))
df.set_index(index,inplace=True)
index
```

Out[31]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [32]:
```
df.drop(['Age_Group.1'],axis=1,inplace=True)
df
```

Out[32]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Estim
0	101	18-22	4	Taj	Veg	1450	2	
1	102	23-27	5	Marriott	Non-Veg	2100	3	
2	103	28-32	6	Hyatt	Veg	1580	2	
3	104	18-22	-1	Marriott	Vegetarian	1320	2	
4	105	33+	3	Taj	Veg	980	2	
5	106	33+	4	Tajj	Non-Veg	1950	2	
6	107	33+	4	Hyatt	Vegetarian	1100	-1	
7	108	18-22	7	Marriott	Veg	3100	-10	
8	109	28-32	2	Taj	Non-Veg	3600	3	
9	110	23-27	5	Hyatt	non-Veg	4	87700	

In [33]:
```
df.loc[df['Bill'] <0,'Bill']=np.nan
df
```

Out[33]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3	
2	103	28-32	6	Hyatt	Veg	1580.0	2	
3	104	18-22	-1	Marriott	Vegetarian	1320.0	2	
4	105	33+	3	Taj	Veg	980.0	2	
5	106	33+	4	Tajj	Non-Veg	1950.0	2	
6	107	33+	4	Hyatt	Vegetarian	1100.0	-1	
7	108	18-22	7	Marriott	Veg	3100.0	-10	
8	109	28-32	2	Taj	Non-Veg	3600.0	3	
9	110	23-27	5	Hyatt	non-Veg	4.0	87700	

In [34]:
```
df.loc[df['EstimatedSalary'] <0,'EstimatedSalary'] =np.nan
df
```

Out[34]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3	
2	103	28-32	6	Hyatt	Veg	1580.0	2	
3	104	18-22	-1	Marriott	Vegetarian	1320.0	2	
4	105	33+	3	Taj	Veg	980.0	2	
5	106	33+	4	Tajj	Non-Veg	1950.0	2	
6	107	33+	4	Hyatt	Vegetarian	1100.0	-1	
7	108	18-22	7	Marriott	Veg	3100.0	-10	
8	109	28-32	2	Taj	Non-Veg	3600.0	3	
9	110	23-27	5	Hyatt	non-Veg	4.0	87700	

In [35]:
```
df.loc[(df['NoOfPax'] <1)|(df['NoOfPax'] >20),'NoOfPax']=np.nan
df
```

Out[35]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2.0	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3.0	
2	103	28-32	6	Hyatt	Veg	1580.0	2.0	
3	104	18-22	-1	Marriott	Vegetarian	1320.0	2.0	
4	105	33+	3	Taj	Veg	980.0	2.0	
5	106	33+	4	Tajj	Non-Veg	1950.0	2.0	
6	107	33+	4	Hyatt	Vegetarian	1100.0	NaN	
7	108	18-22	7	Marriott	Veg	3100.0	NaN	
8	109	28-32	2	Taj	Non-Veg	3600.0	3.0	
9	110	23-27	5	Hyatt	non-Veg	4.0	NaN	

In [36]:
```
df.Age_Group.unique()
```

Out[36]:
```
<ArrowStringArray>
['18-22', '23-27', '28-32', '33+']
Length: 4, dtype: str
```

In [37]:
```
df.Hotel.unique()
```

Out[37]:
```
<ArrowStringArray>
['Taj', 'Marriott', 'Hyatt', 'Tajj']
Length: 4, dtype: str
```

In [42]:
```
df['Hotel']=df['Hotel'].replace('Tajj','Taj')
df
```

Out[42]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2.0	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3.0	
2	103	28-32	6	Hyatt	Veg	1580.0	2.0	
3	104	18-22	-1	Marriott	Vegetarian	1320.0	2.0	
4	105	33+	3	Taj	Veg	980.0	2.0	
5	106	33+	4	Taj	Non-Veg	1950.0	2.0	
6	107	33+	4	Hyatt	Vegetarian	1100.0	NaN	
7	108	18-22	7	Marriott	Veg	3100.0	NaN	
8	109	28-32	2	Taj	Non-Veg	3600.0	3.0	
9	110	23-27	5	Hyatt	non-Veg	4.0	NaN	

In [43]:
```
df.FoodPreference.unique()
```

Out[43]:
```
<ArrowStringArray>
['Veg', 'Non-Veg', 'Vegetarian', 'non-Veg']
Length: 4, dtype: str
```

In [45]:
```
df['FoodPreference']=df['FoodPreference'].replace(['Vegetarian','veg'] ,'Veg')
df
```

Out[45]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2.0	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3.0	
2	103	28-32	6	Hyatt	Veg	1580.0	2.0	
3	104	18-22	-1	Marriott	Veg	1320.0	2.0	
4	105	33+	3	Taj	Veg	980.0	2.0	
5	106	33+	4	Taj	Non-Veg	1950.0	2.0	
6	107	33+	4	Hyatt	Veg	1100.0	NaN	
7	108	18-22	7	Marriott	Veg	3100.0	NaN	
8	109	28-32	2	Taj	Non-Veg	3600.0	3.0	
9	110	23-27	5	Hyatt	non-Veg	4.0	NaN	

In [46]:
```
df['FoodPreference']=df['FoodPreference'].replace('non-Veg','Non-Veg')
df
```

Out[46]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2.0	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3.0	
2	103	28-32	6	Hyatt	Veg	1580.0	2.0	
3	104	18-22	-1	Marriott	Veg	1320.0	2.0	
4	105	33+	3	Taj	Veg	980.0	2.0	
5	106	33+	4	Taj	Non-Veg	1950.0	2.0	
6	107	33+	4	Hyatt	Veg	1100.0	NaN	
7	108	18-22	7	Marriott	Veg	3100.0	NaN	
8	109	28-32	2	Taj	Non-Veg	3600.0	3.0	
9	110	23-27	5	Hyatt	Non-Veg	4.0	NaN	

In [47]:
```
df['EstimatedSalary']=df['EstimatedSalary'].fillna(round(df['EstimatedSalary'].m
df
```

Out[47]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2.0	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3.0	
2	103	28-32	6	Hyatt	Veg	1580.0	2.0	
3	104	18-22	-1	Marriott	Veg	1320.0	2.0	
4	105	33+	3	Taj	Veg	980.0	2.0	
5	106	33+	4	Taj	Non-Veg	1950.0	2.0	
6	107	33+	4	Hyatt	Veg	1100.0	NaN	
7	108	18-22	7	Marriott	Veg	3100.0	NaN	
8	109	28-32	2	Taj	Non-Veg	3600.0	3.0	
9	110	23-27	5	Hyatt	Non-Veg	4.0	NaN	

In [48]:
```
df['NoOfPax'] =df['NoOfPax'].fillna(round(df['NoOfPax'].median()))
df
```

Out[48]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2.0	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3.0	
2	103	28-32	6	Hyatt	Veg	1580.0	2.0	
3	104	18-22	-1	Marriott	Veg	1320.0	2.0	
4	105	33+	3	Taj	Veg	980.0	2.0	
5	106	33+	4	Taj	Non-Veg	1950.0	2.0	
6	107	33+	4	Hyatt	Veg	1100.0	2.0	
7	108	18-22	7	Marriott	Veg	3100.0	2.0	
8	109	28-32	2	Taj	Non-Veg	3600.0	3.0	
9	110	23-27	5	Hyatt	Non-Veg	4.0	2.0	

In [49]:
```
df['Rating(1-5)']=df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()))
df
```

Out[49]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2.0	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3.0	
2	103	28-32	6	Hyatt	Veg	1580.0	2.0	
3	104	18-22	-1	Marriott	Veg	1320.0	2.0	
4	105	33+	3	Taj	Veg	980.0	2.0	
5	106	33+	4	Taj	Non-Veg	1950.0	2.0	
6	107	33+	4	Hyatt	Veg	1100.0	2.0	
7	108	18-22	7	Marriott	Veg	3100.0	2.0	
8	109	28-32	2	Taj	Non-Veg	3600.0	3.0	
9	110	23-27	5	Hyatt	Non-Veg	4.0	2.0	

In [50]:
```
df['Bill']=df['Bill'].fillna(round(df['Bill'].mean()))
df
```

Out[50]:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	Esti
0	101	18-22	4	Taj	Veg	1450.0	2.0	
1	102	23-27	5	Marriott	Non-Veg	2100.0	3.0	
2	103	28-32	6	Hyatt	Veg	1580.0	2.0	
3	104	18-22	-1	Marriott	Veg	1320.0	2.0	
4	105	33+	3	Taj	Veg	980.0	2.0	
5	106	33+	4	Taj	Non-Veg	1950.0	2.0	
6	107	33+	4	Hyatt	Veg	1100.0	2.0	
7	108	18-22	7	Marriott	Veg	3100.0	2.0	
8	109	28-32	2	Taj	Non-Veg	3600.0	3.0	
9	110	23-27	5	Hyatt	Non-Veg	4.0	2.0	

In [ ]:

In [2]:
```
import numpy as np
import pandas as pd

df=pd.read_csv("Book3.csv")

df
```

Out[2]:

	Student_ID	Department	Age	Marks	Passed
0	1	CSE	19.0	85.0	Yes
1	2	ECE	20.0	72.0	Yes
2	3	CSE	18.0	65.0	No
3	4	EEE	19.0	78.0	Yes
4	5	ECE	21.0	NaN	Yes
5	6	CSE	20.0	55.0	No
6	7	EEE	NaN	68.0	No
7	8	CSE	19.0	91.0	Yes
8	9	NaN	20.0	74.0	No
9	10	ECE	18.0	88.0	Yes

In [3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Student_ID  10 non-null     int64
 1   Department  9 non-null      str
 2   Age         9 non-null      float64
 3   Marks       9 non-null      float64
 4   Passed      10 non-null     str
dtypes: float64(2), int64(1), str(2)
memory usage: 587.0 bytes
```

In [4]:
```
df.Department.mode()
```

Out[4]:
```
0    CSE
Name: Department, dtype: str
```

In [5]:
```
df.Department.mode()[0]
```

Out[5]:
```
'CSE'
```

In [8]:
```
df['Department'] = df['Department'].fillna(df['Department'].mode()[0])

df['Age'] = df['Age'].fillna(df['Age'].median())

df['Marks'] = df['Marks'].fillna(round(df['Marks'].mean()))
df
```

Out[8]:

	Student_ID	Department	Age	Marks	Passed
0	1	CSE	19.0	85.0	Yes
1	2	ECE	20.0	72.0	Yes
2	3	CSE	18.0	65.0	No
3	4	EEE	19.0	78.0	Yes
4	5	ECE	21.0	75.0	Yes
5	6	CSE	20.0	55.0	No
6	7	EEE	19.0	68.0	No
7	8	CSE	19.0	91.0	Yes
8	9	CSE	20.0	74.0	No
9	10	ECE	18.0	88.0	Yes

In [9]:
```
pd.get_dummies(df.Department)
```

Out[9]:

	CSE	ECE	EEE
0	True	False	False
1	False	True	False
2	True	False	False
3	False	False	True
4	False	True	False
5	True	False	False
6	False	False	True
7	True	False	False
8	True	False	False
9	False	True	False

In [10]:
```
updated_dataset=pd.concat(
    [pd.get_dummies(df.Department),df.iloc[:,[0,2,3,4]]],
    axis=1
)

updated_dataset
```

Out[10]:

	CSE	ECE	EEE	Student_ID	Age	Marks	Passed
0	True	False	False	1	19.0	85.0	Yes
1	False	True	False	2	20.0	72.0	Yes
2	True	False	False	3	18.0	65.0	No
3	False	False	True	4	19.0	78.0	Yes
4	False	True	False	5	21.0	75.0	Yes
5	True	False	False	6	20.0	55.0	No
6	False	False	True	7	19.0	68.0	No
7	True	False	False	8	19.0	91.0	Yes
8	True	False	False	9	20.0	74.0	No
9	False	True	False	10	18.0	88.0	Yes

In [11]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype  
---  ------      --------------  -----  
 0   Student_ID  10 non-null     int64  
 1   Department  10 non-null     str    
 2   Age         10 non-null     float64
 3   Marks       10 non-null     float64
 4   Passed      10 non-null     str    
dtypes: float64(2), int64(1), str(2)
memory usage: 588.0 bytes
```

In [12]:
```
updated_dataset['Passed']=updated_dataset['Passed'].replace(['No','Yes'],[0,1])
updated_dataset
```

Out[12]:

	CSE	ECE	EEE	Student_ID	Age	Marks	Passed
0	True	False	False	1	19.0	85.0	1
1	False	True	False	2	20.0	72.0	1
2	True	False	False	3	18.0	65.0	0
3	False	False	True	4	19.0	78.0	1
4	False	True	False	5	21.0	75.0	1
5	True	False	False	6	20.0	55.0	0
6	False	False	True	7	19.0	68.0	0
7	True	False	False	8	19.0	91.0	1
8	True	False	False	9	20.0	74.0	0
9	False	True	False	10	18.0	88.0	1

In [ ]:

In [16]:
```
import numpy as np
array=np.random.randint(20,100,20)
array
```

Out[16]:
```
array([44, 52, 47, 47, 69, 74, 79, 84, 98, 25, 40, 97, 35, 72, 64, 77, 61,
       91, 77, 84], dtype=int32)
```

In [17]:
```
array.mean()
```

Out[17]:
```
np.float64(65.85)
```

In [18]:
```
np.percentile(array,25)
```

Out[18]:
```
np.float64(47.0)
```

In [19]:
```
np.percentile(array,50)
```

Out[19]:
```
np.float64(70.5)
```

In [20]:
```
np.percentile(array,75)
```

Out[20]:
```
np.float64(80.25)
```

In [21]:
```
np.percentile(array,100)
```

Out[21]:
```
np.float64(98.0)
```

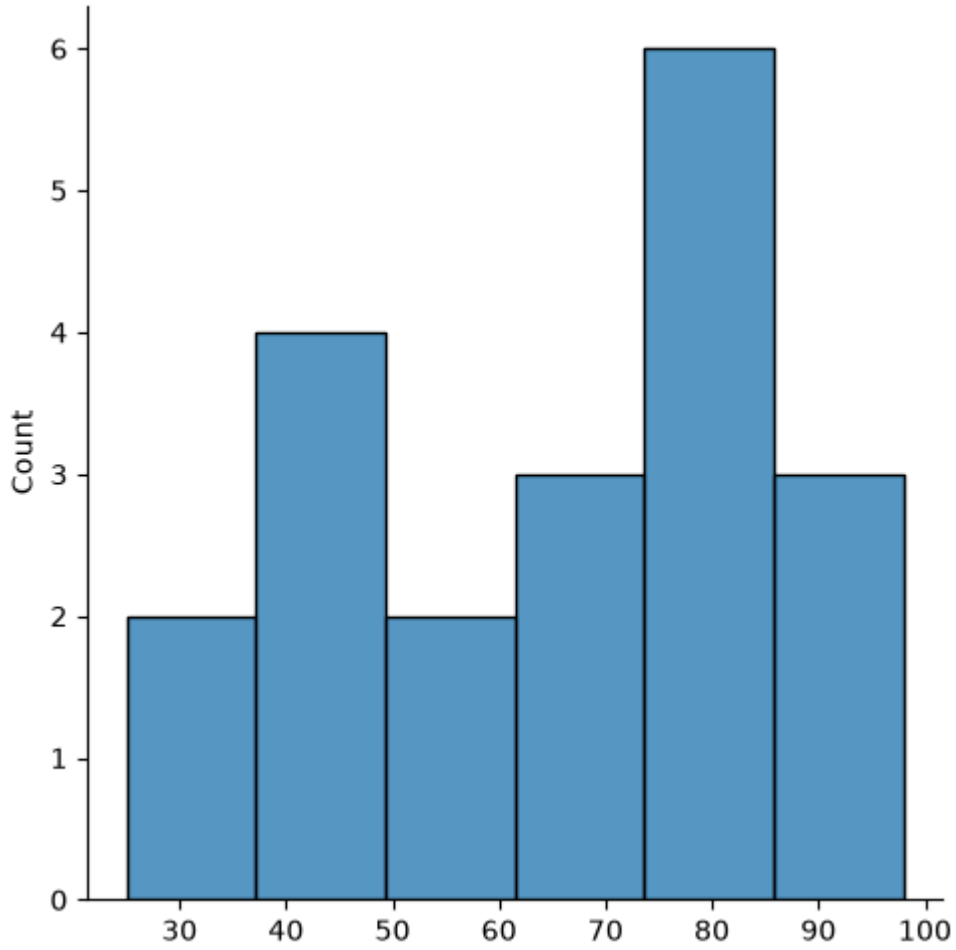
In [22]:
```
def outDetection(array):
    sorted(array)
    Q1,Q3=np.percentile(array,[25,75])
    IQR=Q3-Q1
    lr=Q1-(1.5*IQR)
    ur=Q3+(1.5*IQR)
    return lr,ur
```

In [23]:
```
lr,ur=outDetection(array)
lr,ur
```

Out[23]:
```
(np.float64(-2.875), np.float64(130.125))
```

In [24]:
```
import seaborn as sns
sns.displot(array)
```

Out[24]:
```
<seaborn.axisgrid.FacetGrid at 0x239d2b3f9d0>
```

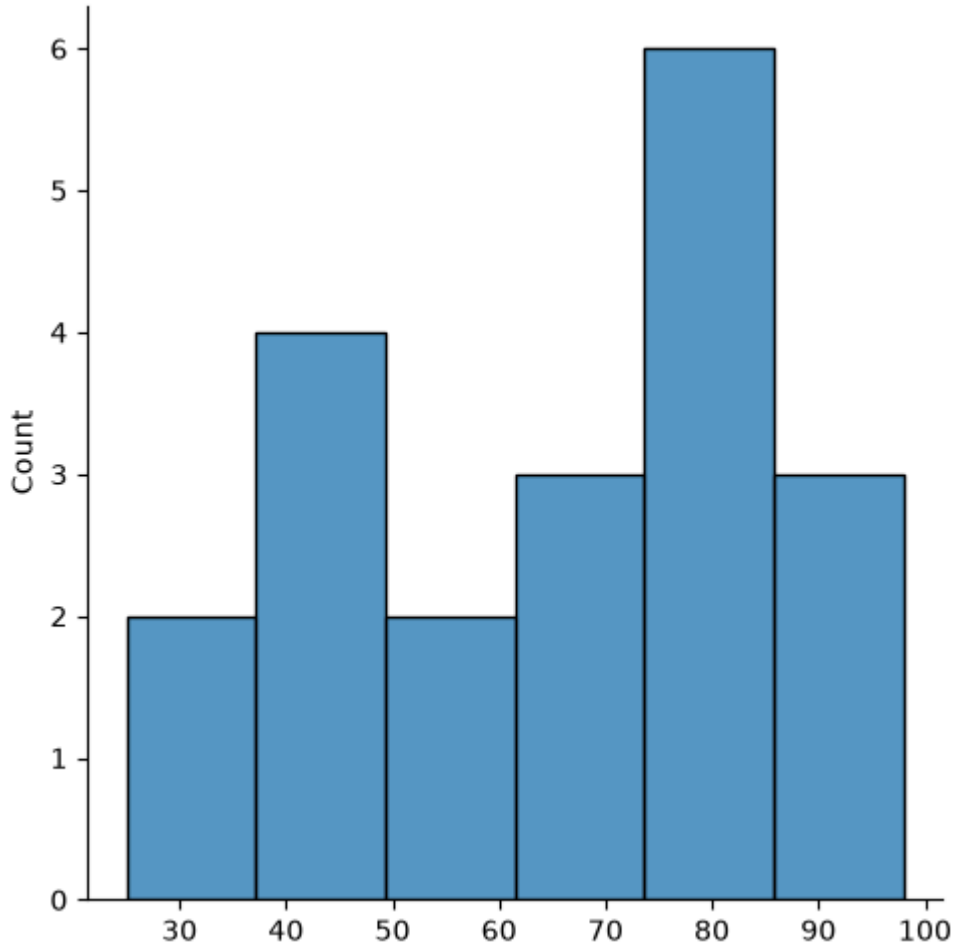


In [25]:
```
new_array=array[(array>lr) & (array < ur)]
new_array
```

Out[25]:
```
array([44, 52, 47, 47, 69, 74, 79, 84, 98, 25, 40, 97, 35, 72, 64, 77, 61,
       91, 77, 84], dtype=int32)
```

In [26]:
```
sns.displot(new_array)
```

Out[26]:
```
<seaborn.axisgrid.FacetGrid at 0x239d2bd6210>
```



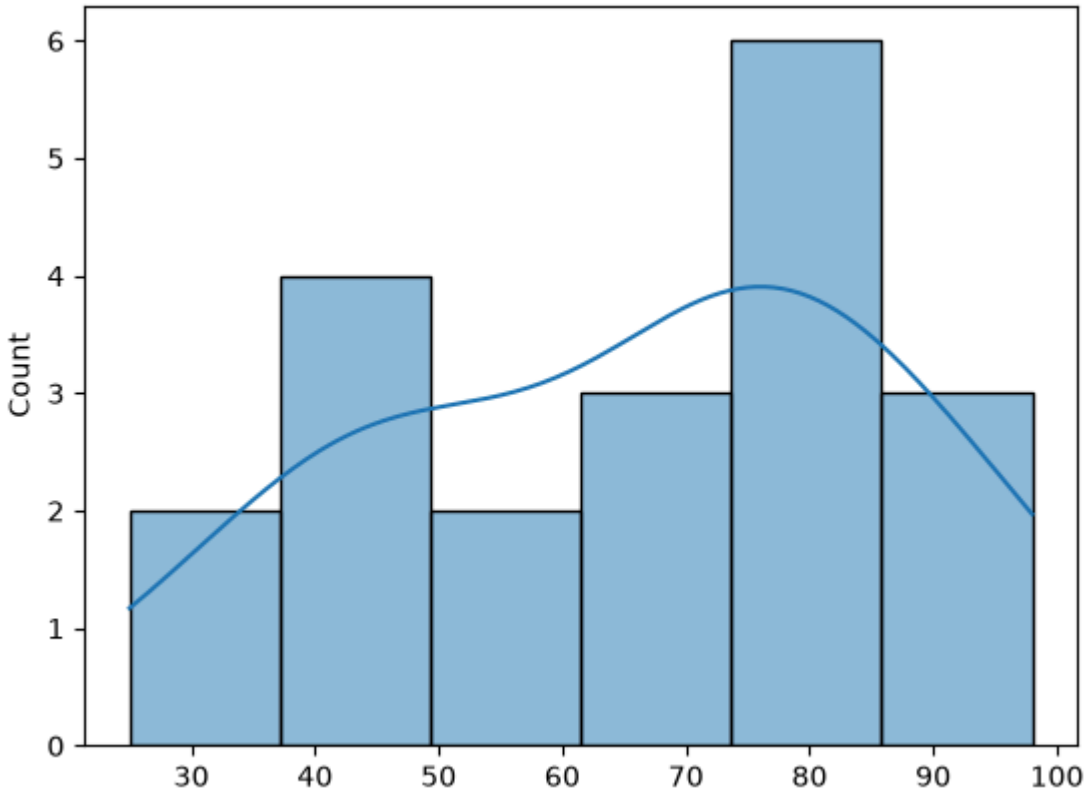
In [27]:
```
lr1,ur1=outDetection(new_array)
lr1,ur1
```

Out[27]:
```
(np.float64(-2.875), np.float64(130.125))
```

In [28]:
```
final_array=new_array[(new_array > lr1) &(new_array <ur1)]
final_array
```

Out[28]:
```
array([44, 52, 47, 47, 69, 74, 79, 84, 98, 25, 40, 97, 35, 72, 64, 77, 61,
       91, 77, 84], dtype=int32)
```

In [31]:
```
import seaborn as sns
import matplotlib.pyplot as plt
sns.histplot(final_array,kde=True)
plt.show()
```



In [ ]:

In [8]:
```
import numpy as np
import pandas as pd
```

In [9]:
```
data = {
    'Country': ['India', 'Japan', 'Brazil', 'Japan', 'Brazil',
                'India', 'Japan', 'Brazil', np.nan, 'India'],
    'Age': [25, 31, 28, 45, 39, 33, np.nan, 52, 41, 29],
    'Salary': [35000, 62000, 48000, 75000, np.nan,
               54000, 68000, 92000, 71000, 45000],
    'Purchased': ['Yes', 'No', 'Yes', 'No', 'Yes',
                  'No', 'Yes', 'No', 'Yes', 'No']
}

df = pd.DataFrame(data)
df
```

Out[9]:

	Country	Age	Salary	Purchased
0	India	25.0	35000.0	Yes
1	Japan	31.0	62000.0	No
2	Brazil	28.0	48000.0	Yes
3	Japan	45.0	75000.0	No
4	Brazil	39.0	NaN	Yes
5	India	33.0	54000.0	No
6	Japan	NaN	68000.0	Yes
7	Brazil	52.0	92000.0	No
8	NaN	41.0	71000.0	Yes
9	India	29.0	45000.0	No

In [10]:
```
df.head()
```

Out[10]:

	Country	Age	Salary	Purchased
0	India	25.0	35000.0	Yes
1	Japan	31.0	62000.0	No
2	Brazil	28.0	48000.0	Yes
3	Japan	45.0	75000.0	No
4	Brazil	39.0	NaN	Yes

In [11]:
```
df['Country'] = df['Country'].fillna(df['Country'].mode()[0])

df
```

Out[11]:

	Country	Age	Salary	Purchased
0	India	25.0	35000.0	Yes
1	Japan	31.0	62000.0	No
2	Brazil	28.0	48000.0	Yes
3	Japan	45.0	75000.0	No
4	Brazil	39.0	NaN	Yes
5	India	33.0	54000.0	No
6	Japan	NaN	68000.0	Yes
7	Brazil	52.0	92000.0	No
8	Brazil	41.0	71000.0	Yes
9	India	29.0	45000.0	No

In [12]:
```
features = df.iloc[:, :-1].values
features
```

Out[12]:
```
array([['India', 25.0, 35000.0],
       ['Japan', 31.0, 62000.0],
       ['Brazil', 28.0, 48000.0],
       ['Japan', 45.0, 75000.0],
       ['Brazil', 39.0, nan],
       ['India', 33.0, 54000.0],
       ['Japan', nan, 68000.0],
       ['Brazil', 52.0, 92000.0],
       ['Brazil', 41.0, 71000.0],
       ['India', 29.0, 45000.0]], dtype=object)
```

In [13]:
```
label = df.iloc[:, -1].values
label
```

Out[13]:
```
<ArrowStringArray>
['Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No']
Length: 10, dtype: str
```

In [14]:
```
from sklearn.impute import SimpleImputer
```

In [15]:
```
age = SimpleImputer(strategy="mean", missing_values=np.nan)
Salary = SimpleImputer(strategy="mean", missing_values=np.nan)
```

In [16]:
```
age.fit(features[:, [1]])
```

Out[16]:

▾ SimpleImputer

▸ Parameters

▸ Fitted attributes

▸ 1 feature

In [17]:
```
Salary.fit(features[:, [2]])
```

Out[17]:

▾ SimpleImputer

▸ Parameters

▸ Fitted attributes

▸ 1 feature

In [18]:
```
features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = Salary.transform(features[:, [2]])

features
```

Out[18]:
```
array([['India', 25.0, 35000.0],
       ['Japan', 31.0, 62000.0],
       ['Brazil', 28.0, 48000.0],
       ['Japan', 45.0, 75000.0],
       ['Brazil', 39.0, 61111.11111111111],
       ['India', 33.0, 54000.0],
       ['Japan', 35.888888888888886, 68000.0],
       ['Brazil', 52.0, 92000.0],
       ['Brazil', 41.0, 71000.0],
       ['India', 29.0, 45000.0]], dtype=object)
```

In [19]:
```
from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])

Country
```

Out[19]:
```
array([[0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [1., 0., 0.],
       [0., 1., 0.]])
```

In [20]:
```
final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)

final_set
```

Out[20]:
```
array([[0.0, 1.0, 0.0, 25.0, 35000.0],
       [0.0, 0.0, 1.0, 31.0, 62000.0],
       [1.0, 0.0, 0.0, 28.0, 48000.0],
       [0.0, 0.0, 1.0, 45.0, 75000.0],
       [1.0, 0.0, 0.0, 39.0, 61111.11111111111],
       [0.0, 1.0, 0.0, 33.0, 54000.0],
       [0.0, 0.0, 1.0, 35.888888888888886, 68000.0],
       [1.0, 0.0, 0.0, 52.0, 92000.0],
       [1.0, 0.0, 0.0, 41.0, 71000.0],
       [0.0, 1.0, 0.0, 29.0, 45000.0]], dtype=object)
```

In [21]:
```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc.fit(final_set)

feat_standard_scaler = sc.transform(final_set)

feat_standard_scaler
```

Out[21]:
```
array([[-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -1.36229417e+00, -1.66583735e+00],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -6.11642281e-01,  5.67093565e-02],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -9.86968227e-01, -8.36463009e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
         1.13987880e+00,  8.86083696e-01],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         3.89226906e-01,  0.00000000e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -3.61424984e-01, -4.53674852e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -8.88950413e-16,  4.39497513e-01],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         2.01563934e+00,  1.97065014e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         6.39444203e-01,  6.30891591e-01],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -8.61859578e-01, -1.02785709e+00]])
```

In [22]:
```
from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)

feat_minmax_scaler = mms.transform(final_set)

feat_minmax_scaler
```

Out[22]:
```
array([[0.        , 1.        , 0.        , 0.        , 0.        ],
       [0.        , 0.        , 1.        , 0.22222222, 0.47368421],
       [1.        , 0.        , 0.        , 0.11111111, 0.22807018],
       [0.        , 0.        , 1.        , 0.74074074, 0.70175439],
       [1.        , 0.        , 0.        , 0.51851852, 0.45808967],
       [0.        , 1.        , 0.        , 0.2962963 , 0.33333333],
       [0.        , 0.        , 1.        , 0.40329218, 0.57894737],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.59259259, 0.63157895],
       [0.        , 1.        , 0.        , 0.14814815, 0.1754386 ]])
```

In [ ]:

In [6]:
```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline
```

In [7]:
```
tips = sns.load_dataset('tips')
```

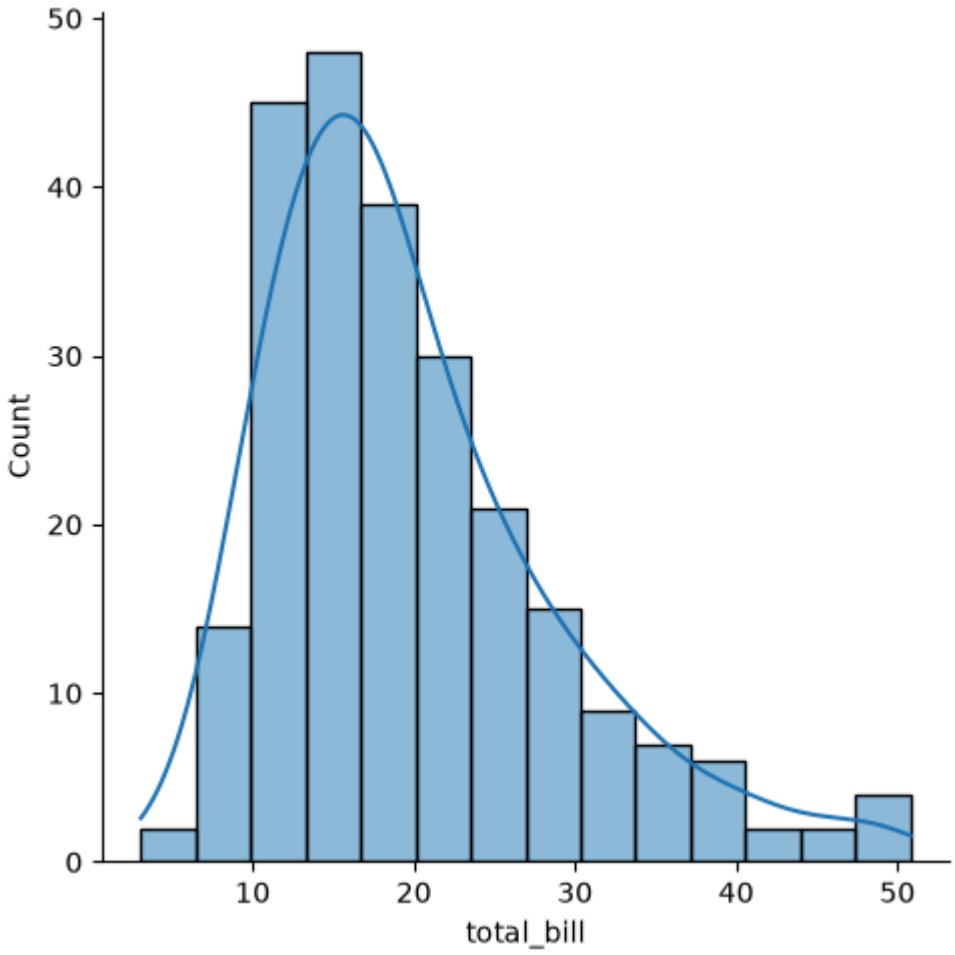
In [8]:
```
tips.head()
```

Out[8]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

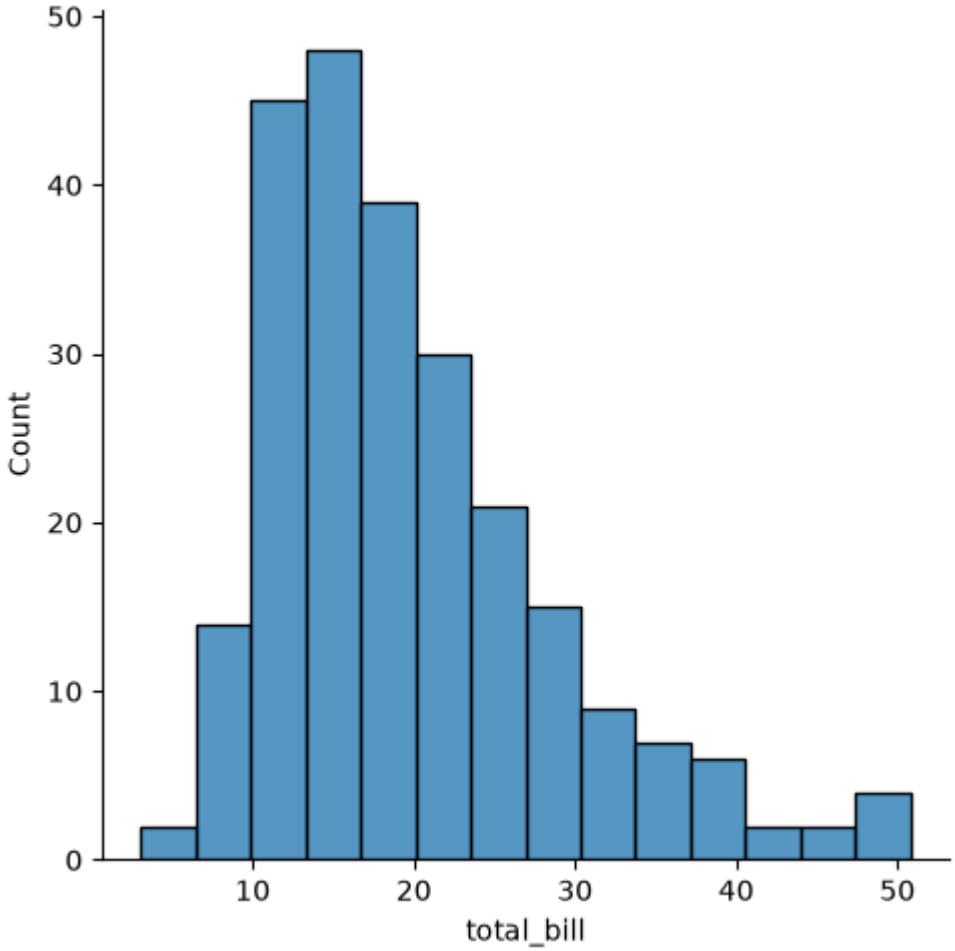
In [9]:
```
sns.displot(tips.total_bill, kde=True)
```

Out[9]:
```
<seaborn.axisgrid.FacetGrid at 0x11770340a50>
```



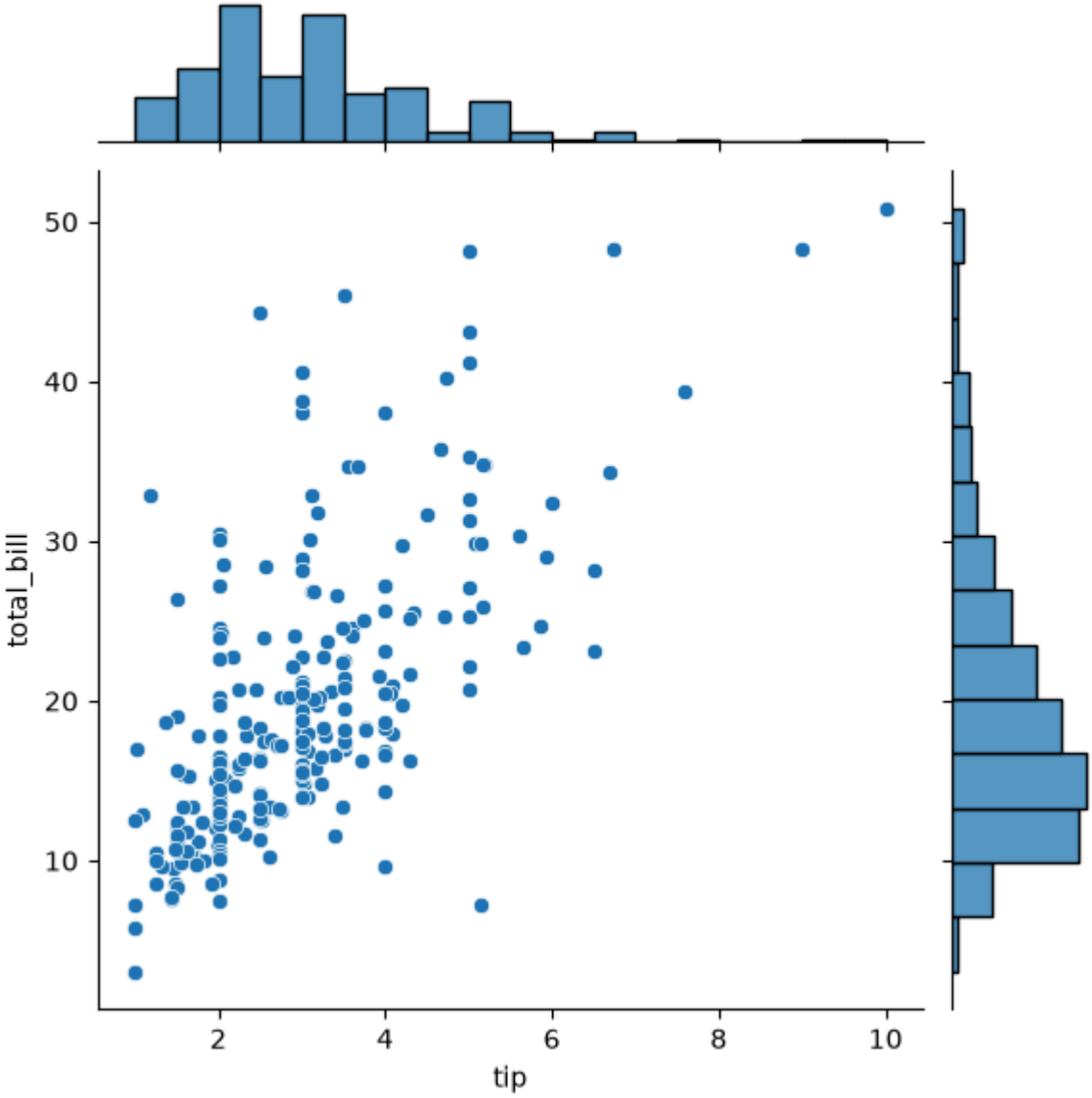
In [10]:
```
sns.displot(tips.total_bill, kde=False)
```

Out[10]: <seaborn.axisgrid.FacetGrid at 0x11770397b10>



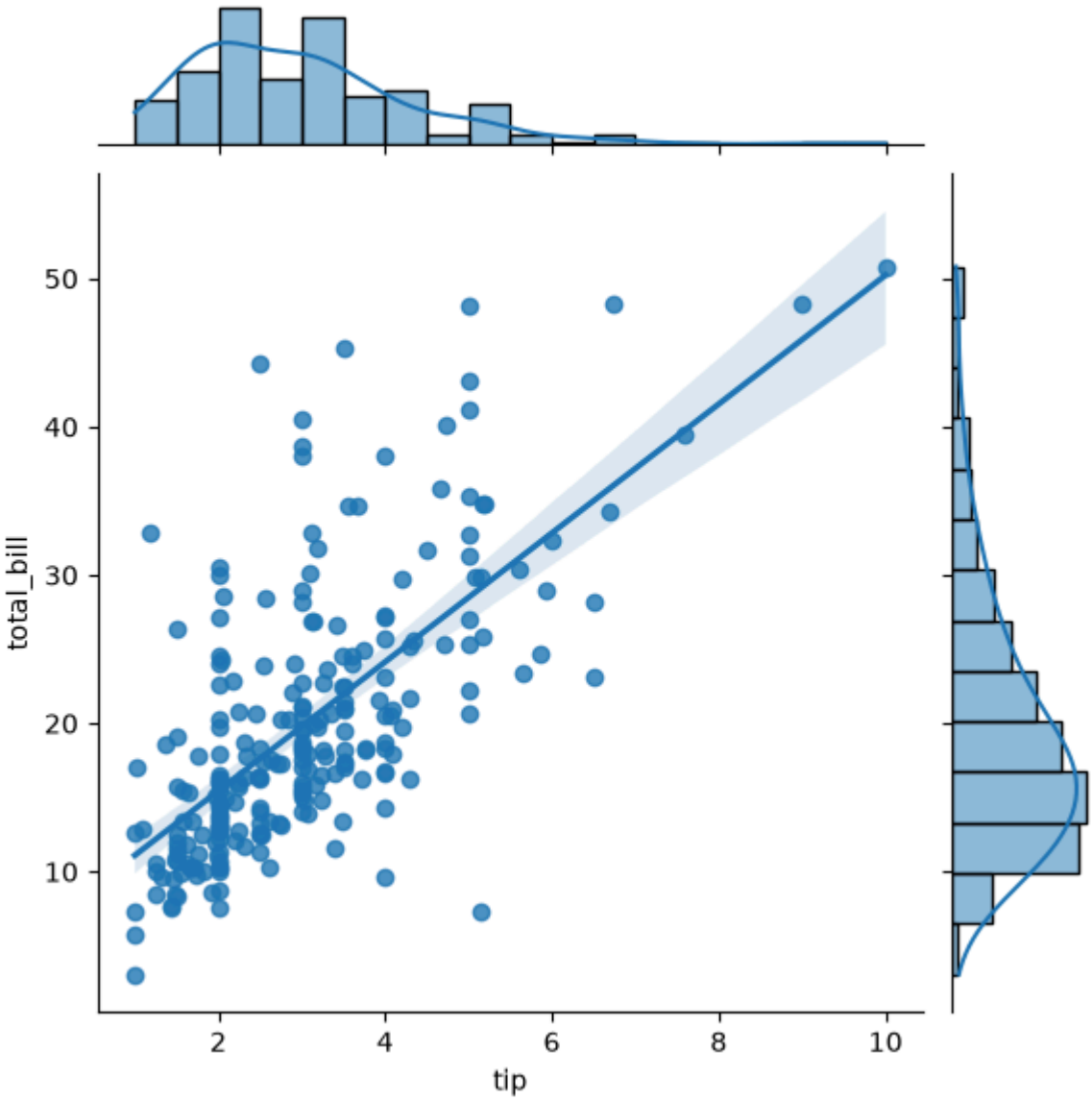
In [11]:
```
sns.jointplot(x=tips.tip, y=tips.total_bill)
```

Out[11]: <seaborn.axisgrid.JointGrid at 0x117713dee40>



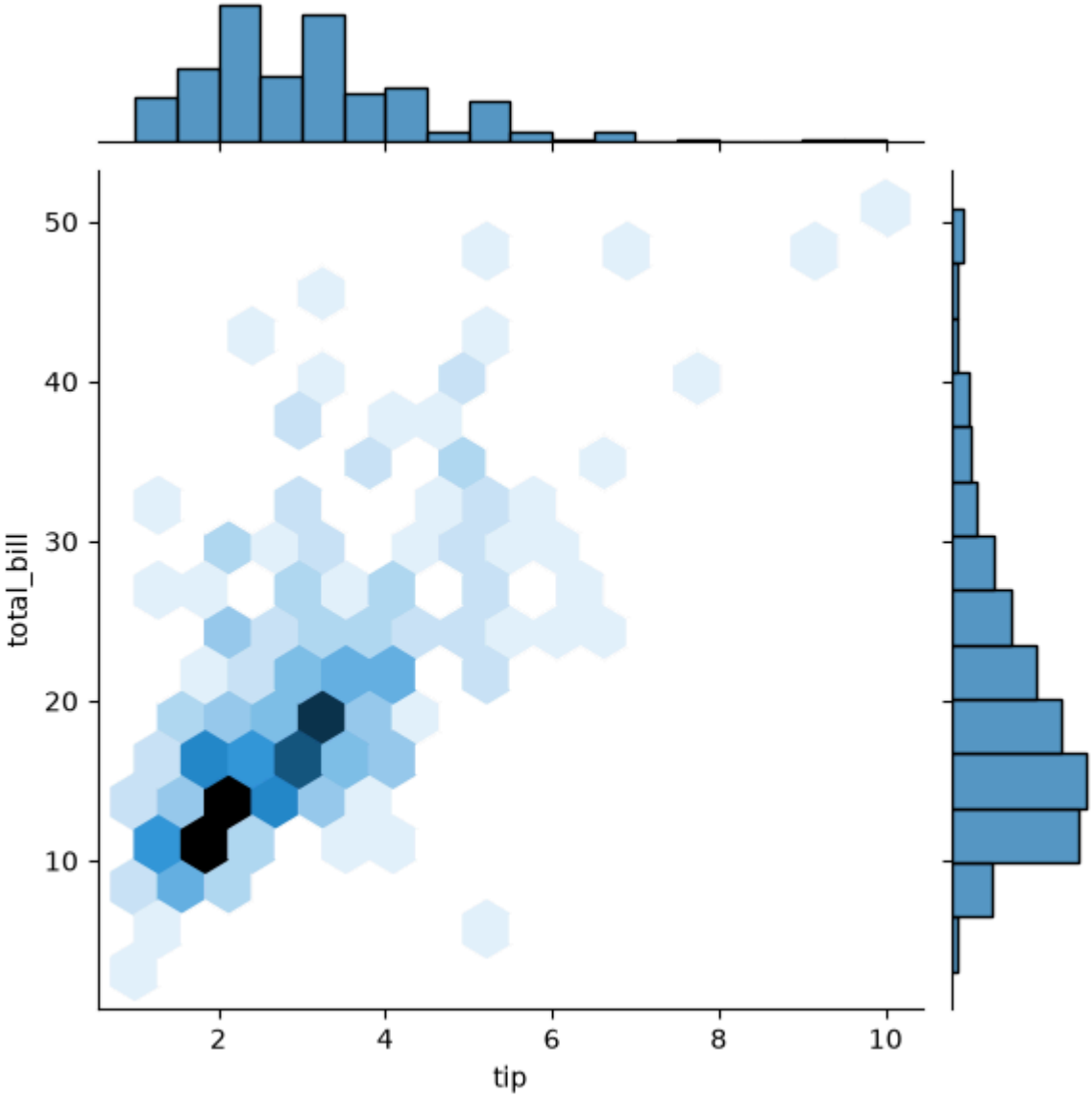
In [12]:
```
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
```

Out[12]:
```
<seaborn.axisgrid.JointGrid at 0x117715516d0>
```



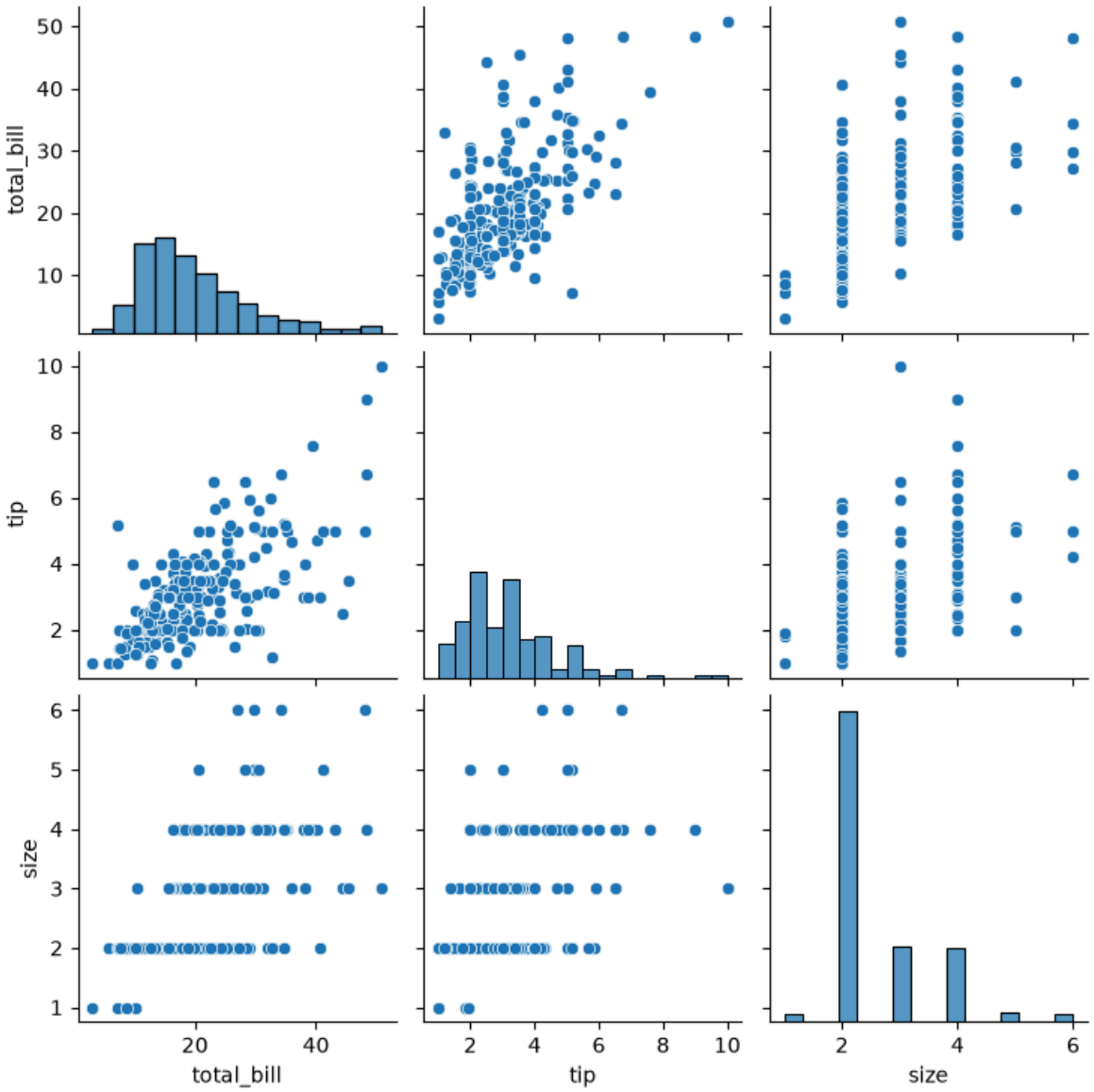
In [13]:
```
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")
```

Out[13]: <seaborn.axisgrid.JointGrid at 0x11771ad5090>



In [14]:
```
sns.pairplot(tips)
```

Out[14]:
```
<seaborn.axisgrid.PairGrid at 0x11771ea5e80>
```

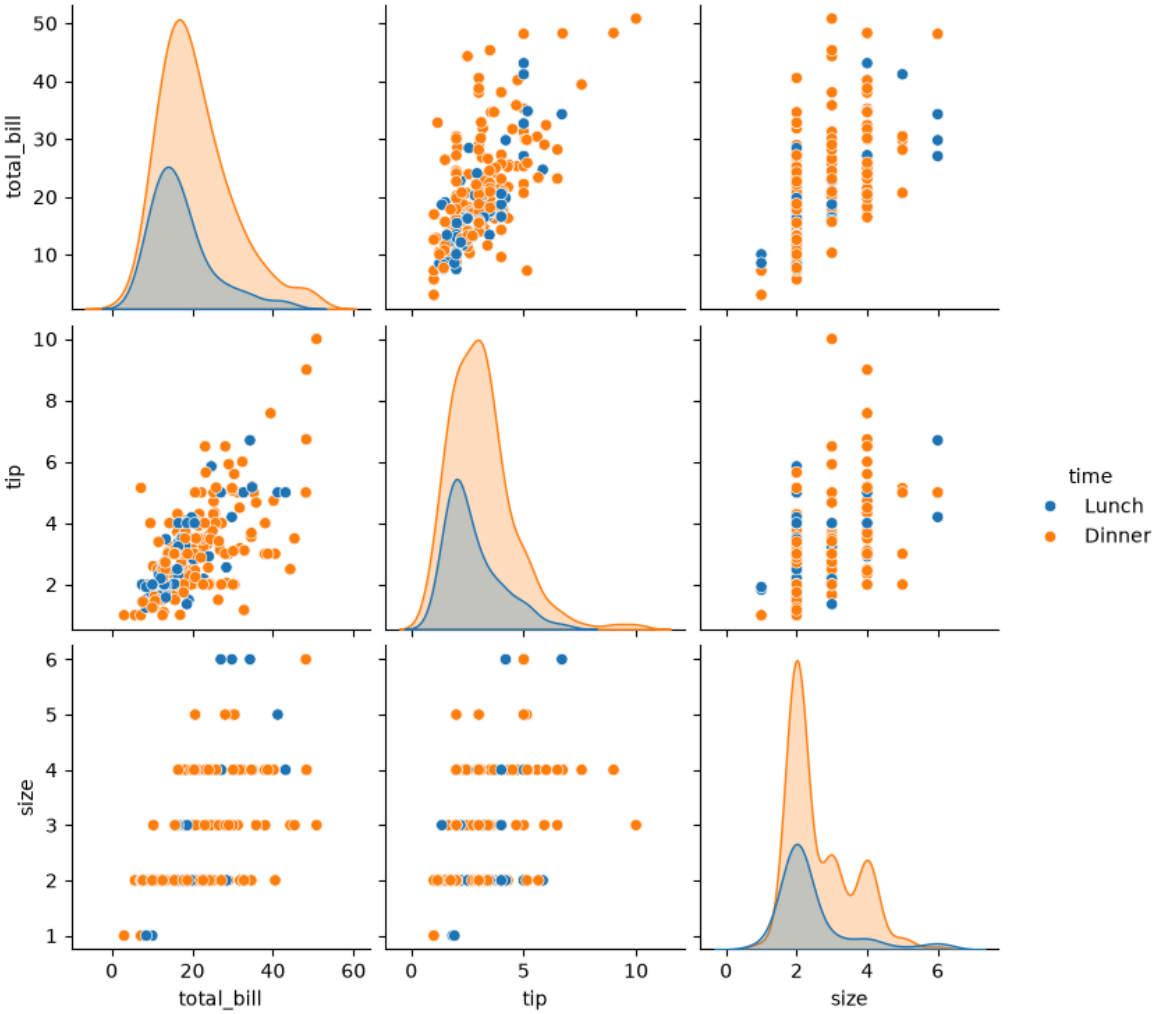


In [15]:
```
tips.time.value_counts()
```

Out[15]:
```
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

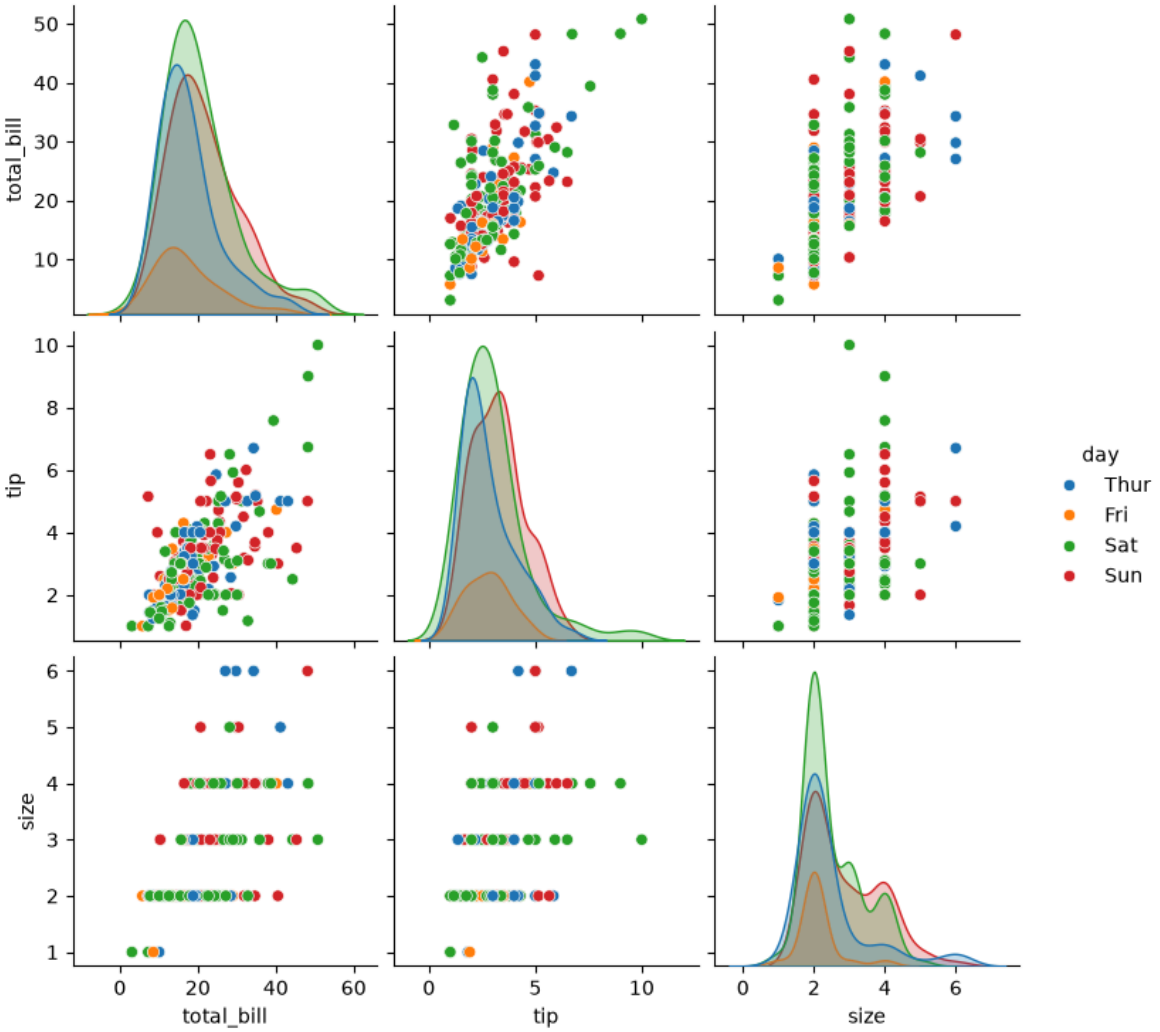
In [16]:
```
sns.pairplot(tips, hue='time')
```

Out[16]:
```
<seaborn.axisgrid.PairGrid at 0x11772701450>
```



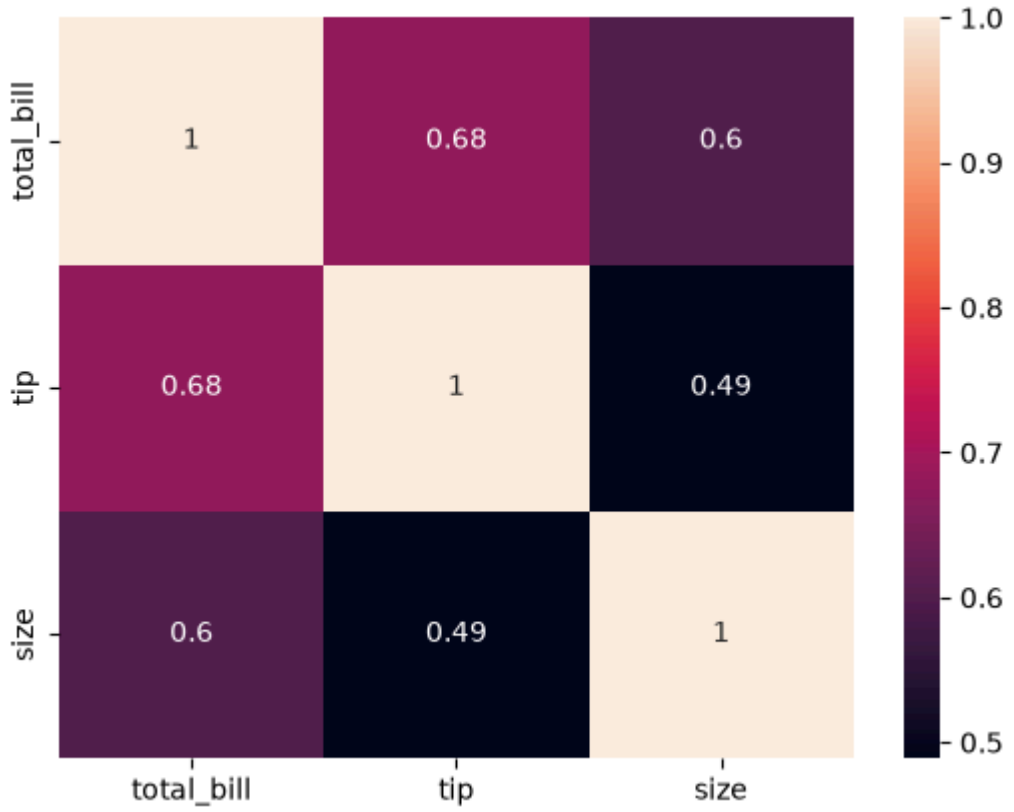
In [17]:
```
sns.pairplot(tips, hue='day')
```

Out[17]:
```
<seaborn.axisgrid.PairGrid at 0x117733e82d0>
```



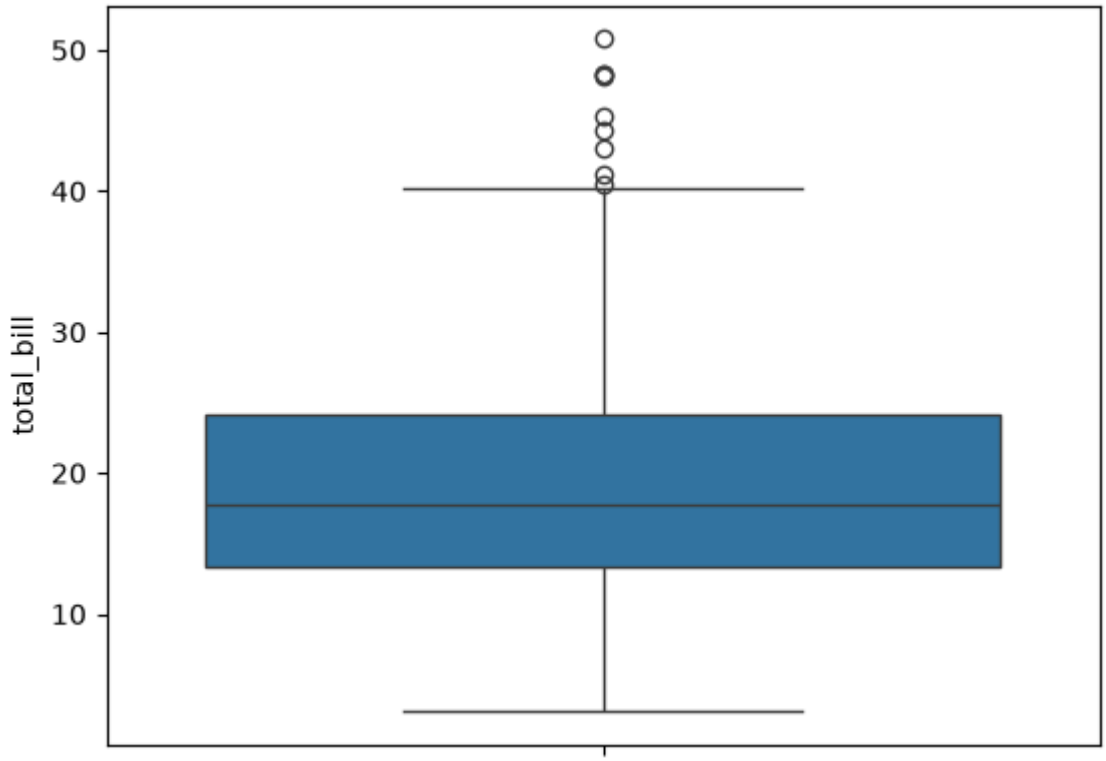
In [18]:
```
sns.heatmap(tips.corr(numeric_only=True), annot=True)
```

Out[18]: <Axes: >



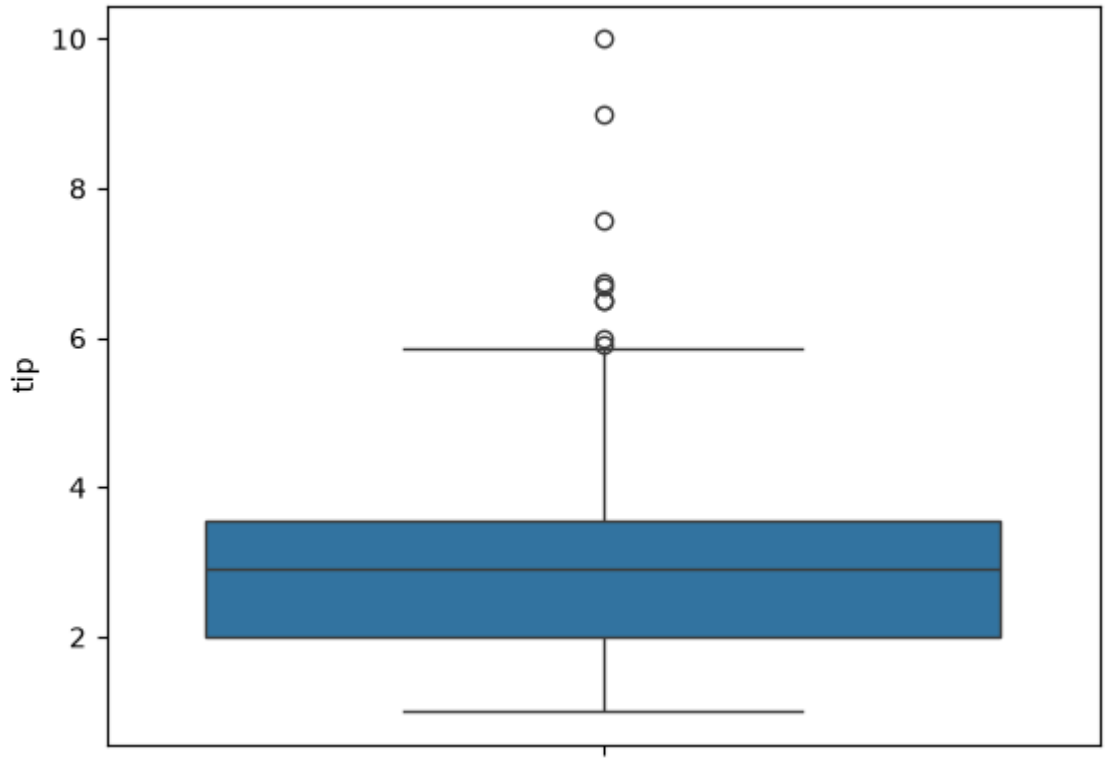
In [19]:
```
sns.boxplot(tips.total_bill)
```

Out[19]: <Axes: ylabel='total_bill'>



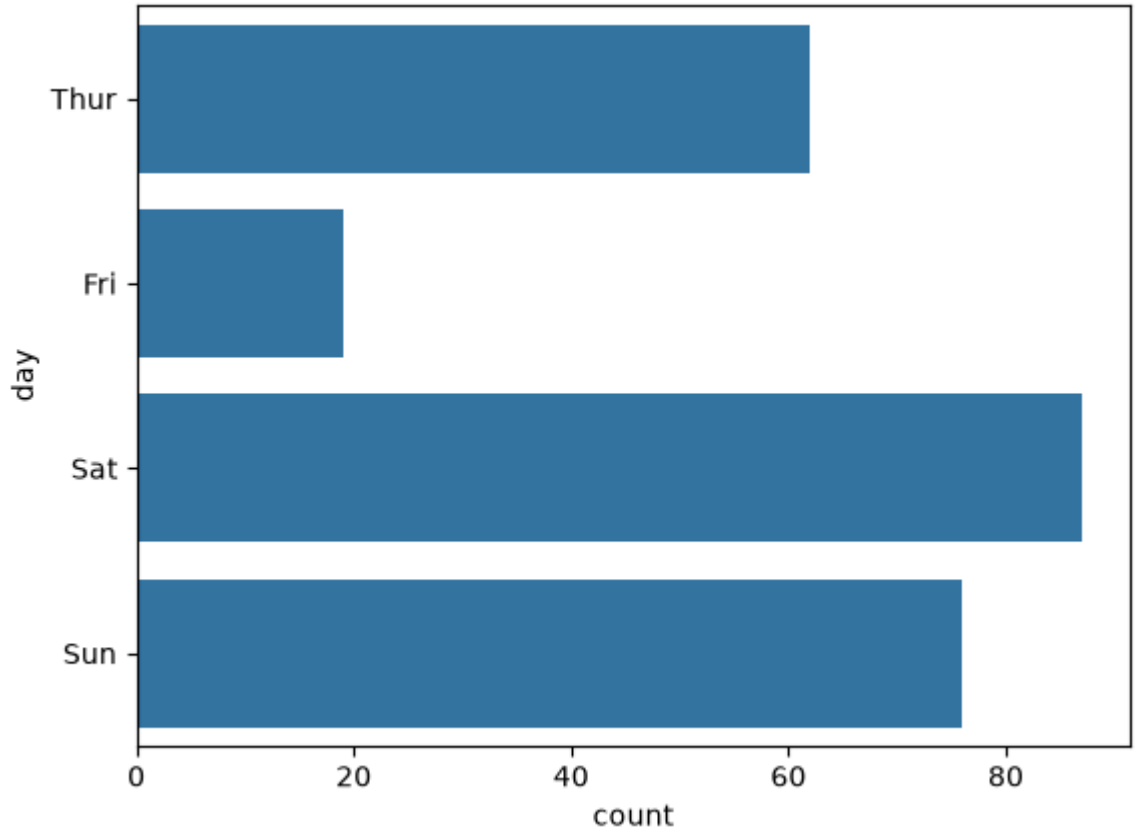
In [20]:
```
sns.boxplot(tips.tip)
```

Out[20]: <Axes: ylabel='tip'>



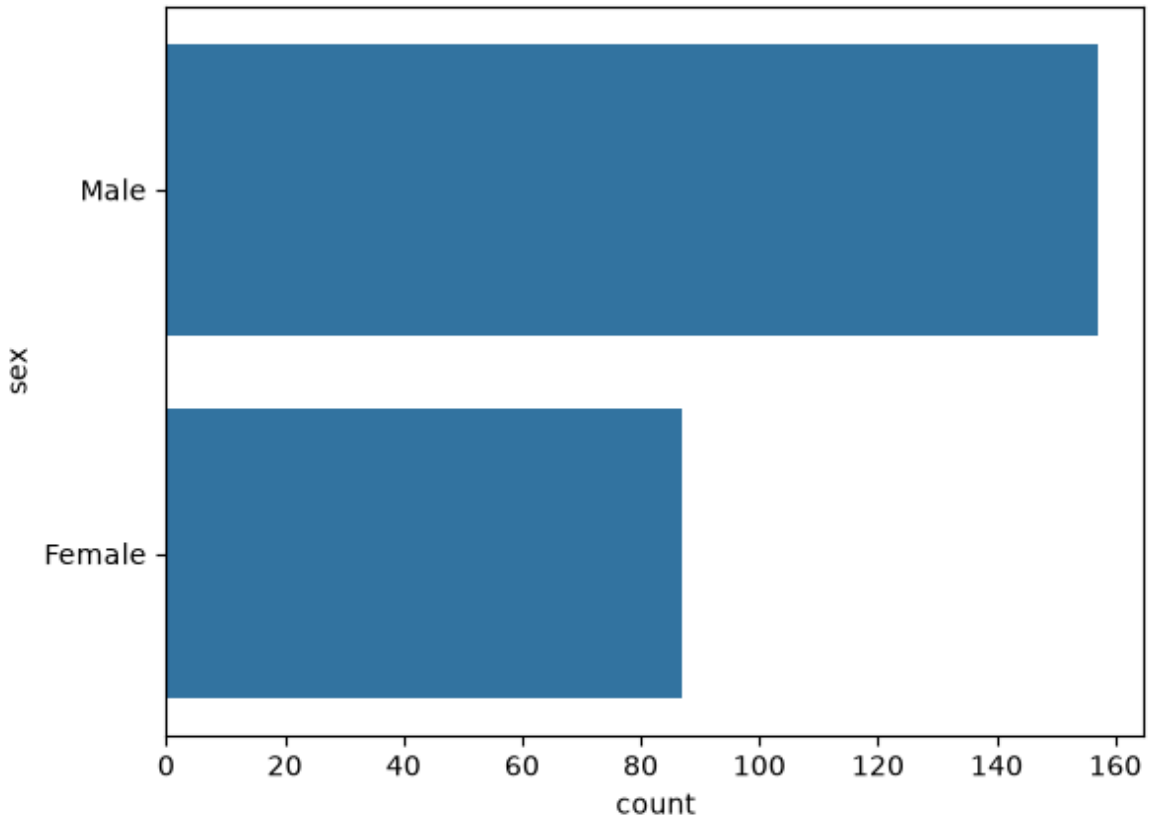
In [21]:
```
sns.countplot(tips.day)
```

Out[21]:
```
<Axes: xlabel='count', ylabel='day'>
```



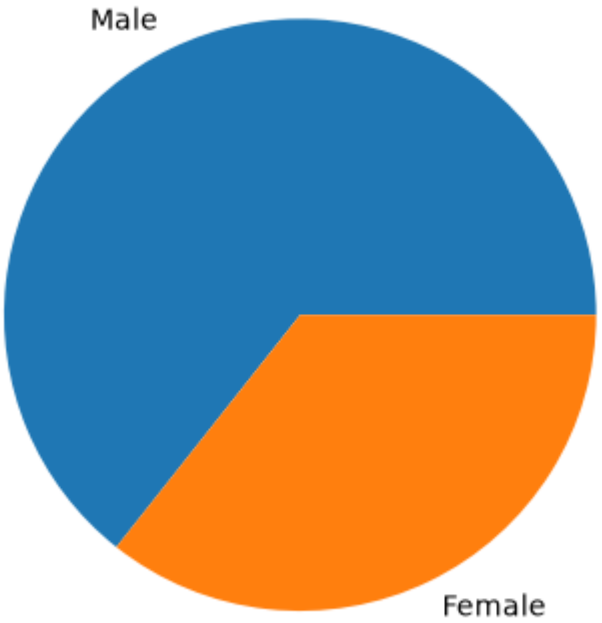
In [22]:
```
sns.countplot(tips.sex)
```

Out[22]:
```
<Axes: xlabel='count', ylabel='sex'>
```



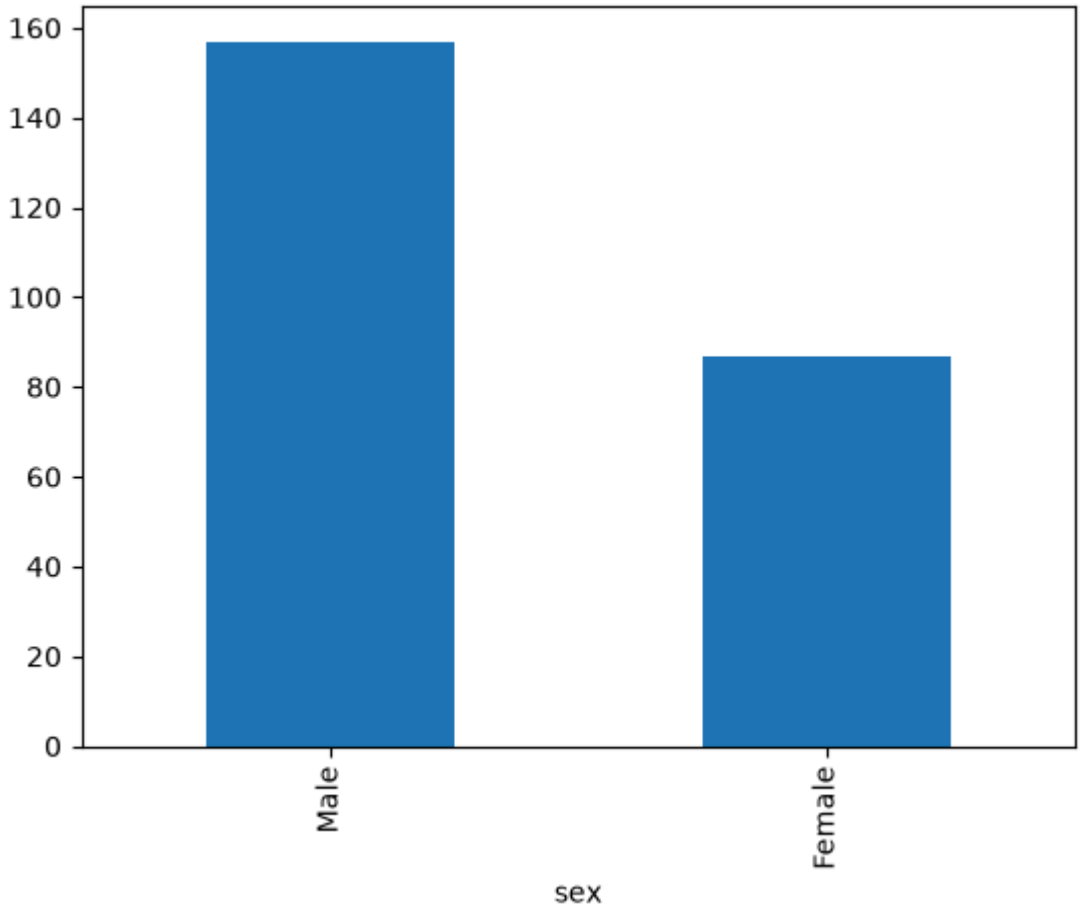
In [23]:
```
tips.sex.value_counts().plot(kind='pie')
```

Out[23]: <Axes: >



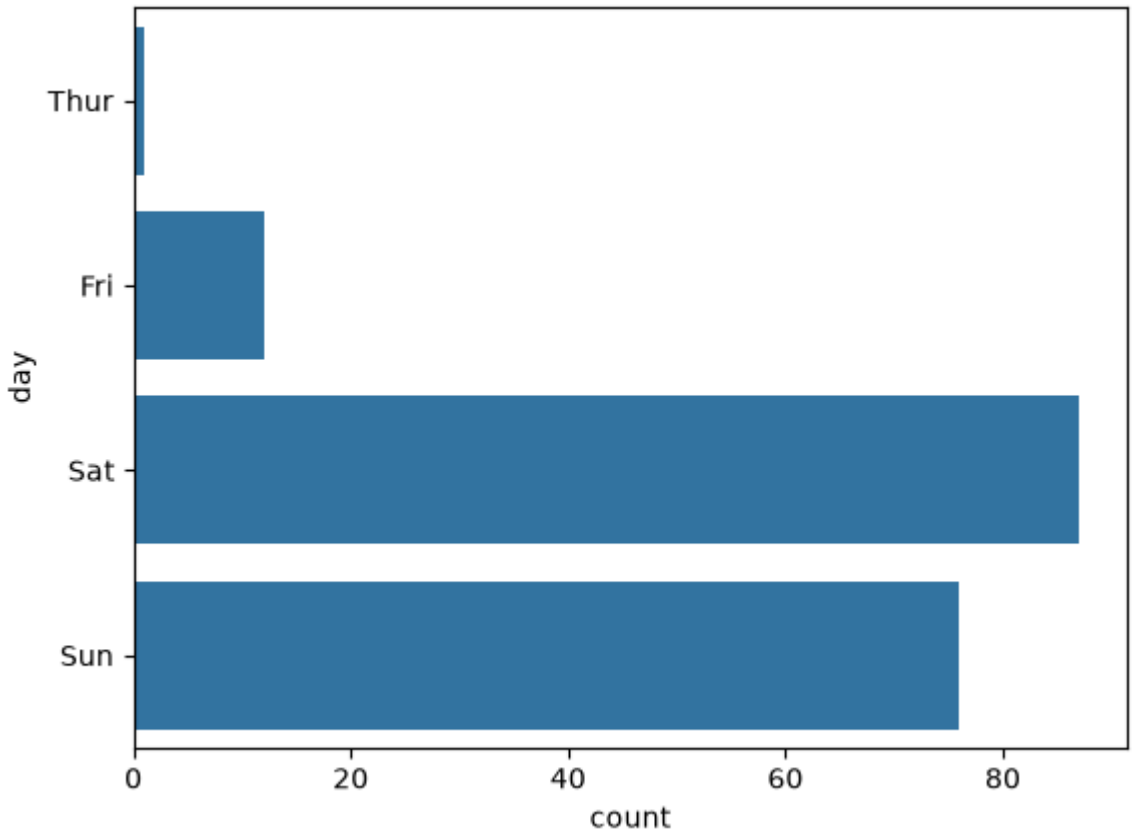
In [24]:
```
tips.sex.value_counts().plot(kind='bar')
```

Out[24]: <Axes: xlabel='sex'>



In [25]:
```
sns.countplot(tips[tips.time == 'Dinner']['day'])
```

Out[25]:
```
<Axes: xlabel='count', ylabel='day'>
```



In [ ]:

In [1]:
```
import numpy as np
import pandas as pd

data = {
    "YearsExperience": [
        1.1, 1.3, 1.5, 2, 2.2, 2.9, 3, 3.2, 3.2, 3.7,
        3.9, 4, 4, 4.1, 4.5, 4.9, 5.1, 5.3, 5.9, 6,
        6.8, 7.1, 7.9, 8.2, 8.7, 9, 9.5, 9.6, 10.3, 10.5
    ],
    "Salary": [
        39343, 46205, 37731, 43525, 39891, 56642, 60150, 54445, 64445, 57189,
        63218, 55794, 56957, 57081, 61111, 67938, 66029, 83088, 81363, 93940,
        91738, 98273, 101302, 113812, 109431, 105582, 116969, 112635, 122391, 12
    ]
}

df = pd.DataFrame(data)

df
```

Out[1]:

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891
5	2.9	56642
6	3.0	60150
7	3.2	54445
8	3.2	64445
9	3.7	57189
10	3.9	63218
11	4.0	55794
12	4.0	56957
13	4.1	57081
14	4.5	61111
15	4.9	67938
16	5.1	66029
17	5.3	83088
18	5.9	81363
19	6.0	93940
20	6.8	91738
21	7.1	98273
22	7.9	101302
23	8.2	113812
24	8.7	109431
25	9.0	105582
26	9.5	116969
27	9.6	112635
28	10.3	122391
29	10.5	121872

In [2]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

In [3]:
```
df.dropna(inplace=True)
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

In [4]:
```
df.describe()
```

Out[4]:

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

In [5]:
```
features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values
```

In [6]:
```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=0
)
```

In [7]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(x_train, y_train)
```

Out[7]:
```
▾ LinearRegression
▸ Parameters
▸ Fitted attributes
```

In [8]:
```
model.score(x_train, y_train)
```

Out[8]:
```
0.9411949620562126
```

In [9]:
```
model.score(x_test, y_test)
```

Out[9]:
```
0.988169515729126
```

In [10]:
```
model.coef_
```

Out[10]:
```
array([[9312.57512673]])
```

In [11]:
```
model.intercept_
```

Out[11]:
```
array([26780.09915063])
```

In [12]:
```
import pickle

pickle.dump(model, open('SalaryPred.model', 'wb'))
```

In [13]:
```
model = pickle.load(open('SalaryPred.model', 'rb'))
```

In [14]:
```
yr_of_exp = float(input("Enter Years of Experience: "))

yr_of_exp_NP = np.array([[yr_of_exp]])

Salary = model.predict(yr_of_exp_NP)

print("Estimated Salary for {} years of experience is {}:".format(
    yr_of_exp, Salary
))
```

```
Estimated Salary for 44.0 years of experience is [[436533.40472671]]:
```

In [ ]:

In [1]:
```
import numpy as np
import pandas as pd
from io import StringIO
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

data = """User ID,Gender,Age,EstimatedSalary,Purchased
15624510,Male,19,19000,0
15810944,Male,35,20000,0
15668575,Female,26,43000,0
15603246,Female,27,57000,0
15804002,Male,19,76000,0
15728773,Male,27,58000,0
15598044,Female,27,84000,0
15694829,Female,32,150000,1
15600575,Male,25,33000,0
15727311,Female,35,65000,0
15570769,Female,26,80000,0
15606274,Female,26,52000,0
15746139,Male,20,86000,0
15704987,Male,32,18000,0
15628972,Male,18,82000,0
15697686,Male,29,80000,0
15733883,Male,47,25000,1
15617482,Male,45,26000,1
15704583,Male,46,28000,1
15621083,Female,48,29000,1
15649487,Male,45,22000,1
15736760,Female,47,49000,1
15714658,Male,48,41000,1
15599081,Female,45,22000,1
15705113,Male,46,23000,1
15631159,Male,47,20000,1
15792818,Male,49,28000,1
15633531,Female,47,30000,1
15744529,Male,29,43000,0
15669656,Male,31,18000,0
15581198,Male,31,74000,0
15729054,Female,27,137000,1
15573452,Female,21,16000,0
15776733,Female,28,44000,0
15724858,Male,27,90000,0
15713144,Male,35,27000,0
15690188,Female,33,28000,0
15689425,Male,30,49000,0
15671766,Female,26,72000,0
15782806,Female,27,31000,0
15764419,Female,27,17000,0
15591915,Female,33,51000,0
15772798,Male,35,108000,0
15792008,Male,30,15000,0
15715541,Female,28,84000,0
15639277,Male,23,20000,0
15798850,Male,25,79000,0
15776348,Female,27,54000,0
15727696,Male,30,135000,1
15793813,Female,31,89000,0
```

```
15694395,Female,24,32000,0
15764195,Female,18,44000,0
15744919,Female,29,83000,0
15671655,Female,35,23000,0
15654901,Female,27,58000,0
15649136,Female,24,55000,0
15775562,Female,23,48000,0
15807481,Male,28,79000,0
15642885,Male,22,18000,0
15789109,Female,32,117000,0
15814004,Male,27,20000,0
15673619,Male,25,87000,0
15595135,Female,23,66000,0
15583681,Male,32,120000,1
15605000,Female,59,83000,0
15718071,Male,24,58000,0
15679760,Male,24,19000,0
15654574,Female,23,82000,0
15577178,Female,22,63000,0
15595324,Female,31,68000,0
15756932,Male,25,80000,0
15726358,Female,24,27000,0
15595228,Female,20,23000,0
15782530,Female,33,113000,0
15592877,Male,32,18000,0
15651983,Male,34,112000,1
15746737,Male,18,52000,0
15774179,Female,22,27000,0
15667265,Female,28,87000,0
15655123,Female,26,17000,0
15595917,Male,30,80000,0
15668385,Male,39,42000,0
15709476,Male,20,49000,0
15711218,Male,35,88000,0
15798659,Female,30,62000,0
15663939,Female,31,118000,1
15694946,Male,24,55000,0
15631912,Female,28,85000,0
15768816,Male,26,81000,0
15682268,Male,35,50000,0
15684801,Male,22,81000,0
15636428,Female,30,116000,0
15809823,Male,26,15000,0
15699284,Female,29,28000,0
15786993,Female,29,83000,0
15709441,Female,35,44000,0
15710257,Female,35,25000,0
15582492,Male,28,123000,1
15575694,Male,35,73000,0
15756820,Female,28,37000,0
15766289,Male,27,88000,0
15593014,Male,28,59000,0
15584545,Female,32,86000,0
15675949,Female,33,149000,1
15672091,Female,19,21000,0
15801658,Male,21,72000,0
15706185,Female,26,35000,0
15789863,Male,27,89000,0
15720943,Male,26,86000,0
15697997,Female,38,80000,0
```

```
15665416,Female,39,71000,0
15660200,Female,37,71000,0
15619653,Male,38,61000,0
15773447,Male,37,55000,0
15739160,Male,42,80000,0
15689237,Male,40,57000,0
15679297,Male,35,75000,0
15591433,Male,36,52000,0
15642725,Male,40,59000,0
15701962,Male,41,59000,0
15811613,Female,36,75000,0
15741049,Male,37,72000,0
15724423,Female,40,75000,0
15574305,Male,35,53000,0
15678168,Female,41,51000,0
15697020,Female,39,61000,0
15610801,Male,42,65000,0
15745232,Male,26,32000,0
15722758,Male,30,17000,0
15792102,Female,26,84000,0
15675185,Male,31,58000,0
15801247,Male,33,31000,0
15725660,Male,30,87000,0
15638963,Female,21,68000,0
15800061,Female,28,55000,0
15578006,Male,23,63000,0
15668504,Female,20,82000,0
15687491,Male,30,107000,1
15610403,Female,28,59000,0
15741094,Male,19,25000,0
15807909,Male,19,85000,0
15666141,Female,18,68000,0
15617134,Male,35,59000,0
15783029,Male,30,89000,0
15622833,Female,34,25000,0
15746422,Female,24,89000,0
15750839,Female,27,96000,1
15749130,Female,41,30000,0
15779862,Male,29,61000,0
15767871,Male,20,74000,0
15679651,Female,26,15000,0
15576219,Male,41,45000,0
15699247,Male,31,76000,0
15619087,Female,36,50000,0
15605327,Male,40,47000,0
15610140,Female,31,15000,0
15791174,Male,46,59000,0
15602373,Male,29,75000,0
15762605,Male,26,30000,0
15598840,Female,32,135000,1
15744279,Male,32,100000,1
15670619,Male,25,90000,0
15599533,Female,37,33000,0
15757837,Male,35,38000,0
15697574,Female,33,69000,0
15578738,Female,18,86000,0
15762228,Female,22,55000,0
15614827,Female,35,71000,0
15789815,Male,29,148000,1
15579781,Female,29,47000,0
```

```
15587013,Male,21,88000,0
15570932,Male,34,115000,0
15794661,Female,26,118000,0
15581654,Female,34,43000,0
15644296,Female,34,72000,0
15614420,Female,23,28000,0
15609653,Female,35,47000,0
15594577,Male,25,22000,0
15584114,Male,24,23000,0
15673367,Female,31,34000,0
15685576,Male,26,16000,0
15774727,Female,31,71000,0
15694288,Female,32,117000,1
15603319,Male,33,43000,0
15759066,Female,33,60000,0
15814816,Male,31,66000,0
15724402,Female,20,82000,0
15571059,Female,33,41000,0
15674206,Male,35,72000,0
15715160,Male,28,32000,0
15730448,Male,24,84000,0
15662067,Female,19,26000,0
15779581,Male,29,43000,0
15662901,Male,19,70000,0
15689751,Male,28,89000,0
15667742,Male,34,43000,0
15738448,Female,30,79000,0
15680243,Female,20,36000,0
15745083,Male,26,80000,0
15708228,Male,35,22000,0
15628523,Male,35,39000,0
15708196,Male,49,74000,0
15735549,Female,39,134000,1
15809347,Female,41,71000,0
15660866,Female,58,101000,1
15766609,Female,47,47000,0
15654230,Female,55,130000,1
15794566,Female,52,114000,0
15800890,Female,40,142000,1
15697424,Female,46,22000,0
15724536,Female,48,96000,1
15735878,Male,52,150000,1
15707596,Female,59,42000,0
15657163,Male,35,58000,0
15622478,Male,47,43000,0
15779529,Female,60,108000,1
15636023,Male,49,65000,0
15582066,Male,40,78000,0
15666675,Female,46,96000,0
15732987,Male,59,143000,1
15789432,Female,41,80000,0
15663161,Male,35,91000,1
15694879,Male,37,144000,1
15593715,Male,60,102000,1
15575002,Female,35,60000,0
15622171,Male,37,53000,0
15795224,Female,36,126000,1
15685346,Male,56,133000,1
15691808,Female,40,72000,0
15721007,Female,42,80000,1
```

```
15794253,Female,35,147000,1
15694453,Male,39,42000,0
15813113,Male,40,107000,1
15614187,Male,49,86000,1
15619407,Female,38,112000,0
15646227,Male,46,79000,1
15660541,Male,40,57000,0
15753874,Female,37,80000,0
15617877,Female,46,82000,0
15772073,Female,53,143000,1
15701537,Male,42,149000,1
15736228,Male,38,59000,0
15780572,Female,50,88000,1
15769596,Female,56,104000,1
15586996,Female,41,72000,0
15722061,Female,51,146000,1
15638003,Female,35,50000,0
15775590,Female,57,122000,1
15730688,Male,41,52000,0
15753102,Female,35,97000,1
15810075,Female,44,39000,0
15723373,Male,37,52000,0
15795298,Female,48,134000,1
15584320,Female,37,146000,1
15724161,Female,50,44000,0
15750056,Female,52,90000,1
15609637,Female,41,72000,0
15794493,Male,40,57000,0
15569641,Female,58,95000,1
15815236,Female,45,131000,1
15811177,Female,35,77000,0
15680587,Male,36,144000,1
15672821,Female,55,125000,1
15767681,Female,35,72000,0
15600379,Male,48,90000,1
15801336,Female,42,108000,1
15721592,Male,40,75000,0
15581282,Male,37,74000,0
15746203,Female,47,144000,1
15583137,Male,40,61000,0
15680752,Female,43,133000,0
15688172,Female,59,76000,1
15791373,Male,60,42000,1
15589449,Male,39,106000,1
15692819,Female,57,26000,1
15727467,Male,57,74000,1
15734312,Male,38,71000,0
15764604,Male,49,88000,1
15613014,Female,52,38000,1
15759684,Female,50,36000,1
15609669,Female,59,88000,1
15685536,Male,35,61000,0
15750447,Male,37,70000,1
15663249,Female,52,21000,1
15638646,Male,48,141000,0
15734161,Female,37,93000,1
15631070,Female,37,62000,0
15761950,Female,48,138000,1
15649668,Male,41,79000,0
15713912,Female,37,78000,1
```

```
15586757,Male,39,134000,1
15596522,Male,49,89000,1
15625395,Male,55,39000,1
15760570,Male,37,77000,0
15566689,Female,35,57000,0
15725794,Female,36,63000,0
15673539,Male,42,73000,1
15705298,Female,43,112000,1
15675791,Male,45,79000,0
15747043,Male,46,117000,1
15736397,Female,58,38000,1
15678201,Male,48,74000,1
15720745,Female,37,137000,1
15637593,Male,37,79000,1
15598070,Female,40,60000,0
15787550,Male,42,54000,0
15603942,Female,51,134000,0
15733973,Female,47,113000,1
15596761,Male,36,125000,1
15652400,Female,38,50000,0
15717893,Female,42,70000,0
15622585,Male,39,96000,1
15733964,Female,38,50000,0
15753861,Female,49,141000,1
15747097,Female,39,79000,0
15594762,Female,39,75000,1
15667417,Female,54,104000,1
15684861,Male,35,55000,0
15742204,Male,45,32000,1
15623502,Male,36,60000,0
15774872,Female,52,138000,1
15611191,Female,53,82000,1
15674331,Male,41,52000,0
15619465,Female,48,30000,1
15575247,Female,48,131000,1
15695679,Female,41,60000,0
15713463,Male,41,72000,0
15785170,Female,42,75000,0
15796351,Male,36,118000,1
15639576,Female,47,107000,1
15693264,Male,38,51000,0
15589715,Female,48,119000,1
15769902,Male,42,65000,0
15587177,Male,40,65000,0
15814553,Male,57,60000,1
15601550,Female,36,54000,0
15664907,Male,58,144000,1
15612465,Male,35,79000,0
15810800,Female,38,55000,0
15665760,Male,39,122000,1
15588080,Female,53,104000,1
15776844,Male,35,75000,0
15717560,Female,38,65000,0
15629739,Female,47,51000,1
15729908,Male,47,105000,1
15716781,Female,41,63000,0
15646936,Male,53,72000,1
15768151,Female,54,108000,1
15579212,Male,39,77000,0
15721835,Male,38,61000,0
```

```
15800515,Female,38,113000,1
15591279,Male,37,75000,0
15587419,Female,42,90000,1
15750335,Female,37,57000,0
15699619,Male,36,99000,1
15606472,Male,60,34000,1
15778368,Male,54,70000,1
15671387,Female,41,72000,0
15573926,Male,40,71000,1
15709183,Male,42,54000,0
15577514,Male,43,129000,1
15778830,Female,53,34000,1
15768072,Female,47,50000,1
15768293,Female,42,79000,0
15654456,Male,42,104000,1
15807525,Female,59,29000,1
15574372,Female,58,47000,1
15671249,Male,46,88000,1
15779744,Male,38,71000,0
15624755,Female,54,26000,1
15611430,Female,60,46000,1
15774744,Male,60,83000,1
15629885,Female,39,73000,0
15708791,Male,59,130000,1
15793890,Female,37,80000,0
15646091,Female,46,32000,1
15596984,Female,46,74000,0
15800215,Female,42,53000,0
15577806,Male,41,87000,1
15749381,Female,58,23000,1
15683758,Male,42,64000,0
15670615,Male,48,33000,1
15715622,Female,44,139000,1
15707634,Male,49,28000,1
15806901,Female,57,33000,1
15775335,Male,56,60000,1
15724150,Female,49,39000,1
15627220,Male,39,71000,0
15672330,Male,47,34000,1
15668521,Female,48,35000,1
15807837,Male,48,33000,1
15592570,Male,47,23000,1
15748589,Female,45,45000,1
15635893,Male,60,42000,1
15757632,Female,39,59000,0
15691863,Female,46,41000,1
15706071,Male,51,23000,1
15654296,Female,50,20000,1
15755018,Male,36,33000,0
15594041,Female,49,36000,1"""

df = pd.read_csv(StringIO(data))

features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values

for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=i
    )
```

```
    model = LogisticRegression()
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    if test_score > train_score:
        print("Test {} Train {} Random State {}".format(
            test_score, train_score, i
        ))

x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=37
)

finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)

print(finalModel.score(x_train, y_train))
print(finalModel.score(x_test, y_test))
print(classification_report(label, finalModel.predict(features)))
```

```
Test 0.9 Train 0.840625 Random State 4
Test 0.8625 Train 0.85 Random State 5
Test 0.8625 Train 0.859375 Random State 6
Test 0.8875 Train 0.8375 Random State 7
Test 0.8625 Train 0.8375 Random State 9
Test 0.9 Train 0.840625 Random State 10
Test 0.8625 Train 0.85625 Random State 14
Test 0.85 Train 0.84375 Random State 15
Test 0.8625 Train 0.85625 Random State 16
Test 0.875 Train 0.834375 Random State 18
Test 0.85 Train 0.84375 Random State 19
Test 0.875 Train 0.84375 Random State 20
Test 0.8625 Train 0.834375 Random State 21
Test 0.875 Train 0.840625 Random State 22
Test 0.875 Train 0.840625 Random State 24
Test 0.85 Train 0.834375 Random State 26
Test 0.85 Train 0.840625 Random State 27
Test 0.8625 Train 0.834375 Random State 30
Test 0.8625 Train 0.85625 Random State 31
Test 0.875 Train 0.853125 Random State 32
Test 0.8625 Train 0.84375 Random State 33
Test 0.875 Train 0.83125 Random State 35
Test 0.8625 Train 0.853125 Random State 36
Test 0.8875 Train 0.840625 Random State 38
Test 0.875 Train 0.8375 Random State 39
Test 0.8875 Train 0.8375 Random State 42
Test 0.875 Train 0.846875 Random State 46
Test 0.9125 Train 0.83125 Random State 47
Test 0.875 Train 0.83125 Random State 51
Test 0.9 Train 0.84375 Random State 54
Test 0.85 Train 0.84375 Random State 57
Test 0.875 Train 0.84375 Random State 58
Test 0.925 Train 0.8375 Random State 61
Test 0.8875 Train 0.834375 Random State 65
Test 0.8875 Train 0.840625 Random State 68
Test 0.9 Train 0.83125 Random State 72
Test 0.8875 Train 0.8375 Random State 75
Test 0.925 Train 0.825 Random State 76
Test 0.8625 Train 0.840625 Random State 77
Test 0.8625 Train 0.859375 Random State 81
Test 0.875 Train 0.8375 Random State 82
Test 0.8875 Train 0.8375 Random State 83
Test 0.8625 Train 0.853125 Random State 84
Test 0.8625 Train 0.840625 Random State 85
Test 0.8625 Train 0.840625 Random State 87
Test 0.875 Train 0.846875 Random State 88
Test 0.9125 Train 0.8375 Random State 90
Test 0.8625 Train 0.85 Random State 95
Test 0.875 Train 0.85 Random State 99
Test 0.85 Train 0.840625 Random State 101
Test 0.85 Train 0.840625 Random State 102
Test 0.9 Train 0.825 Random State 106
Test 0.8625 Train 0.840625 Random State 107
Test 0.85 Train 0.834375 Random State 109
Test 0.85 Train 0.840625 Random State 111
Test 0.9125 Train 0.840625 Random State 112
Test 0.8625 Train 0.85 Random State 115
Test 0.8625 Train 0.840625 Random State 116
Test 0.875 Train 0.834375 Random State 119
Test 0.9125 Train 0.828125 Random State 120
```

```
Test 0.8625 Train 0.859375 Random State 125
Test 0.85 Train 0.846875 Random State 128
Test 0.875 Train 0.85 Random State 130
Test 0.9 Train 0.84375 Random State 133
Test 0.925 Train 0.834375 Random State 134
Test 0.8625 Train 0.85 Random State 135
Test 0.875 Train 0.83125 Random State 138
Test 0.8625 Train 0.85 Random State 141
Test 0.85 Train 0.846875 Random State 143
Test 0.85 Train 0.846875 Random State 146
Test 0.85 Train 0.84375 Random State 147
Test 0.8625 Train 0.85 Random State 148
Test 0.875 Train 0.8375 Random State 150
Test 0.8875 Train 0.83125 Random State 151
Test 0.925 Train 0.84375 Random State 152
Test 0.85 Train 0.840625 Random State 153
Test 0.9 Train 0.84375 Random State 154
Test 0.9 Train 0.840625 Random State 155
Test 0.8875 Train 0.846875 Random State 156
Test 0.8875 Train 0.834375 Random State 158
Test 0.875 Train 0.828125 Random State 159
Test 0.9 Train 0.83125 Random State 161
Test 0.85 Train 0.8375 Random State 163
Test 0.875 Train 0.83125 Random State 164
Test 0.8625 Train 0.85 Random State 169
Test 0.875 Train 0.840625 Random State 171
Test 0.85 Train 0.840625 Random State 172
Test 0.9 Train 0.825 Random State 180
Test 0.85 Train 0.834375 Random State 184
Test 0.925 Train 0.821875 Random State 186
Test 0.9 Train 0.83125 Random State 193
Test 0.8625 Train 0.85 Random State 195
Test 0.8625 Train 0.840625 Random State 196
Test 0.8625 Train 0.8375 Random State 197
Test 0.875 Train 0.840625 Random State 198
Test 0.8875 Train 0.8375 Random State 199
Test 0.8875 Train 0.84375 Random State 200
Test 0.8625 Train 0.8375 Random State 202
Test 0.8625 Train 0.840625 Random State 203
Test 0.8875 Train 0.83125 Random State 206
Test 0.8625 Train 0.834375 Random State 211
Test 0.85 Train 0.84375 Random State 212
Test 0.8625 Train 0.834375 Random State 214
Test 0.875 Train 0.83125 Random State 217
Test 0.9625 Train 0.81875 Random State 220
Test 0.875 Train 0.84375 Random State 221
Test 0.85 Train 0.840625 Random State 222
Test 0.9 Train 0.84375 Random State 223
Test 0.8625 Train 0.853125 Random State 227
Test 0.8625 Train 0.834375 Random State 228
Test 0.9 Train 0.840625 Random State 229
Test 0.85 Train 0.84375 Random State 232
Test 0.875 Train 0.846875 Random State 233
Test 0.9125 Train 0.840625 Random State 234
Test 0.8625 Train 0.840625 Random State 235
Test 0.85 Train 0.846875 Random State 236
Test 0.875 Train 0.846875 Random State 239
Test 0.85 Train 0.84375 Random State 241
Test 0.8875 Train 0.85 Random State 242
Test 0.8875 Train 0.825 Random State 243
```

```
Test 0.875 Train 0.846875 Random State 244
Test 0.875 Train 0.840625 Random State 245
Test 0.875 Train 0.846875 Random State 246
Test 0.8625 Train 0.859375 Random State 247
Test 0.8875 Train 0.84375 Random State 248
Test 0.8625 Train 0.85 Random State 250
Test 0.875 Train 0.83125 Random State 251
Test 0.8875 Train 0.84375 Random State 252
Test 0.8625 Train 0.846875 Random State 255
Test 0.9 Train 0.840625 Random State 257
Test 0.8625 Train 0.85625 Random State 260
Test 0.8625 Train 0.840625 Random State 266
Test 0.8625 Train 0.8375 Random State 268
Test 0.875 Train 0.840625 Random State 275
Test 0.8625 Train 0.85 Random State 276
Test 0.925 Train 0.8375 Random State 277
Test 0.875 Train 0.846875 Random State 282
Test 0.85 Train 0.846875 Random State 283
Test 0.85 Train 0.84375 Random State 285
Test 0.9125 Train 0.834375 Random State 286
Test 0.85 Train 0.840625 Random State 290
Test 0.85 Train 0.840625 Random State 291
Test 0.85 Train 0.846875 Random State 292
Test 0.8625 Train 0.8375 Random State 294
Test 0.8875 Train 0.828125 Random State 297
Test 0.8625 Train 0.834375 Random State 300
Test 0.8625 Train 0.85 Random State 301
Test 0.8875 Train 0.85 Random State 302
Test 0.875 Train 0.846875 Random State 303
Test 0.8625 Train 0.834375 Random State 305
Test 0.9125 Train 0.8375 Random State 306
Test 0.875 Train 0.846875 Random State 308
Test 0.9 Train 0.84375 Random State 311
Test 0.8625 Train 0.834375 Random State 313
Test 0.9125 Train 0.834375 Random State 314
Test 0.875 Train 0.8375 Random State 315
Test 0.9 Train 0.846875 Random State 317
Test 0.9125 Train 0.821875 Random State 319
Test 0.8625 Train 0.85 Random State 321
Test 0.9125 Train 0.828125 Random State 322
Test 0.85 Train 0.846875 Random State 328
Test 0.85 Train 0.8375 Random State 332
Test 0.8875 Train 0.853125 Random State 336
Test 0.85 Train 0.8375 Random State 337
Test 0.875 Train 0.840625 Random State 343
Test 0.8625 Train 0.84375 Random State 346
Test 0.8875 Train 0.83125 Random State 351
Test 0.8625 Train 0.85 Random State 352
Test 0.95 Train 0.81875 Random State 354
Test 0.8625 Train 0.85 Random State 356
Test 0.9125 Train 0.840625 Random State 357
Test 0.8625 Train 0.8375 Random State 358
Test 0.85 Train 0.840625 Random State 362
Test 0.9 Train 0.84375 Random State 363
Test 0.8625 Train 0.853125 Random State 364
Test 0.9375 Train 0.821875 Random State 366
Test 0.9125 Train 0.840625 Random State 369
Test 0.8625 Train 0.853125 Random State 371
Test 0.925 Train 0.834375 Random State 376
Test 0.9125 Train 0.828125 Random State 377
```

```
Test 0.8875 Train 0.85 Random State 378
Test 0.8875 Train 0.85 Random State 379
Test 0.8625 Train 0.840625 Random State 382
Test 0.8625 Train 0.859375 Random State 386
Test 0.85 Train 0.8375 Random State 387
Test 0.875 Train 0.828125 Random State 388
Test 0.85 Train 0.84375 Random State 394
Test 0.8625 Train 0.8375 Random State 395
Test 0.9 Train 0.84375 Random State 397
Test 0.8625 Train 0.84375 Random State 400
0.865625
0.8125
              precision    recall  f1-score   support

           0       0.86      0.92      0.89       257
           1       0.83      0.74      0.79       143

    accuracy                           0.85       400
   macro avg       0.85      0.83      0.84       400
weighted avg       0.85      0.85      0.85       400
```

In [ ]:

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from io import StringIO

data = """CustomerID,Gender,Age,Annual Income (k$),Spending Score (1-100)
1,Male,19,15,39
2,Male,21,15,81
3,Female,20,16,6
4,Female,23,16,77
5,Female,31,17,40
6,Female,22,17,76
7,Female,35,18,6
8,Female,23,18,94
9,Male,64,19,3
10,Female,30,19,72
11,Male,67,19,14
12,Female,35,19,99
13,Female,58,20,15
14,Female,24,20,77
15,Male,37,20,13
16,Male,22,20,79
17,Female,35,21,35
18,Male,20,21,66
19,Male,52,23,29
20,Female,35,23,98
21,Male,35,24,35
22,Male,25,24,73
23,Female,46,25,5
24,Male,31,25,73
25,Female,54,28,14
26,Male,29,28,82
27,Female,45,28,32
28,Male,35,28,61
29,Female,40,29,31
30,Female,23,29,87
31,Male,60,30,4
32,Female,21,30,73
33,Male,53,33,4
34,Male,18,33,92
35,Female,49,33,14
36,Female,21,33,81
37,Female,42,34,17
38,Female,30,34,73
39,Female,36,37,26
40,Female,20,37,75
41,Female,65,38,35
42,Male,24,38,92
43,Male,48,39,36
44,Female,31,39,61
45,Female,49,39,28
46,Female,24,39,65
47,Female,50,40,55
48,Female,27,40,47
49,Female,29,40,42
50,Female,31,40,42
51,Female,49,42,52
```

```
52,Male,33,42,60
53,Female,31,43,54
54,Male,59,43,60
55,Female,50,43,45
56,Male,47,43,41
57,Female,51,44,50
58,Male,69,44,46
59,Female,27,46,51
60,Male,53,46,46
61,Male,70,46,56
62,Male,19,46,55
63,Female,67,47,52
64,Female,54,47,59
65,Male,63,48,51
66,Male,18,48,59
67,Female,43,48,50
68,Female,68,48,48
69,Male,19,48,59
70,Female,32,48,47
71,Male,70,49,55
72,Female,47,49,42
73,Female,60,50,49
74,Female,60,50,56
75,Male,59,54,47
76,Male,26,54,54
77,Female,45,54,53
78,Male,40,54,48
79,Female,23,54,52
80,Female,49,54,42
81,Male,57,54,51
82,Male,38,54,55
83,Male,67,54,41
84,Female,46,54,44
85,Female,21,54,57
86,Male,48,54,46
87,Female,55,57,58
88,Female,22,57,55
89,Female,34,58,60
90,Female,50,58,46
91,Female,68,59,55
92,Male,18,59,41
93,Male,48,60,49
94,Female,40,60,40
95,Female,32,60,42
96,Male,24,60,52
97,Female,47,60,47
98,Female,27,60,50
99,Male,48,61,42
100,Male,20,61,49
101,Female,23,62,41
102,Female,49,62,48
103,Male,67,62,59
104,Male,26,62,55
105,Male,49,62,56
106,Female,21,62,42
107,Female,66,63,50
108,Male,54,63,46
109,Male,68,63,43
110,Male,66,63,48
111,Male,65,63,52
```

```
112,Female,19,63,54
113,Female,38,64,42
114,Male,19,64,46
115,Female,18,65,48
116,Female,19,65,50
117,Female,63,65,43
118,Female,49,65,59
119,Female,51,67,43
120,Female,50,67,57
121,Male,27,67,56
122,Female,38,67,40
123,Female,40,69,58
124,Male,39,69,91
125,Female,23,70,29
126,Female,31,70,77
127,Male,43,71,35
128,Male,40,71,95
129,Male,59,71,11
130,Male,38,71,75
131,Male,47,71,9
132,Male,39,71,75
133,Female,25,72,34
134,Female,31,72,71
135,Male,20,73,5
136,Female,29,73,88
137,Female,44,73,7
138,Male,32,73,73
139,Male,19,74,10
140,Female,35,74,72
141,Female,57,75,5
142,Male,32,75,93
143,Female,28,76,40
144,Female,32,76,87
145,Male,25,77,12
146,Male,28,77,97
147,Male,48,77,36
148,Female,32,77,74
149,Female,34,78,22
150,Male,34,78,90
151,Male,43,78,17
152,Male,39,78,88
153,Female,44,78,20
154,Female,38,78,76
155,Female,47,78,16
156,Female,27,78,89
157,Male,37,78,1
158,Female,30,78,78
159,Male,34,78,1
160,Female,30,78,73
161,Female,56,79,35
162,Female,29,79,83
163,Male,19,81,5
164,Female,31,81,93
165,Male,50,85,26
166,Female,36,85,75
167,Male,42,86,20
168,Female,33,86,95
169,Female,36,87,27
170,Male,32,87,63
171,Male,40,87,13
```

```
172,Male,28,87,75
173,Male,36,87,10
174,Male,36,87,92
175,Female,52,88,13
176,Female,30,88,86
177,Male,58,88,15
178,Male,27,88,69
179,Male,59,93,14
180,Male,35,93,90
181,Female,37,97,32
182,Female,32,97,86
183,Male,46,98,15
184,Female,29,98,88
185,Female,41,99,39
186,Male,30,99,97
187,Female,54,101,24
188,Male,28,101,68
189,Female,41,103,17
190,Female,36,103,85
191,Female,34,103,23
192,Female,32,103,69
193,Male,33,113,8
194,Female,38,113,91
195,Female,47,120,16
196,Female,35,120,79
197,Female,45,126,28
198,Male,32,126,74
199,Male,32,137,18
200,Male,30,137,83"""

df = pd.read_csv(StringIO(data))

df
```

Out[2]:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
...	...	...	...	...	...
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

200 rows × 5 columns

In [3]:
```
df.info()
```

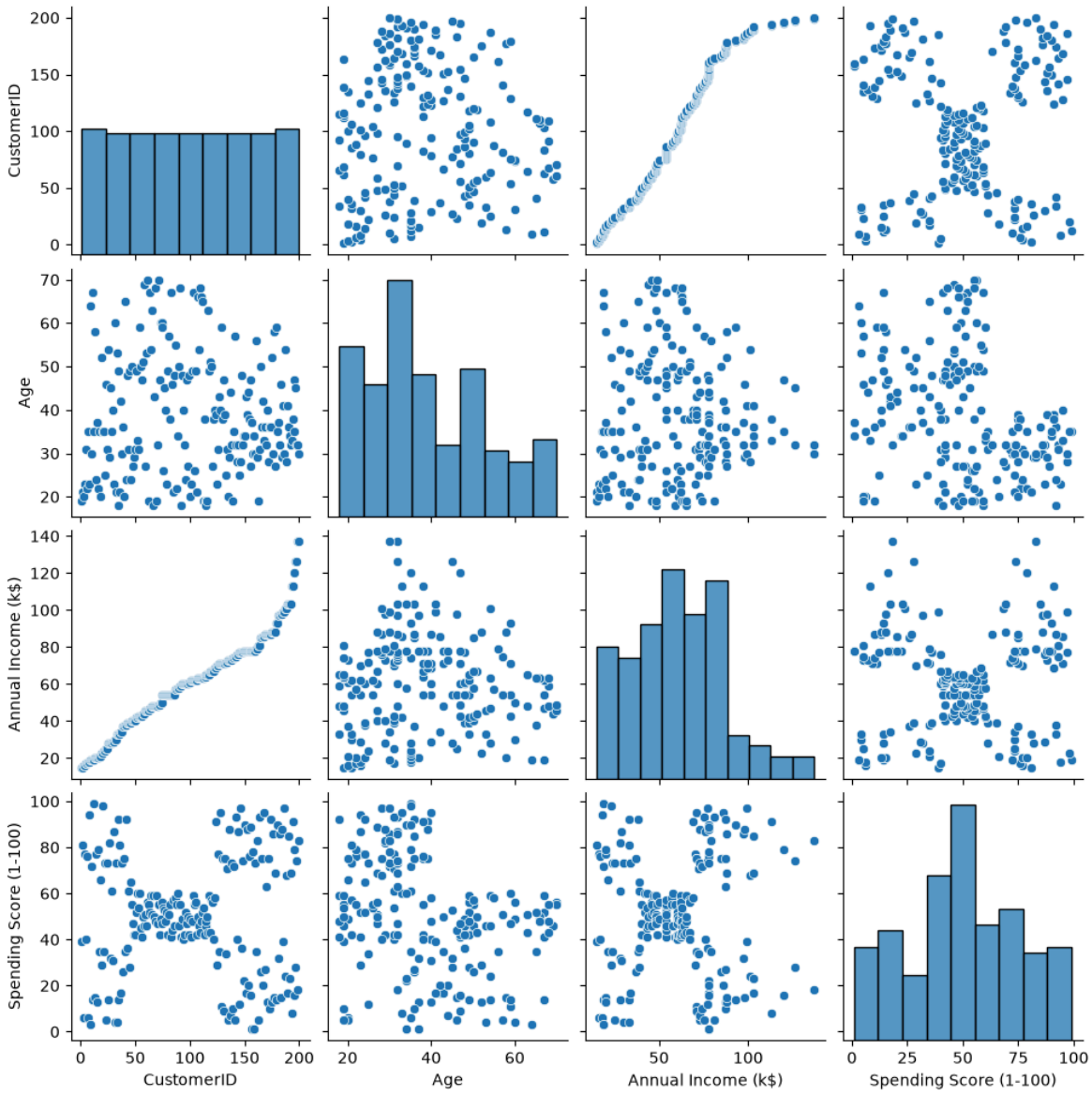
```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    str
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), str(1)
memory usage: 8.9 KB
```

In [4]:
```
df.head()
```

Out[4]:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

In [5]:
```
sns.pairplot(df)
plt.show()
```



In [6]:
```
features = df.iloc[:, [3, 4]].values

features
```

Out[6]:
```
array([[ 15,  39],
       [ 15,  81],
       [ 16,   6],
       [ 16,  77],
       [ 17,  40],
       [ 17,  76],
       [ 18,   6],
       [ 18,  94],
       [ 19,   3],
       [ 19,  72],
       [ 19,  14],
       [ 19,  99],
       [ 20,  15],
       [ 20,  77],
       [ 20,  13],
       [ 20,  79],
       [ 21,  35],
       [ 21,  66],
       [ 23,  29],
       [ 23,  98],
       [ 24,  35],
       [ 24,  73],
       [ 25,   5],
       [ 25,  73],
       [ 28,  14],
       [ 28,  82],
       [ 28,  32],
       [ 28,  61],
       [ 29,  31],
       [ 29,  87],
       [ 30,   4],
       [ 30,  73],
       [ 33,   4],
       [ 33,  92],
       [ 33,  14],
       [ 33,  81],
       [ 34,  17],
       [ 34,  73],
       [ 37,  26],
       [ 37,  75],
       [ 38,  35],
       [ 38,  92],
       [ 39,  36],
       [ 39,  61],
       [ 39,  28],
       [ 39,  65],
       [ 40,  55],
       [ 40,  47],
       [ 40,  42],
       [ 40,  42],
       [ 42,  52],
       [ 42,  60],
       [ 43,  54],
       [ 43,  60],
       [ 43,  45],
       [ 43,  41],
       [ 44,  50],
       [ 44,  46],
       [ 46,  51],
       [ 46,  46],
```

```
       [ 46,  56],
       [ 46,  55],
       [ 47,  52],
       [ 47,  59],
       [ 48,  51],
       [ 48,  59],
       [ 48,  50],
       [ 48,  48],
       [ 48,  59],
       [ 48,  47],
       [ 49,  55],
       [ 49,  42],
       [ 50,  49],
       [ 50,  56],
       [ 54,  47],
       [ 54,  54],
       [ 54,  53],
       [ 54,  48],
       [ 54,  52],
       [ 54,  42],
       [ 54,  51],
       [ 54,  55],
       [ 54,  41],
       [ 54,  44],
       [ 54,  57],
       [ 54,  46],
       [ 57,  58],
       [ 57,  55],
       [ 58,  60],
       [ 58,  46],
       [ 59,  55],
       [ 59,  41],
       [ 60,  49],
       [ 60,  40],
       [ 60,  42],
       [ 60,  52],
       [ 60,  47],
       [ 60,  50],
       [ 61,  42],
       [ 61,  49],
       [ 62,  41],
       [ 62,  48],
       [ 62,  59],
       [ 62,  55],
       [ 62,  56],
       [ 62,  42],
       [ 63,  50],
       [ 63,  46],
       [ 63,  43],
       [ 63,  48],
       [ 63,  52],
       [ 63,  54],
       [ 64,  42],
       [ 64,  46],
       [ 65,  48],
       [ 65,  50],
       [ 65,  43],
       [ 65,  59],
       [ 67,  43],
       [ 67,  57],
```

```
       [ 67,  56],
       [ 67,  40],
       [ 69,  58],
       [ 69,  91],
       [ 70,  29],
       [ 70,  77],
       [ 71,  35],
       [ 71,  95],
       [ 71,  11],
       [ 71,  75],
       [ 71,   9],
       [ 71,  75],
       [ 72,  34],
       [ 72,  71],
       [ 73,   5],
       [ 73,  88],
       [ 73,   7],
       [ 73,  73],
       [ 74,  10],
       [ 74,  72],
       [ 75,   5],
       [ 75,  93],
       [ 76,  40],
       [ 76,  87],
       [ 77,  12],
       [ 77,  97],
       [ 77,  36],
       [ 77,  74],
       [ 78,  22],
       [ 78,  90],
       [ 78,  17],
       [ 78,  88],
       [ 78,  20],
       [ 78,  76],
       [ 78,  16],
       [ 78,  89],
       [ 78,   1],
       [ 78,  78],
       [ 78,   1],
       [ 78,  73],
       [ 79,  35],
       [ 79,  83],
       [ 81,   5],
       [ 81,  93],
       [ 85,  26],
       [ 85,  75],
       [ 86,  20],
       [ 86,  95],
       [ 87,  27],
       [ 87,  63],
       [ 87,  13],
       [ 87,  75],
       [ 87,  10],
       [ 87,  92],
       [ 88,  13],
       [ 88,  86],
       [ 88,  15],
       [ 88,  69],
       [ 93,  14],
       [ 93,  90],
```

```
       [ 97,  32],
       [ 97,  86],
       [ 98,  15],
       [ 98,  88],
       [ 99,  39],
       [ 99,  97],
       [101,  24],
       [101,  68],
       [103,  17],
       [103,  85],
       [103,  23],
       [103,  69],
       [113,   8],
       [113,  91],
       [120,  16],
       [120,  79],
       [126,  28],
       [126,  74],
       [137,  18],
       [137,  83]])
```

In [7]:
```
from sklearn.cluster import KMeans

model = KMeans(n_clusters=5, random_state=42, n_init=10)

model.fit(features)
```

```
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
```

Out[7]:

▾ KMeans ⓘ ?

▸ Parameters

▸ Fitted attributes

▸ 5 features

In [8]:
```
Final = df.iloc[:, [3, 4]].copy()

Final["label"] = model.predict(features)

Final.head()
```

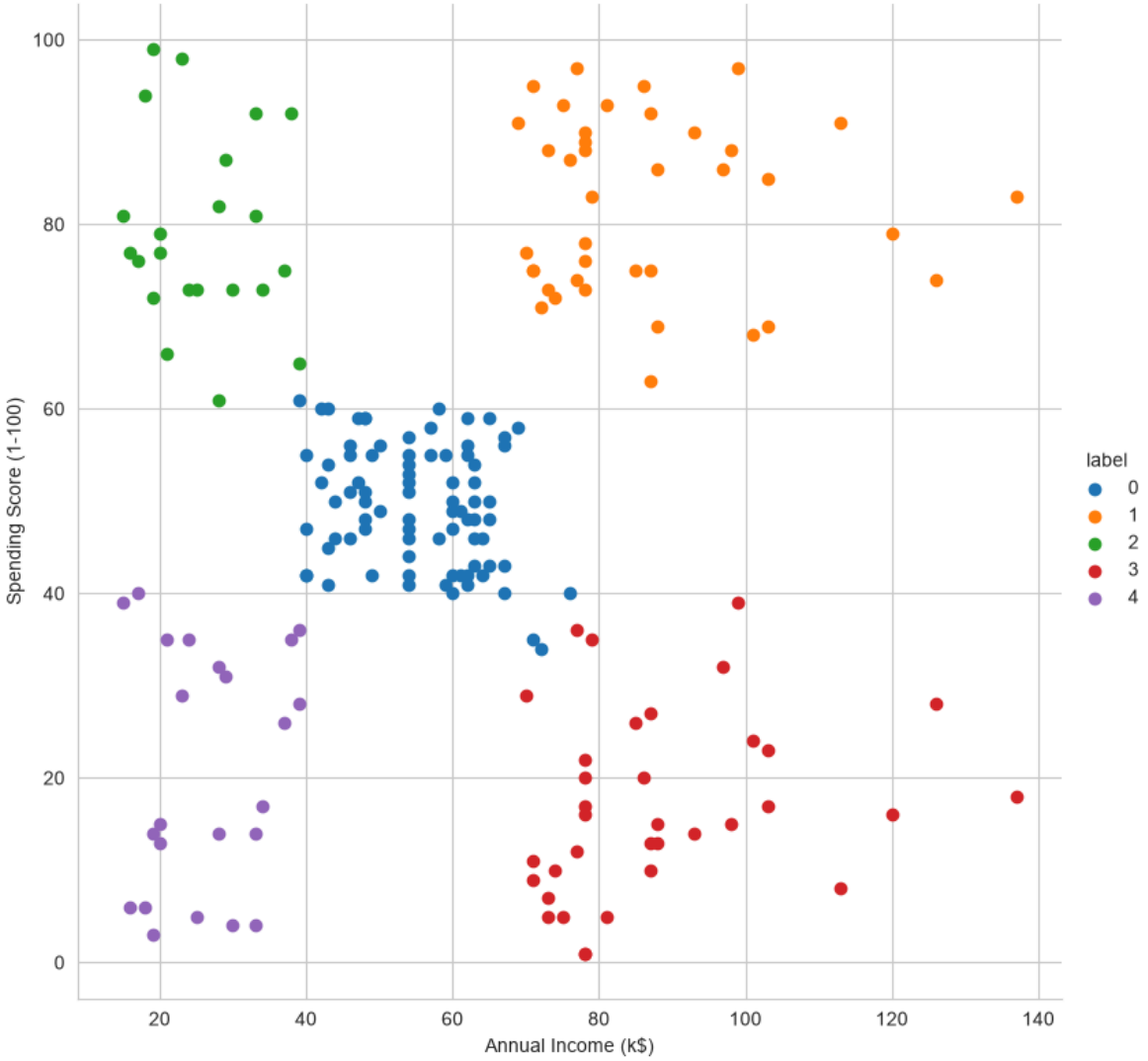
Out[8]:

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	4
1	15	81	2
2	16	6	4
3	16	77	2
4	17	40	4

In [9]:
```
sns.set_style("whitegrid")

sns.FacetGrid(Final, hue="label", height=8) \
    .map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)") \
    .add_legend()

plt.show()
```



In [10]:
```
features_el = df.iloc[:, [2, 3, 4]].values

wcss = []

for i in range(1, 10):
    model = KMeans(n_clusters=i, random_state=42, n_init=10)
```

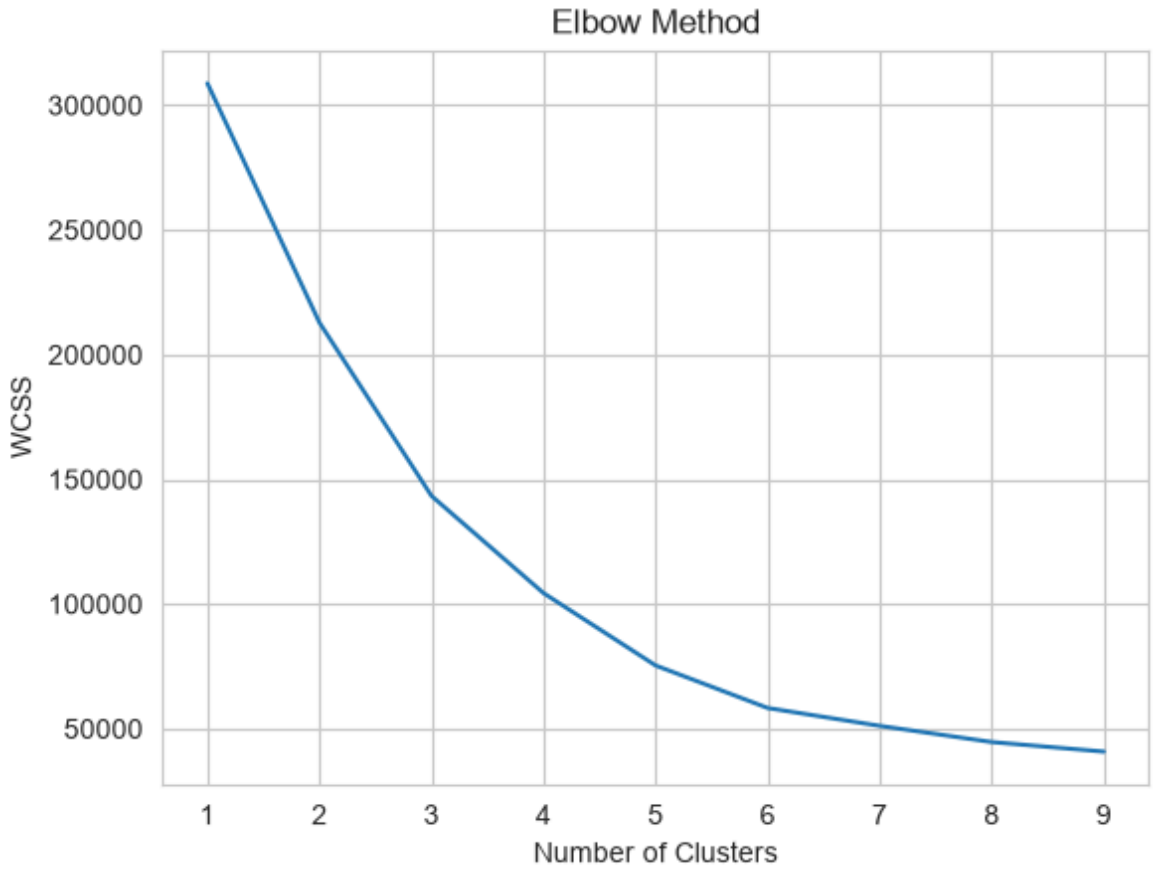
```
    model.fit(features_el)
    wcss.append(model.inertia_)
```

```
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sasmi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
```

In [11]:
```
plt.plot(range(1, 10), wcss)

plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")

plt.show()
```


Elbow Method
WCSS
300000
250000
200000
150000
100000
50000
1
2
3
4
5
6
7
8
9
Number of Clusters

In [ ]:

In [7]:
```
import numpy as np
import pandas as pd

data = {
    "sepal.length": [
        5.1,4.9,4.7,4.6,5.0,5.4,4.6,5.0,4.4,4.9,
        5.4,4.8,4.8,4.3,5.8,5.7,5.4,5.1,5.7,5.1,
        5.4,5.1,4.6,5.1,4.8,5.0,5.0,5.2,5.2,4.7,
        4.8,5.4,5.2,5.5,4.9,5.0,5.5,4.9,4.4,5.1,
        5.0,4.5,4.4,5.0,5.1,4.8,5.1,4.6,5.3,5.0,
        7.0,6.4,6.9,5.5,6.5,5.7,6.3,4.9,6.6,5.2,
        5.0,5.9,6.0,6.1,5.6,6.7,5.6,5.8,6.2,5.6,
        5.9,6.1,6.3,6.1,6.4,6.6,6.8,6.7,6.0,5.7,
        5.5,5.5,5.8,6.0,5.4,6.0,6.7,6.3,5.6,5.5,
        5.5,6.1,5.8,5.0,5.6,5.7,5.7,6.2,5.1,5.7,
        6.3,5.8,7.1,6.3,6.5,7.6,4.9,7.3,6.7,7.2,
        6.5,6.4,6.8,5.7,5.8,6.4,6.5,7.7,7.7,6.0,
        6.9,5.6,7.7,6.3,6.7,7.2,6.2,6.1,6.4,7.2,
        7.4,7.9,6.4,6.3,6.1,7.7,6.3,6.4,6.0,6.9,
        6.7,6.9,5.8,6.8,6.7,6.7,6.3,6.5,6.2,5.9
    ],
    "sepal.width": [
        3.5,3.0,3.2,3.1,3.6,3.9,3.4,3.4,2.9,3.1,
        3.7,3.4,3.0,3.0,4.0,4.4,3.9,3.5,3.8,3.8,
        3.4,3.7,3.6,3.3,3.4,3.0,3.4,3.5,3.4,3.2,
        3.1,3.4,4.1,4.2,3.1,3.2,3.5,3.1,3.0,3.4,
        3.5,2.3,3.2,3.5,3.8,3.0,3.8,3.2,3.7,3.3,
        3.2,3.2,3.1,2.3,2.8,2.8,3.3,2.4,2.9,2.7,
        2.0,3.0,2.2,2.9,2.9,3.1,3.0,2.7,2.2,2.5,
        2.8,2.5,2.8,2.9,3.0,2.8,3.2,3.3,2.7,3.0,
        2.9,2.6,2.4,2.4,2.7,2.7,3.0,3.4,3.0,2.3,
        2.5,2.3,2.5,3.0,3.4,3.0,2.9,2.6,2.7,3.0,
        2.9,2.9,3.0,2.5,3.0,3.3,2.5,2.9,2.5,3.6,
        3.2,2.7,3.0,2.5,2.8,3.2,3.0,3.8,2.6,2.2,
        3.2,2.8,2.8,2.7,3.3,3.2,2.8,3.0,2.8,3.0,
        2.8,3.8,2.8,2.8,2.6,3.0,3.4,3.1,3.0,3.1,
        3.1,3.1,2.7,3.2,3.3,3.0,2.5,3.0,3.4,3.0
    ],
    "petal.length": [
        1.4,1.4,1.3,1.5,1.4,1.7,1.4,1.5,1.4,1.5,
        1.5,1.6,1.4,1.1,1.2,1.5,1.3,1.4,1.7,1.5,
        1.7,1.5,1.0,1.7,1.9,1.6,1.6,1.5,1.4,1.6,
        1.6,1.5,1.5,1.4,1.5,1.2,1.3,1.5,1.3,1.5,
        1.3,1.3,1.3,1.6,1.9,1.4,1.6,1.4,1.5,1.4,
        4.7,4.5,4.9,4.0,4.6,4.5,4.7,3.3,4.6,3.9,
        3.5,4.2,4.0,4.7,3.6,4.4,4.5,4.1,4.5,3.9,
        4.8,4.0,4.9,4.7,4.3,4.4,4.8,5.0,4.5,3.5,
        3.8,3.7,3.9,5.1,4.5,4.5,4.7,4.4,4.1,4.0,
        4.4,4.6,4.0,3.3,4.2,4.2,4.2,4.3,3.0,4.1,
        6.0,5.1,5.9,5.6,5.8,6.6,4.5,6.3,5.8,6.1,
        5.1,5.3,5.5,5.0,5.1,5.3,5.5,6.7,6.9,5.0,
        5.7,4.9,6.7,4.9,5.7,6.0,4.8,4.9,5.6,5.8,
        6.1,5.1,5.3,5.5,5.0,5.1,5.3,5.5,6.7,6.9,
        5.0,5.7,4.9,6.7,4.9,5.7,6.0,4.8,4.9,5.6
    ],
    "petal.width": [
        0.2,0.2,0.2,0.2,0.2,0.4,0.3,0.2,0.2,0.1,
        0.2,0.2,0.1,0.1,0.2,0.4,0.4,0.3,0.3,0.3,
```

```
        0.2,0.4,0.2,0.5,0.2,0.2,0.4,0.2,0.2,0.2,
        0.2,0.4,0.1,0.2,0.2,0.2,0.2,0.1,0.2,0.2,
        0.3,0.3,0.2,0.6,0.4,0.3,0.2,0.2,0.2,0.2,
        1.4,1.5,1.5,1.3,1.5,1.3,1.6,1.0,1.3,1.4,
        1.0,1.5,1.0,1.4,1.3,1.4,1.5,1.0,1.5,1.1,
        1.8,1.3,1.5,1.2,1.3,1.4,1.4,1.7,1.5,1.0,
        1.1,1.0,1.2,1.6,1.5,1.6,1.5,1.3,1.3,1.3,
        1.2,1.4,1.2,1.0,1.3,1.2,1.3,1.3,1.1,1.3,
        2.5,1.9,2.1,1.8,2.2,2.1,1.7,1.8,1.8,1.8,
        2.5,2.0,1.9,2.1,2.0,2.4,2.3,1.8,2.2,2.3,
        1.5,2.3,2.0,2.0,1.8,2.1,1.8,1.8,1.8,2.1,
        1.6,1.9,2.0,2.2,1.5,1.4,2.3,2.4,1.8,1.8,
        2.1,2.4,2.3,1.9,2.3,1.8,1.8,2.1,2.4,2.3
    ],
    "variety": ["Setosa"]*50 + ["Versicolor"]*50 + ["Virginica"]*50
}

df = pd.DataFrame(data)

df
```

Out[7]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.7	1.8	Virginica
146	6.3	2.5	6.0	1.8	Virginica
147	6.5	3.0	4.8	2.1	Virginica
148	6.2	3.4	4.9	2.4	Virginica
149	5.9	3.0	5.6	2.3	Virginica

150 rows × 5 columns

In [8]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str    
dtypes: float64(4), str(1)
memory usage: 7.2 KB
```

In [9]:
```
df.variety.value_counts()
```

Out[9]:
```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

In [10]:
```
df.head()
```

Out[10]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

In [11]:
```
features = df.iloc[:, :-1].to_numpy()
label = df.iloc[:, 4].to_numpy()
```

In [12]:
```
from sklearn.model_selection import train_test_split

xtrain, xtest, ytrain, ytest = train_test_split(
    features,
    label,
    test_size=0.2,
    random_state=42
)
```

In [13]:
```
from sklearn.neighbors import KNeighborsClassifier

model_KNN = KNeighborsClassifier(n_neighbors=5)
```

In [14]:
```
model_KNN.fit(xtrain, ytrain)
```

Out[14]:

▾ KNeighborsClassifier

▸ Parameters

▸ Fitted attributes

In [15]:
```
print(model_KNN.score(xtrain, ytrain))
```

```
0.9583333333333334
```

In [16]:
```
print(model_KNN.score(xtest, ytest))
```

```
1.0
```

In [17]:
```
from sklearn.metrics import confusion_matrix
```

```
confusion_matrix(label, model_KNN.predict(features))
```

Out[17]:
```
array([[50,  0,  0],
       [ 0, 49,  1],
       [ 0,  4, 46]])
```

In [18]:
```
from sklearn.metrics import classification_report

print(classification_report(label, model_KNN.predict(features)))
```

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.92      0.98      0.95        50
   Virginica       0.98      0.92      0.95        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

In [ ]:
```

```

In [1]:
```
import numpy as np
from scipy import stats

marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])
mu_0 = 70

t_stat, p_value = stats.ttest_1samp(marks, mu_0)

print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 70.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
T-statistic: 1.993
P-value: 0.0774
Fail to Reject Null Hypothesis → No significant difference.
```

In [2]:
```
from math import sqrt
from scipy.stats import norm

x_bar = 51.2
mu_0 = 50
sigma = 3
n = 36

z_stat = (x_bar - mu_0) / (sigma / sqrt(n))
p_value = 2 * (1 - norm.cdf(abs(z_stat)))

print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 50 g.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis → Mean is significantly different from 50 g.
```

In [3]:
```
import numpy as np
from scipy import stats

A = [20, 22, 23]
B = [19, 20, 18]
C = [25, 27, 26]

f_stat, p_value = stats.f_oneway(A, B, C)

print(f"F-statistic: {f_stat:.3f}")
print(f"P-value: {p_value:.4f}")
```

```
alpha = 0.05

if p_value < alpha:
    print("Reject Null Hypothesis → Means are significantly different.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
F-statistic: 25.923
P-value: 0.0011
Reject Null Hypothesis → Means are significantly different.
```

In [ ]:

In [2]:
```
import matplotlib.pyplot as cricket

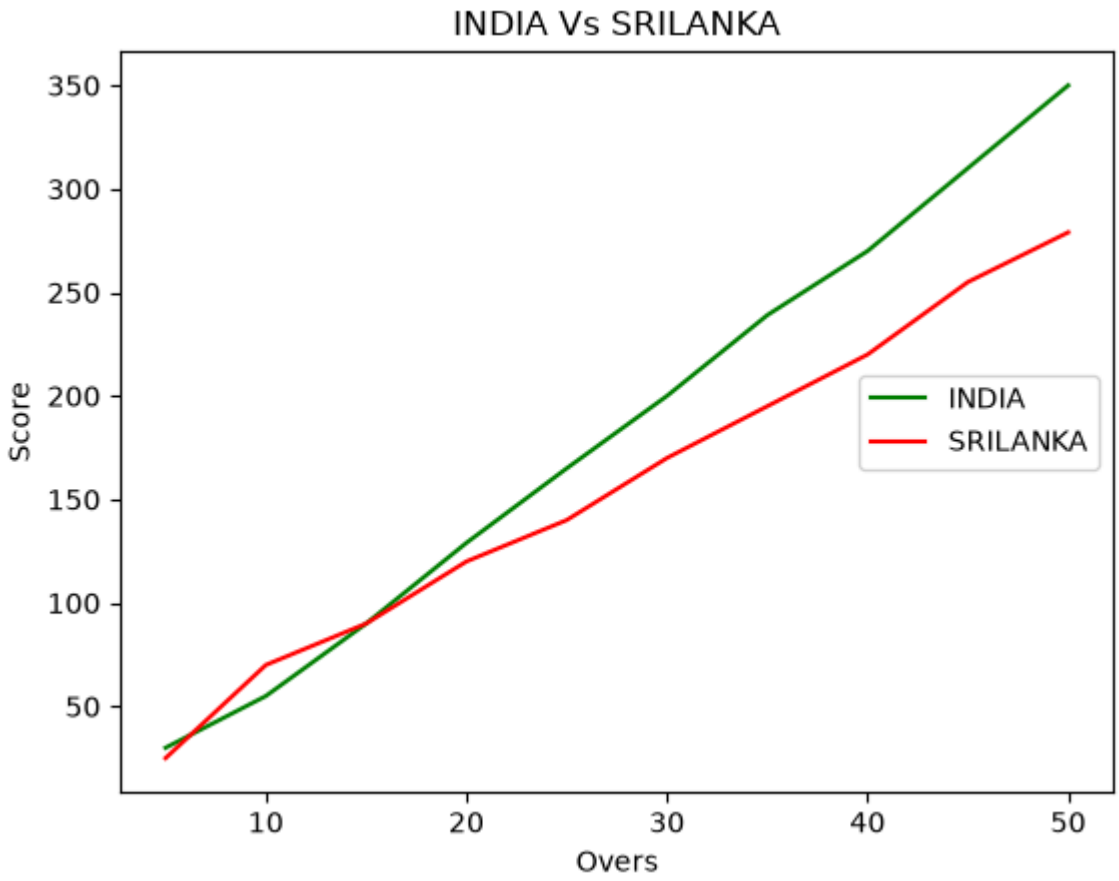
Overs = list(range(5, 51, 5))

Indian_Score = [30, 55, 90, 129, 165, 200, 239, 270, 310, 350]
Srilankan_Score = [25, 70, 90, 120, 140, 170, 195, 220, 255, 279]

cricket.plot(Overs, Indian_Score, color="green", label="INDIA")
cricket.plot(Overs, Srilankan_Score, color="red", label="SRILANKA")

cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend(loc="center right")

cricket.show()
```



In [3]:
```
import matplotlib.pyplot as hscmark
import numpy as np

Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']

xaxis = np.arange(len(Names))

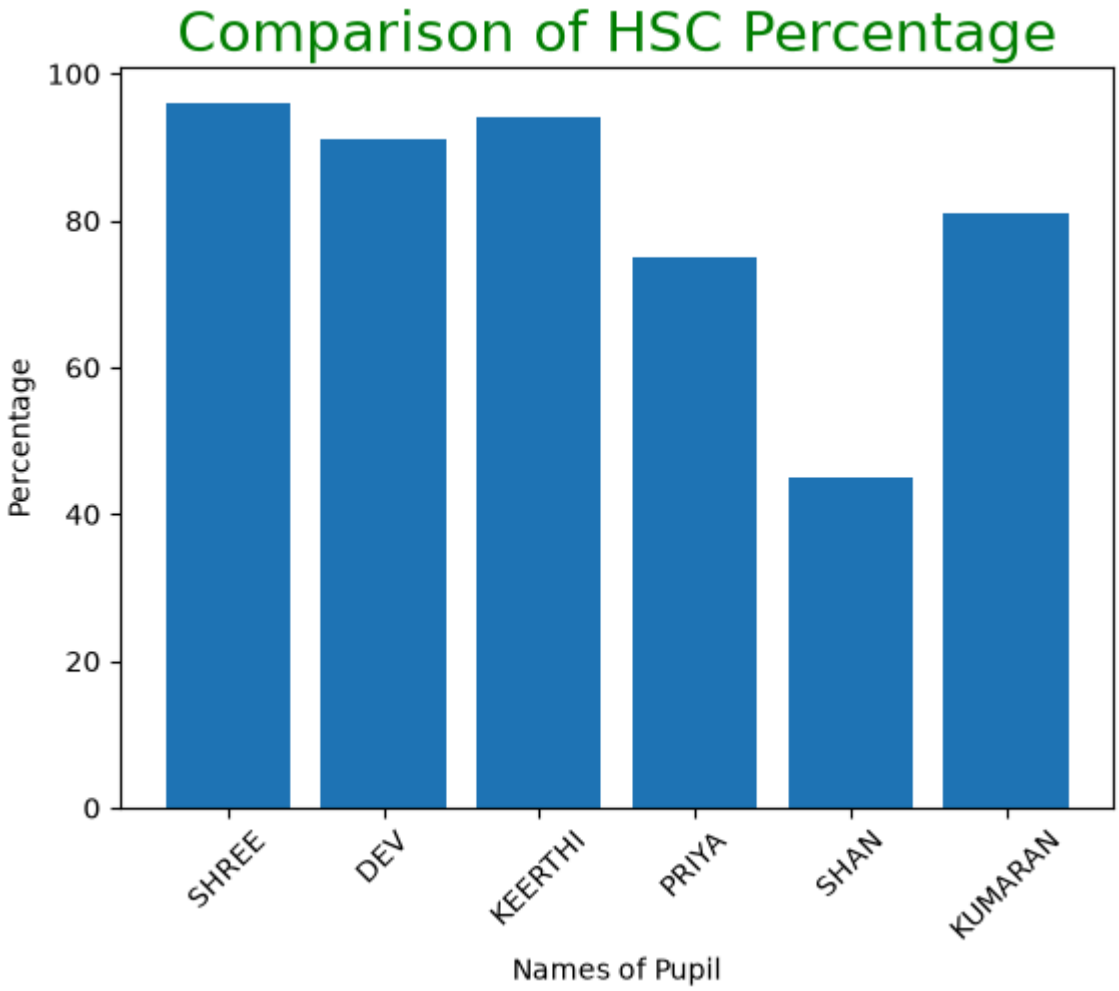
Percentage_hsc = [96, 91, 94, 75, 45, 81]

hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)

hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
```

```
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")

hscmark.show()
```



In [4]:
```
import matplotlib.pyplot as election

labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']

Votes = [315, 130, 245, 210]

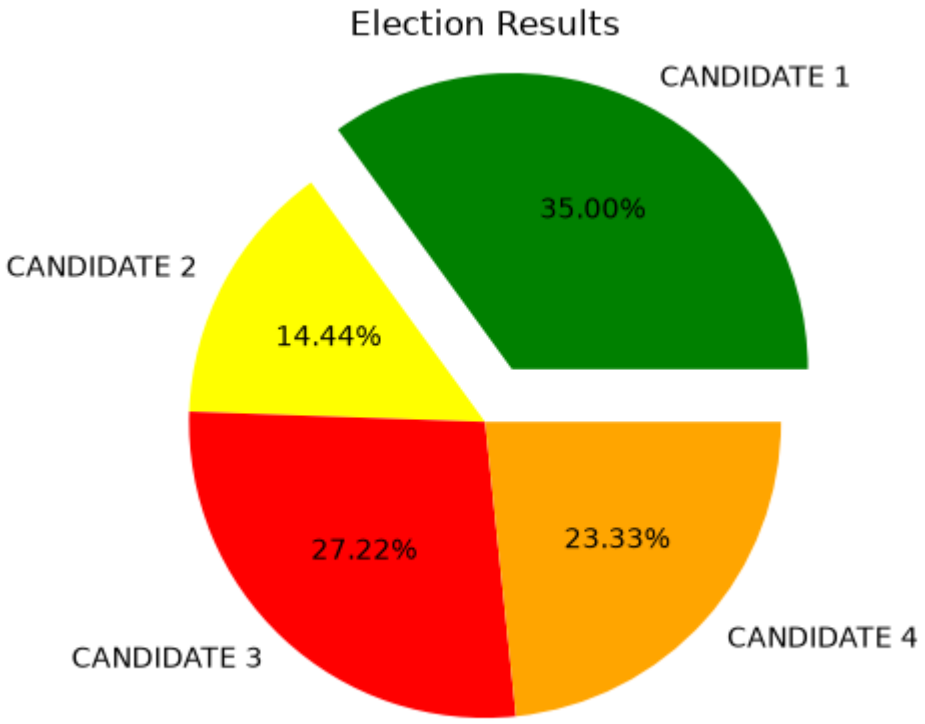
colors = ['green', 'yellow', 'red', 'orange']

explode = (0.2, 0, 0, 0)

election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct="%0.2

election.title("Election Results")

election.show()
```


Election Results
CANDIDATE 1
35.00%
CANDIDATE 2
14.44%
27.22%
23.33%
CANDIDATE 3
CANDIDATE 4

In [ ]:

In [6]:
```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg

sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)

wlist = []

for i in range(50):
    wlist.append(token[i])

wordfreq = [(wlist.count(w), w) for w in wlist]

print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

```
Pairs
[('[', (1, '[')), ('Emma', (2, 'Emma')), ('by', (1, 'by')), ('Jane', (1, 'Jan
e')), ('Austen', (1, 'Austen')), ('1816', (1, '1816')), (']', (1, ']')), ('VOLUM
E', (1, 'VOLUME')), ('I', (2, 'I')), ('CHAPTER', (1, 'CHAPTER')), ('I', (2,
'I')), ('Emma', (2, 'Emma')), ('Woodhouse', (1, 'Woodhouse')), (',', (5, ',')),
('handsome', (1, 'handsome')), (',', (5, ',')), ('clever', (1, 'clever')), (',',
(5, ',')), ('and', (3, 'and')), ('rich', (1, 'rich')), (',', (5, ',')), ('with',
(2, 'with')), ('a', (1, 'a')), ('comfortable', (1, 'comfortable')), ('home', (1,
'home')), ('and', (3, 'and')), ('happy', (1, 'happy')), ('disposition', (1, 'disp
osition')), (',', (5, ',')), ('seemed', (1, 'seemed')), ('to', (1, 'to')), ('unit
e', (1, 'unite')), ('some', (1, 'some')), ('of', (2, 'of')), ('the', (2, 'the')),
('best', (1, 'best')), ('blessings', (1, 'blessings')), ('of', (2, 'of')), ('exis
tence', (1, 'existence')), (';', (1, ';')), ('and', (3, 'and')), ('had', (1, 'ha
d')), ('lived', (1, 'lived')), ('nearly', (1, 'nearly')), ('twenty-one', (1, 'twe
nty-one')), ('years', (1, 'years')), ('in', (1, 'in')), ('the', (2, 'the')), ('wo
rld', (1, 'world')), ('with', (2, 'with'))]
```

In [ ]:

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-

columns = [
    "Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
    "Insulin", "BMI", "DiabetesPedigreeFunction", "Age", "Outcome"
]

diabetes_df = pd.read_csv(url, names=columns)

print(diabetes_df.head())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

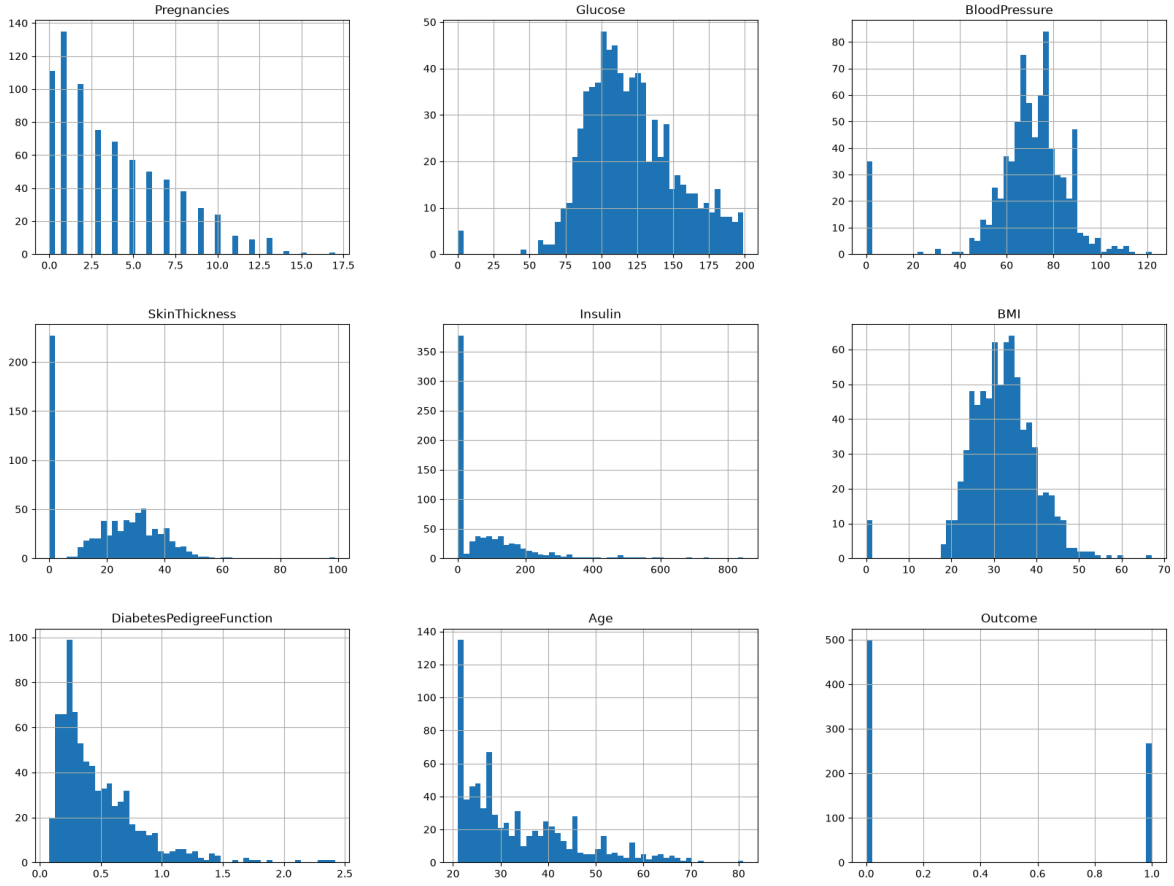
   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

In [3]:
```
print(diabetes_df.info())
print(diabetes_df.describe())
```

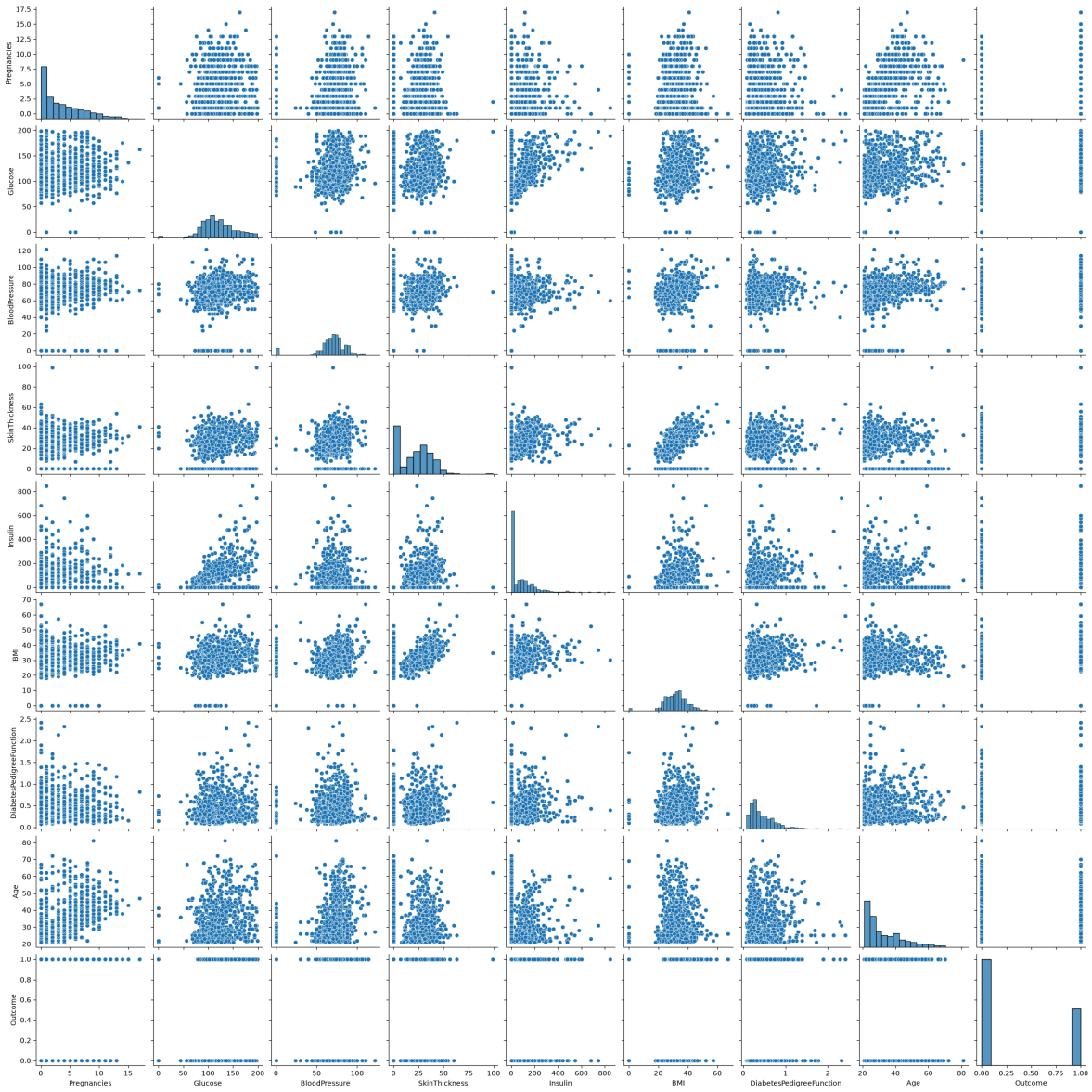
```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Pregnancies               768 non-null    int64  
 1   Glucose                   768 non-null    int64  
 2   BloodPressure             768 non-null    int64  
 3   SkinThickness             768 non-null    int64  
 4   Insulin                   768 non-null    int64  
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64  
 8   Outcome                   768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000   
mean      3.845052  120.894531      69.105469      20.536458   79.799479   
std       3.369578   31.972618      19.355807      15.952218  115.244002   
min       0.000000    0.000000       0.000000       0.000000    0.000000   
25%       1.000000   99.000000      62.000000       0.000000    0.000000   
50%       3.000000  117.000000      72.000000      23.000000   30.500000   
75%       6.000000  140.250000      80.000000      32.000000  127.250000   
max      17.000000  199.000000     122.000000      99.000000  846.000000   

              BMI  DiabetesPedigreeFunction         Age     Outcome  
count  768.000000                768.000000  768.000000  768.000000  
mean    31.992578                  0.471876   33.240885    0.348958  
std      7.884160                  0.331329   11.760232    0.476951  
min      0.000000                  0.078000   21.000000    0.000000  
25%     27.300000                  0.243750   24.000000    0.000000  
50%     32.000000                  0.372500   29.000000    0.000000  
75%     36.600000                  0.626250   41.000000    1.000000  
max     67.100000                  2.420000   81.000000    1.000000  
```

In [4]:
```
diabetes_df.hist(bins=50, figsize=(20, 15))
plt.show()
```



In [5]:
```
sns.pairplot(diabetes_df)
plt.show()
```


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome

In [ ]:

In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

np.random.seed(42)

dates = pd.date_range(start='2025-01-01', periods=180, freq='D')
x = np.arange(180)

data = pd.DataFrame({
    'Date': dates,
    'Page_Views': 200 + x * 2 + 50 * np.sin(2 * np.pi * x / 30) + np.random.norm
    'Unique_Visitors': 100 + x + 20 * np.sin(2 * np.pi * x / 30) + np.random.nor
    'Bounce_Rate': 40 + 5 * np.sin(2 * np.pi * x / 7) + np.random.normal(0, 2, 1
})

data.set_index('Date', inplace=True)
data = data.interpolate(method='linear')

print(data.head())

decomposition = seasonal_decompose(data['Page_Views'], model='additive', period=
decomposition.plot()
plt.show()

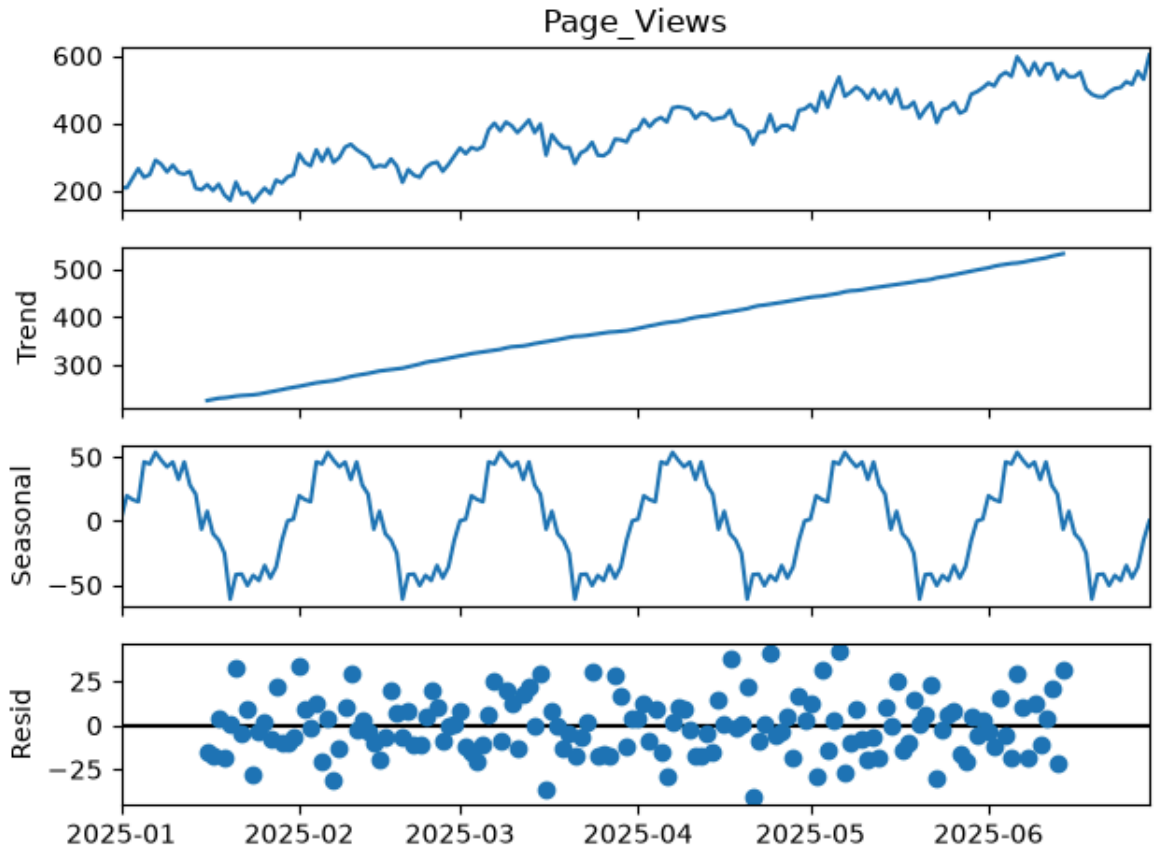
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot(figsize=(12, 5))
plt.title('Page Views with Moving Average')
plt.show()

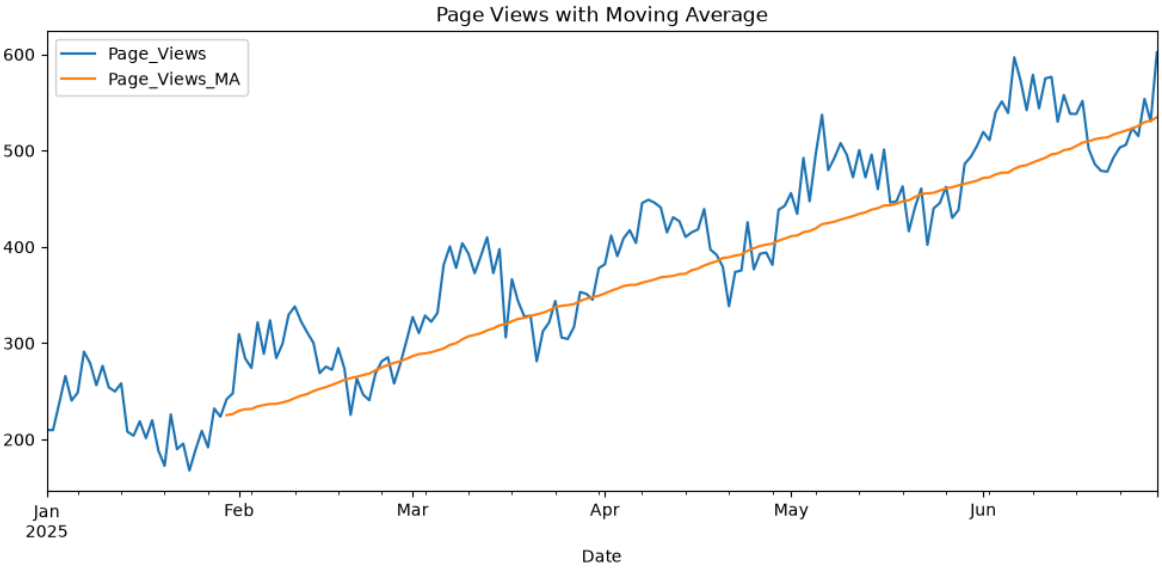
data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) / data['
anomalies = data[data['Page_Views_Z'].abs() > 3]

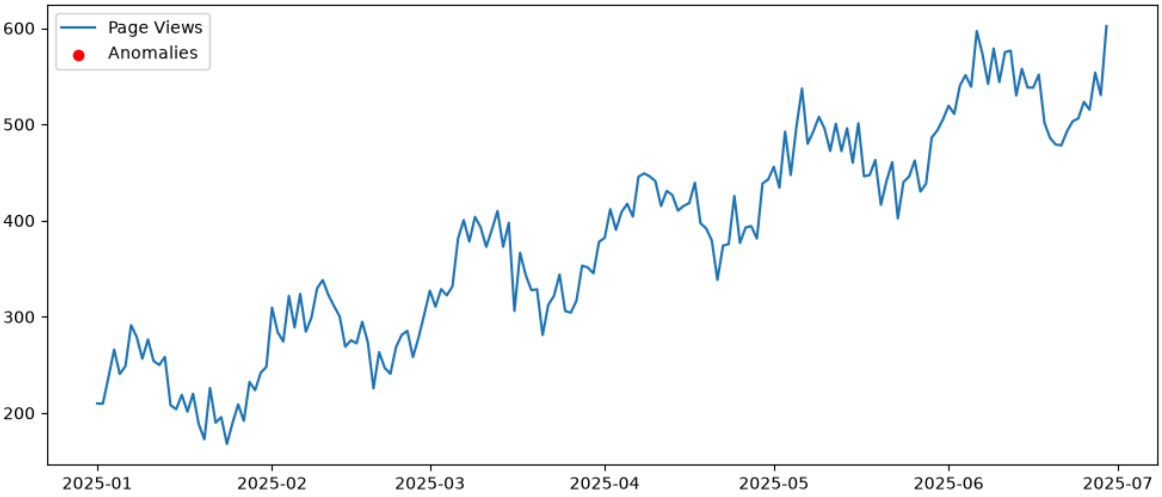
plt.figure(figsize=(12, 5))
plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomal
plt.legend()
plt.show()

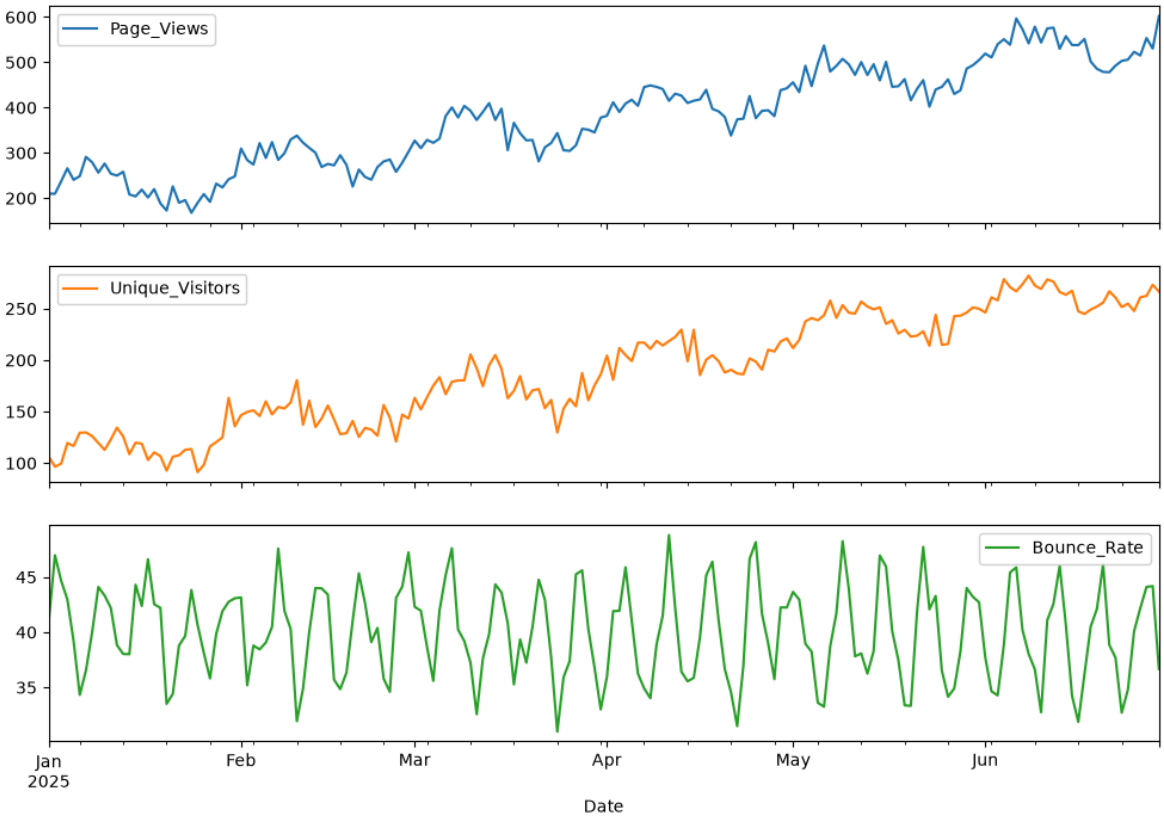
data[['Page_Views', 'Unique_Visitors', 'Bounce_Rate']].plot(subplots=True, figsi
plt.show()
```

```
            Page_Views  Unique_Visitors  Bounce_Rate
Date
2025-01-01  209.934283       106.256673    41.038693
2025-01-02  209.630299        96.586658    46.974635
2025-01-03  237.290603        99.425808    44.657119
2025-01-04  265.849860       119.580429    42.972842
2025-01-05  240.474174       116.628269    39.210869
```


Page_Views
600
400
200
Trend
500
400
300
Seasonal
50
0
−50
Resid
25
0
−25
2025-01
2025-02
2025-03
2025-04
2025-05
2025-06


Page Views with Moving Average
Page_Views
Page_Views_MA
600
500
400
300
200
Jan
2025
Feb
Mar
Apr
May
Jun
Date


Page Views
Anomalies
600
500
400
300
200
2025-01
2025-02
2025-03
2025-04
2025-05
2025-06
2025-07


Page_Views
600
500
400
300
200
Unique_Visitors
250
200
150
100
Bounce_Rate
45
40
35
Jan
2025
Feb
Mar
Apr
May
Jun
Date

In [ ]:

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

population_mean = 50
population_std = 10
population_size = 100000

population = np.random.normal(population_mean, population_std, population_size)

sample_sizes = [30, 50, 100]
num_samples = 1000

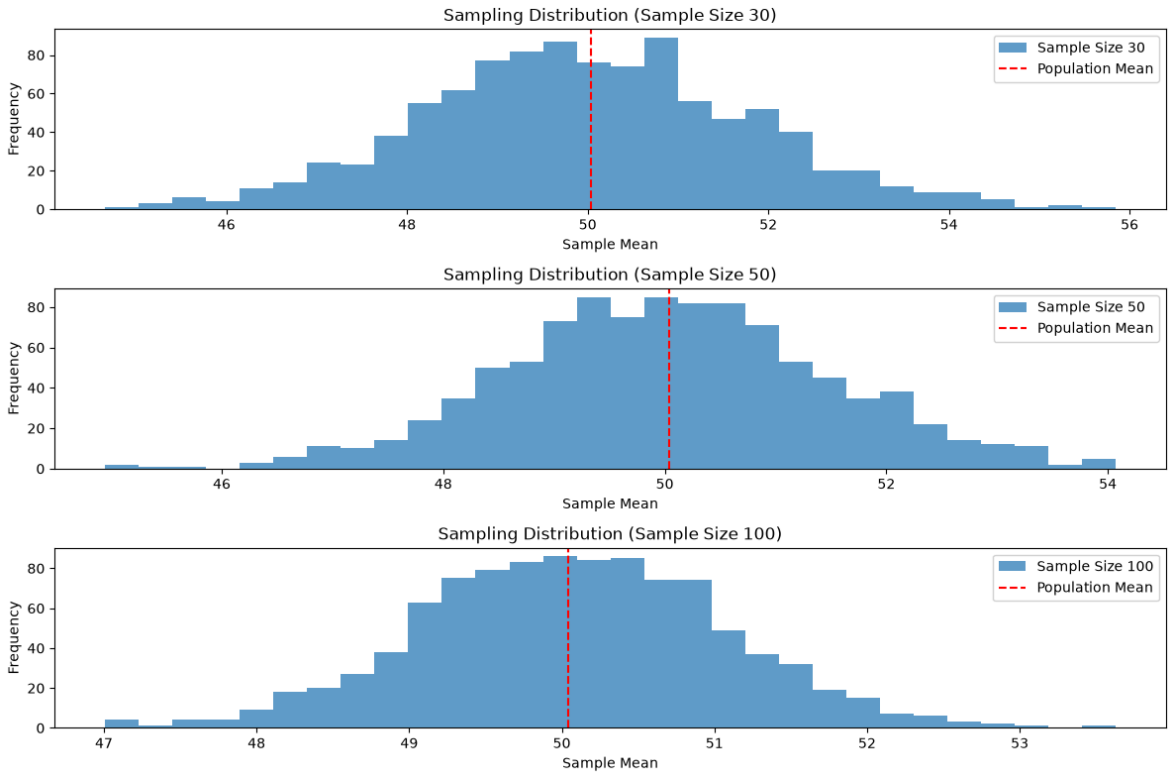
sample_means = {}

for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f"Sample Size {size}"
    plt.axvline(np.mean(population), color='red', linestyle='dashed', linewidth=
    plt.title(f"Sampling Distribution (Sample Size {size})")
    plt.xlabel("Sample Mean")
    plt.ylabel("Frequency")
    plt.legend()

plt.tight_layout()
plt.show()
```



In [ ]:

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris()

data = pd.DataFrame(iris.data, columns=[
    'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm'
])

data['Id'] = range(1, 151)
data['variety'] = ['Iris-' + iris.target_names[i] for i in iris.target]

data = data[['Id', 'SepalLengthCm', 'SepalWidthCm',
             'PetalLengthCm', 'PetalWidthCm', 'variety']]

data

data.info()

data.describe()

print(data.value_counts('variety'))

sns.countplot(x='variety', data=data)
plt.show()

dummies = pd.get_dummies(data.variety)

FinalDataset = pd.concat(
    [pd.get_dummies(data.variety), data.iloc[:, [0, 1, 2, 3]]],
    axis=1
)

print(FinalDataset.head())

sns.scatterplot(x='SepalLengthCm', y='SepalWidthCm',
                hue='variety', data=data)
plt.show()

sns.scatterplot(x='PetalLengthCm', y='PetalWidthCm',
                hue='variety', data=data)
plt.show()

sns.pairplot(data, hue='variety', height=3)
plt.show()

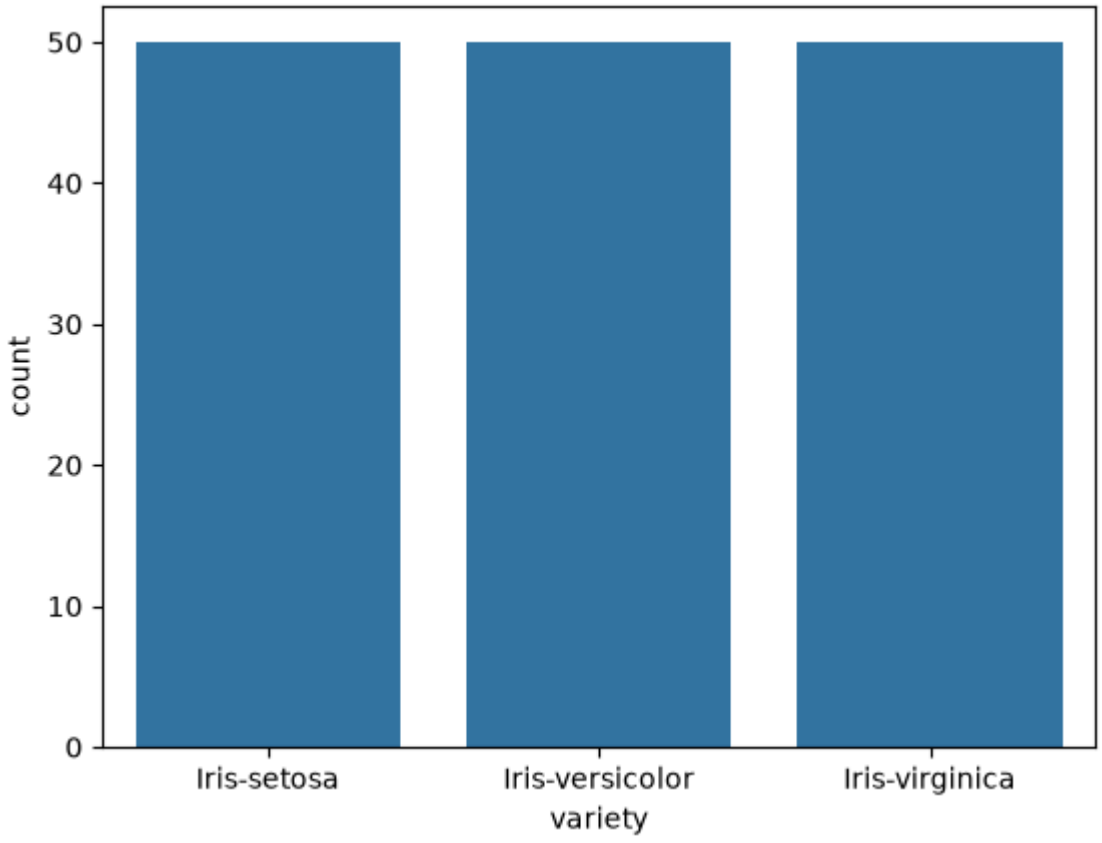
sns.FacetGrid(data, hue='variety', height=3).map(
    sns.histplot, 'PetalLengthCm'
).add_legend()
plt.show()

sns.FacetGrid(data, hue='variety', height=3).map(
    sns.histplot, 'PetalWidthCm'
).add_legend()
plt.show()
```

```
sns.FacetGrid(data, hue='variety', height=3).map(
    sns.histplot, 'SepalLengthCm'
).add_legend()
plt.show()

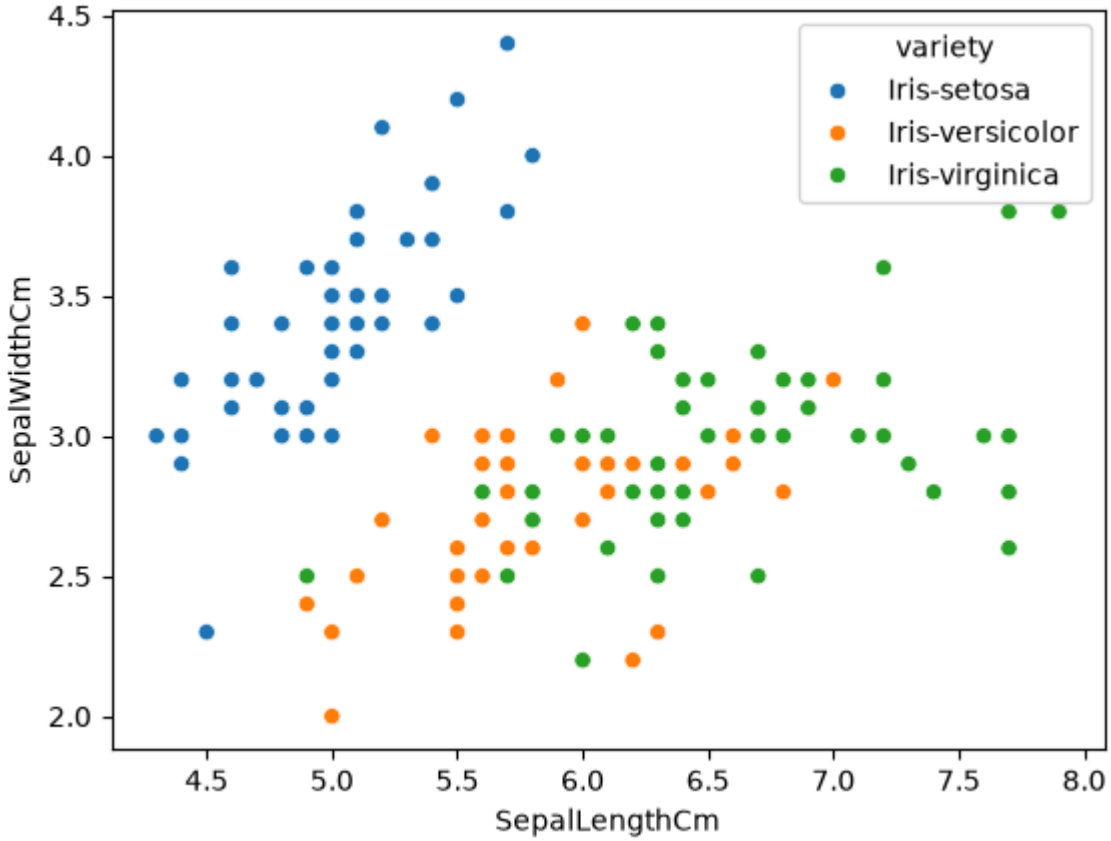
sns.FacetGrid(data, hue='variety', height=3).map(
    sns.histplot, 'SepalWidthCm'
).add_legend()
plt.show()
```

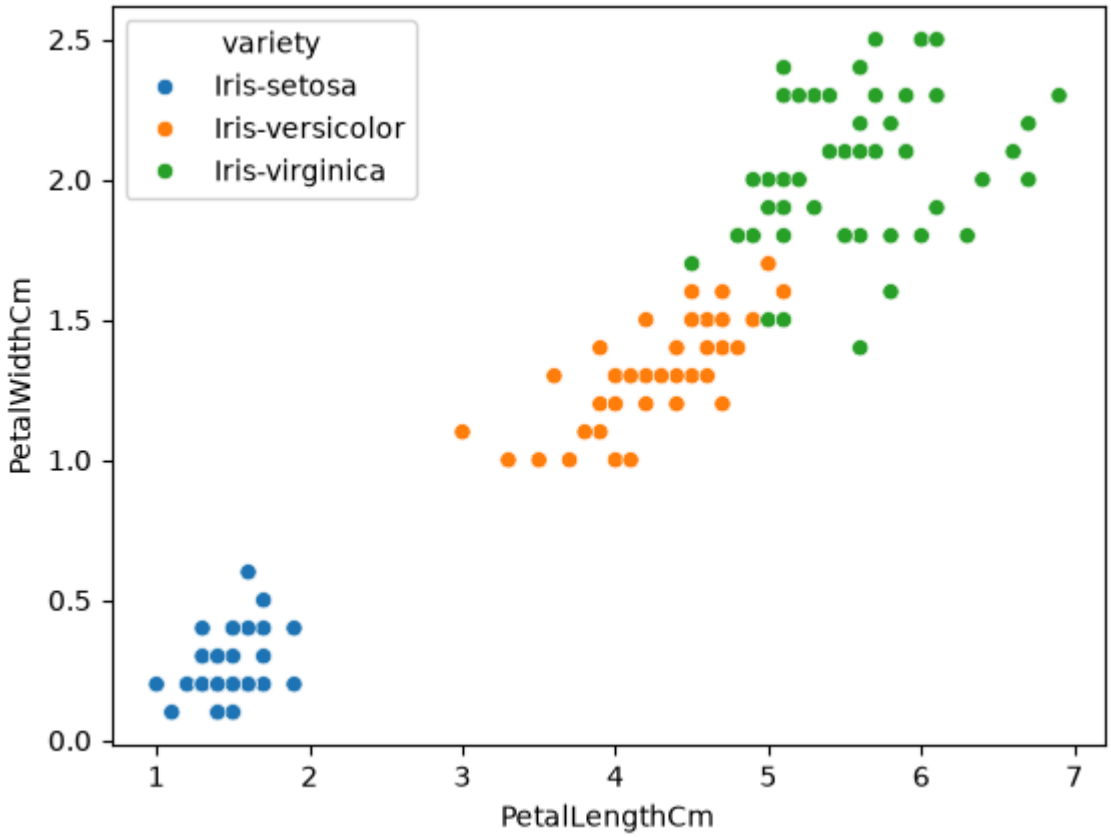
```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    str
dtypes: float64(4), int64(1), str(1)
memory usage: 9.1 KB
variety
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

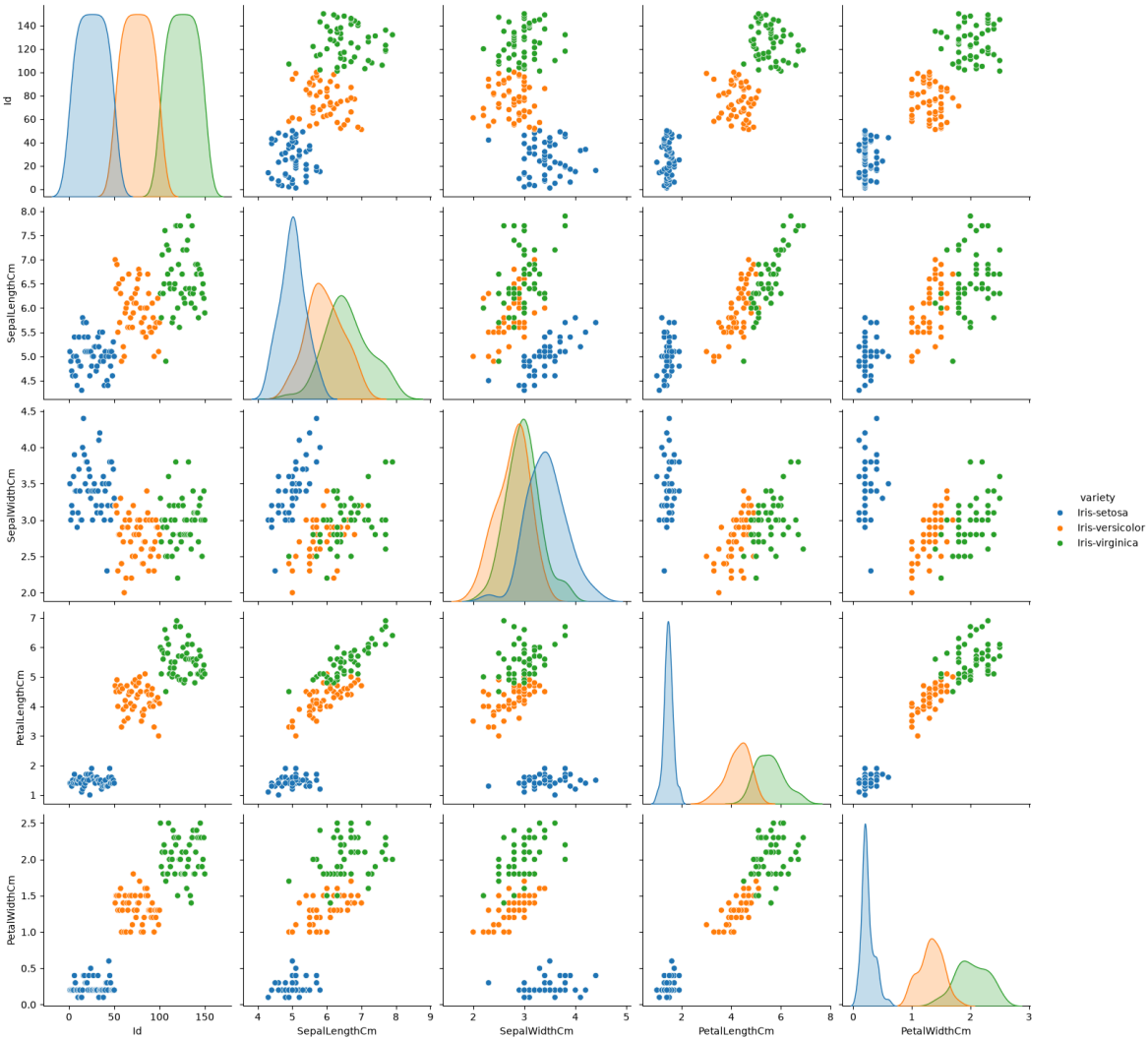


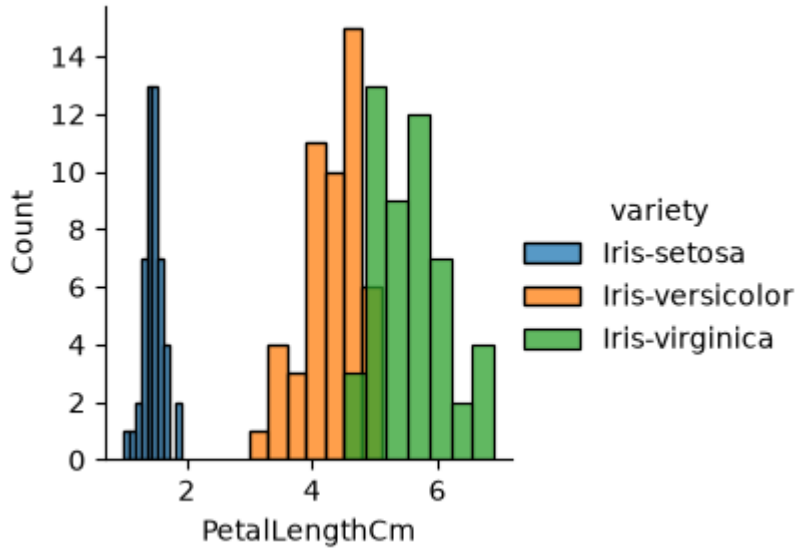
```
   Iris-setosa  Iris-versicolor  Iris-virginica  Id  SepalLengthCm  \
0         True            False           False   1            5.1
1         True            False           False   2            4.9
2         True            False           False   3            4.7
3         True            False           False   4            4.6
4         True            False           False   5            5.0

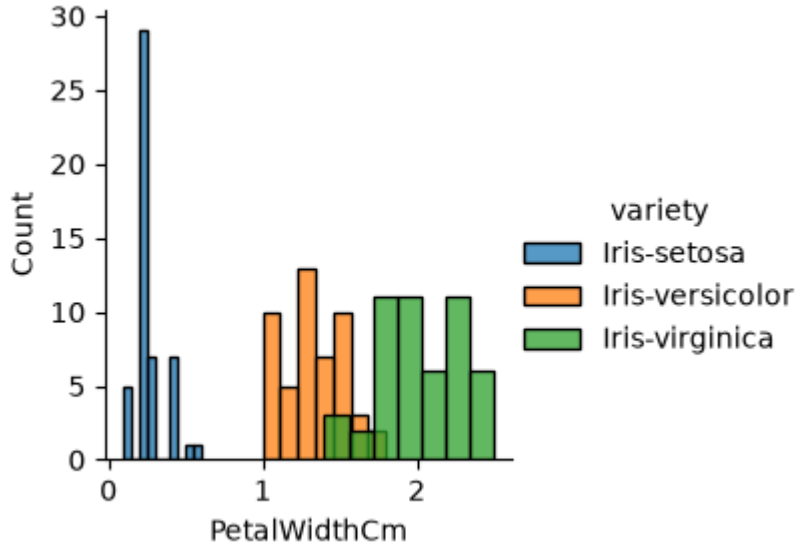
   SepalWidthCm  PetalLengthCm
0           3.5            1.4
1           3.0            1.4
2           3.2            1.3
3           3.1            1.5
4           3.6            1.4
```

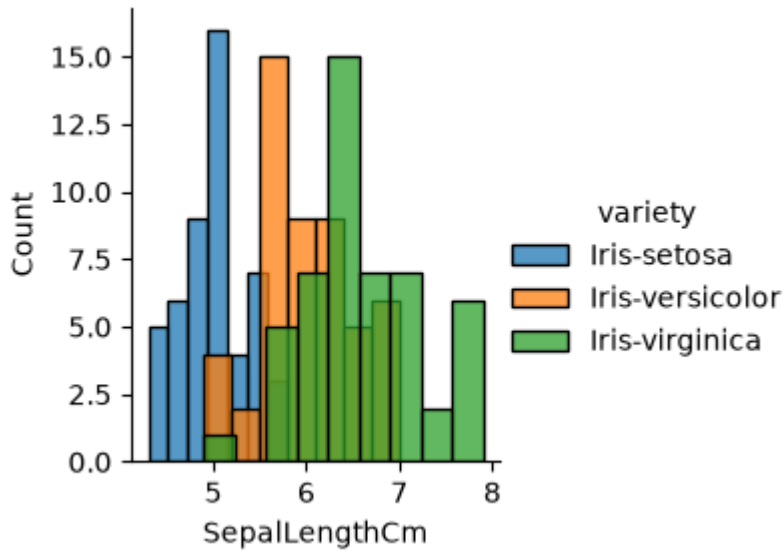


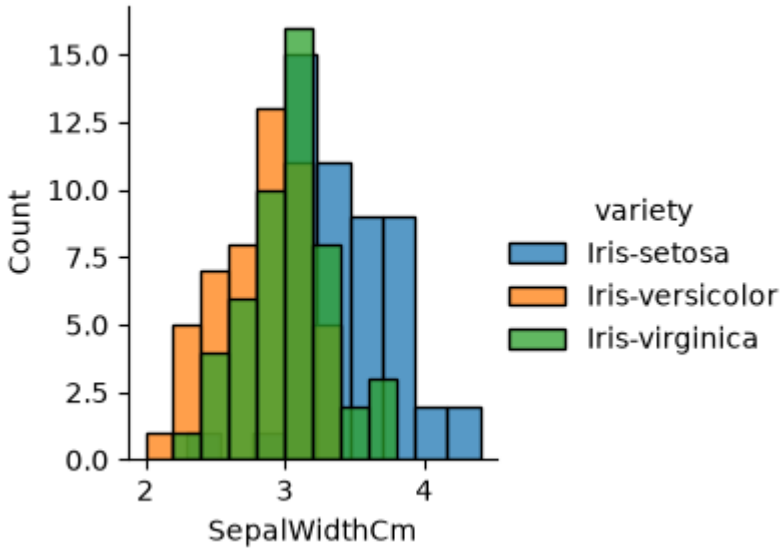

variety
Iris-setosa
Iris-versicolor
Iris-virginica
PetalWidthCm
PetalLengthCm


Id
SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
SepalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


15.0
12.5
10.0
7.5
5.0
2.5
0.0
Count
2
3
4
SepalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica

In [ ]:

In [2]:
```
import numpy as np

array = np.array([83, 25, 19, 47, 62, 15, 96, 39, 51])
array

np.sqrt(array)

array.ndim

new_array = array.reshape(3, 3)
new_array

new_array.ndim

new_array.ravel()

newm = new_array.reshape(3, 3)
newm

newm[2, 1:3]

newm[1:2, 1:3]

new_array[0:3, 0:0]

new_array[0:2, 0:1]

new_array[0:3, 0:1]

new_array[1:3]
```

Out[2]:
```
array([[47, 62, 15],
       [96, 39, 51]])
```

In [ ]:

In [2]:
```
import numpy as np
import pandas as pd

list1 = [[1, 'Smith', 50000], [2, 'Jones', 60000]]

df = pd.DataFrame(list1)
df.columns = ['EmpId', 'Name', 'Salary']
print(df)

df.info()

data = {
    'R&D Spend': [165349.20, 162597.70, 153441.51, 144372.41, 142107.34],
    'Administration': [136897.80, 151377.59, 101145.55, 118671.85, 91391.77],
    'Marketing Spend': [471784.10, 443898.53, 407934.54, 383199.62, 366168.42],
    'State': ['New York', 'California', 'Florida', 'New York', 'Florida'],
    'Profit': [192261.83, 191792.06, 191050.39, 182901.99, 166187.94]
}

df = pd.DataFrame(data)
df.info()
print(df.head())
print(df.tail())

employee_data = {
    'emp id': [1, 2, 3, 4, 5, 6, 7],
    'name': [
        'SREE VARSSINI K S',
        'SREEMATHI B',
        'SREYA G',
        'SREYASKARI MULLAPUDI',
        'SRI AKASH U G',
        'SRI HARSHAVARDHAN R',
        'GIRI HARSHAVARDHAN R'
    ],
    'salary': [5000, 6000, 7000, 5000, 8000, 3000, 6000]
}

df = pd.DataFrame(employee_data)

print(df.head())
print(df.tail())
df.info()

print(df.salary)
print(type(df.salary))
print(df.salary.mean())
print(df.salary.median())
print(df.salary.mode())
print(df.salary.var())
print(df.salary.std())

print(df.describe())
print(df.describe(include='all'))

empCol = df.columns
print(empCol)
```

```
emparray = df.values
print(emparray)

employee_DF = pd.DataFrame(emparray, columns=empCol)
print(employee_DF)
```

```
   EmpId   Name  Salary
0      1  Smith   50000
1      2  Jones   60000
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   EmpId   2 non-null      int64
 1   Name    2 non-null      str
 2   Salary  2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 190.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        5 non-null      float64
 1   Administration   5 non-null      float64
 2   Marketing Spend  5 non-null      float64
 3   State            5 non-null      str
 4   Profit           5 non-null      float64
dtypes: float64(4), str(1)
memory usage: 372.0 bytes
   R&D Spend  Administration  Marketing Spend       State     Profit
0  165349.20       136897.80        471784.10    New York  192261.83
1  162597.70       151377.59        443898.53  California  191792.06
2  153441.51       101145.55        407934.54     Florida  191050.39
3  144372.41       118671.85        383199.62    New York  182901.99
4  142107.34        91391.77        366168.42     Florida  166187.94
   R&D Spend  Administration  Marketing Spend       State     Profit
0  165349.20       136897.80        471784.10    New York  192261.83
1  162597.70       151377.59        443898.53  California  191792.06
2  153441.51       101145.55        407934.54     Florida  191050.39
3  144372.41       118671.85        383199.62    New York  182901.99
4  142107.34        91391.77        366168.42     Florida  166187.94
   emp id                 name  salary
0       1     SREE VARSSINI K S    5000
1       2           SREEMATHI B    6000
2       3               SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000
   emp id                 name  salary
2       3               SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000
5       6   SRI HARSHAVARDHAN R    3000
6       7  GIRI HARSHAVARDHAN R    6000
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      str
 2   salary  7 non-null      int64
dtypes: int64(2), str(1)
memory usage: 407.0 bytes
0    5000
```

```
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64
<class 'pandas.Series'>
5714.285714285715
6000.0
0    5000
1    6000
Name: salary, dtype: int64
2571428.5714285714
1603.5674514745463
         emp id       salary
count  7.000000     7.000000
mean   4.000000  5714.285714
std    2.160247  1603.567451
min    1.000000  3000.000000
25%    2.500000  5000.000000
50%    4.000000  6000.000000
75%    5.500000  6500.000000
max    7.000000  8000.000000
          emp id               name       salary
count   7.000000                  7     7.000000
unique       NaN                  7          NaN
top          NaN  SREE VARSSINI K S          NaN
freq         NaN                  1          NaN
mean    4.000000                NaN  5714.285714
std     2.160247                NaN  1603.567451
min     1.000000                NaN  3000.000000
25%     2.500000                NaN  5000.000000
50%     4.000000                NaN  6000.000000
75%     5.500000                NaN  6500.000000
max     7.000000                NaN  8000.000000
Index(['emp id', 'name', 'salary'], dtype='str')
[[1 'SREE VARSSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
 [5 'SRI AKASH U G' 8000]
 [6 'SRI HARSHAVARDHAN R' 3000]
 [7 'GIRI HARSHAVARDHAN R' 6000]]
  emp id                  name salary
0      1     SREE VARSSINI K S   5000
1      2           SREEMATHI B   6000
2      3               SREYA G   7000
3      4  SREYASKARI MULLAPUDI   5000
4      5         SRI AKASH U G   8000
5      6   SRI HARSHAVARDHAN R   3000
6      7  GIRI HARSHAVARDHAN R   6000
```

In [ ]: